AHAB: Asynchronous, High-throughput, Adaptively-secure, Batched Threshold Schnorr Signatures

Victor Shoup
Category Labs
victor@shoup.net

March 30, 2026

Abstract

We present AHAB, a suite of protocols for threshold Schnorr signatures in the asynchronous communication setting with guaranteed output delivery (robustness). We build on the AVSS and GoAVSS protocols of Shoup–Smart and Groth–Shoup, which allow $t < n/3$ static corruptions. First, we provide protocol enhancements and a full security proof in the adaptive corruption model with erasures. Second, we introduce a signature production pipeline with a player elimination framework that bounds the damage from actively misbehaving parties: if t^* corrupt parties disrupt a presignature batch, the total communication overhead is at most $O(t^*)$ times the happy-path cost, and all t^* parties are identified and eliminated, after which the system runs at the happy-path rate until further active misbehavior occurs. Third, we present a simplified protocol variant for $t < n/4$ adaptive corruptions that achieves worst-case linear communication complexity by eliminating the complaint mechanism entirely and using star-finding at the pipeline level to agree on which dealers and receivers to use. Fourth, we present a hiding variant of GoAVSS that yields an unbiased distributed key generation protocol, preventing an adaptive adversary from biasing the signing key. We also give new and more efficient star-finding algorithms, including an ILP-based optimization that should improve the yield of presignatures per batch in practice. For the main protocol ($t < n/3$), with $t = 16$ and $n = 49$, on a 1 Gbps network with commodity hardware, we estimate a throughput of 100K signatures per second; for the simplified protocol ($t < n/4$), with $t = 16$ and $n = 65$, the estimate is 160–250K signatures per second.

Contents

1 Introduction

Our target application is a threshold signing service operating as part of a blockchain or similar decentralized system. Such a service must operate autonomously and robustly: it must continue producing signatures regardless of adversarial behavior, without requiring manual intervention to recover from failures or to remove misbehaving parties. This rules out protocols that merely provide security with abort or fairness, and demands *guaranteed output delivery*: every signing request must eventually produce a valid signature.

We present new protocols for threshold Schnorr signatures in the asynchronous communication setting, which we collectively call **AHAB** (**A**synchronous, **H**igh-throughput, **A**daptively-secure, **B**atched threshold Schnorr signatures). Building on the AVSS protocol of Shoup and Smart [SS23] and the GoAVSS protocol and threshold Schnorr signing protocol of Groth and Shoup [GS23], our protocols extend and significantly improve upon that work in several directions. Like [GS23], our protocols provide *robustness* (guaranteed output delivery) with optimal resilience, allowing up to $t < n/3$ corrupt parties.

Adaptive security (with erasures). Whereas the analysis in [SS23] and [GS23] assumes static corruptions, our analysis is carried out in the *adaptive corruption* model. To achieve this, we make essential use of **sender-forward-secure channels**: when a dealer transmits secret shares to a receiver, it encrypts them (using standard public-key or hybrid encryption) and then erases the plaintext and encryption randomness. This ensures that if the dealer is subsequently corrupted, the adversary cannot recover the transmitted shares.

The only setup assumption we need is sender-forward-secure channels for sending some messages, as well as ordinary authenticated and/or secure channels for sending others. We can reduce the security of our threshold Schnorr signature protocol to that of ordinary, non-threshold Schnorr signatures under an enhanced attack mode, specifically, the attack mode described in Section 4.1 of [Sho23a] (which models batch randomness extraction with re-randomized presignatures) and analyzed in Sections 4.2 and 4.3 of that paper. This reduction models certain hash functions as random oracles (for some hash functions, we can alternatively get by with specific, concrete intractability assumptions), and otherwise makes use of no other intractability assumptions. Given the results in Section 4 of [Sho23a], one can then either (i) further reduce the security of our scheme in the random oracle model to that of the interactive Schnorr identification scheme (which then reduces by a standard argument to the discrete logarithm assumption), or (ii) prove security directly in the generic group model (either with concrete assumption on a hash function or in the random oracle model).

Offline/online batching. Just as in [GS23], our protocol has an offline/online structure. The offline component generates batches of **presignatures** (collections of message-independent precomputed values, each consisting of a shared random scalar r and an associated public group element $r\mathcal{G}$) and runs concurrently with the online component. The online component uses previously generated presignatures to process signing requests. Signing requests are also processed in batches, but these signature batches are generally smaller than presignature batches — in periods of low demand, a signature batch may consist of just a single signing request, while in periods of high demand larger batches may be used to facilitate higher throughput.

This offline/online structure allows us to hide the latency associated with presignature produc-

tion, keeping the online latency to a minimum (which is just 1–3 rounds of communication, depending on implementation details). However, batching in both the offline and online components allows us to sustain much higher throughput as well, as batching allows us to employ techniques that dramatically reduce the amortized communication and computational complexity of the protocol. To achieve this better amortized complexity, when generating a presignature batch, each party needs to share a batch of $\gg n \log n$ secrets, which are then combined via a batch randomness extraction process to yield a batch of $\gg n^2 \log n$ presignatures; similarly, signature batches need to be of size $\gg n$.

Bounded-damage player elimination. The GoAVSS protocol of [GS23] (building on the AVSS protocol of [SS23]) achieves linear amortized communication complexity per signature on a "happy path" where all parties behave correctly. When a corrupt dealer provably misbehaves, a complaint mechanism allows affected parties to reconstruct their shares, but this increases the communication cost from linear to quadratic per secret for that dealing. Although the corrupt dealer is eventually identified and eliminated, there is no a priori bound on *when* this elimination occurs or how many signing requests are processed with degraded performance.

In this paper, we introduce a **signature production pipeline** that is tightly integrated with a **player elimination framework**. The key property of our design is this: let C denote the total communication complexity on the happy path for producing one presignature batch and all of the signatures derived from it. If t^* corrupt parties actively misbehave during this process, the total communication complexity may increase to $C + O(t^*C)$, but all t^* misbehaving parties will be identified and eliminated by the time that presignature batch has been consumed. After elimination, the system runs at the happy-path rate until further active misbehavior occurs. The mechanism is based on **support-gated complaints**: expensive complaint processing is triggered only when a complaint receives sufficient support, ensuring both liveness (legitimate complaints are always processed, as necessary, to produce signatures) and accountability (every processed complaint leads to the identification and elimination of a corrupt party).

A simplified protocol for $t < n/4$. We also present a simplified variant of the protocol that achieves *worst-case* linear communication complexity, at the cost of a weaker resilience threshold: $t < n/4$ rather than $t < n/3$. In this variant, we forgo the completeness property of the AVSS protocol entirely: some parties may not receive valid shares from a given dealer, and there is no complaint mechanism to remedy this. Instead, we use a **star-finding algorithm** [Can96] at the pipeline level to agree on which dealers and receivers to use for a given presignature batch. This removes the need for the SMD (Secure Message Distribution) subprotocol and all complaint-related machinery, yielding a simpler protocol overall that also *improves* the amortized communication cost per secret compared to [SS23], since the elimination of the SMD subprotocol removes the associated erasure coding overhead.

The use of a weaker corruption bound is well motivated in practice: for example, consensus protocols with a "fast path" (such as Kudzu [SSV25]) assume a smaller corruption bound (typically, $t < n/5$). Such protocols can naturally be paired with our $t < n/4$ signing protocol, since a system that already assumes $t < n/4$ for its consensus layer can use the same assumption for threshold signing.

Concrete performance estimates. We provide detailed performance estimates throughout.

Consider the main protocol (with $t < n/3$) on the happy path. The amortized communication complexity per signature per party (across both the offline and online components) is a constant number of group elements and scalars. Assume the underlying group is the elliptic curve secp256k1. Then we estimate that each party transmits a total of well under 10K bits per signature across the network — that is, 10K bits in total, not 10K bits sent to each party (this assumes the batch sizes are sufficiently large, as noted above).

If the group size is λ bits, the amortized computational complexity per signature per party is $O(\lambda/n + n)$ group *additions* plus other, generally less expensive, operations, like scalar arithmetic, hashing, pseudorandom bit generation, and erasure coding/decoding. For example, using the libsecp256k1 library[1] for group arithmetic, and the reed-solomon-simd library[2] for erasure coding, with $t = 16$ and $n = 49$, we estimate the amortized compute time per signature on an Apple MacBook Pro (Mac17,2, 10-core Apple M5, 24 GB RAM) to be well under 5 µs — about 50% of which is spent on group additions, and about 15% of which is spent on erasure coding/decoding. For context, the time for one group addition is 70 ns–90 ns, so that 5 µs time is the equivalent of 56–71 group additions.

Based on these benchmarks, we can project the protocol's overall throughput. At a communication cost of 10K bits per signature, a standard 1 Gbps network link imposes a theoretical upper bound of roughly 100K signatures per second. Concurrently, the 5 µs compute time yields a raw computational capacity exceeding 200K signatures per second. Therefore, assuming an optimized implementation that effectively pipelines these operations to fully overlap CPU execution with network communication, the system's maximum throughput is strictly network-bound, approaching 100K signatures per second.

These performance results are somewhat better than in [GS23], as we are using faster hardware in our benchmarks. The main takeaway is that the improvements introduced here (adaptive security, bounded-damage player elimination) do not reduce performance.

For the simplified protocol ($t < n/4$), the communication complexity improves. With the same assumptions as above, we estimate that each party transmits a total of at most 4K to 6K bits per signature across the network. The computational complexity, however, is slightly higher. For $t = 16$ and $n = 65$, we estimate the compute time per signature per party to be at most double that stated above. A couple of CPU cores should suffice to achieve a throughput of between 160K and 250K signatures per second.

Star-finding algorithms. In the context of the $t < n/4$ protocol, we revisit star-finding algorithms, which were introduced by Canetti [Can96] in the context of AVSS protocols with $t < n/4$. We give a new (and perhaps more straightforward) proof of correctness of Canetti's algorithm, and present a more efficient version that runs in time $O(n^2)$ rather than $O(n^{2.5})$ — indeed, the algorithm requires time $O(n^2)$ to form the complement of an n-node graph, but otherwise runs in time linear in the size of the complement graph. We also present an ILP-based optimization that can find larger dealer and receiver sets in practice, directly improving the yield of presignatures per batch and hence the amortized communication cost per signature.

Unbiased distributed key generation. We also present a hiding variant of GoAVSS in which the group elements associated with the shared secrets remain hidden until a set of dealers has been

[1] https://github.com/bitcoin-core/secp256k1

[2] https://github.com/AndersTrier/reed-solomon-simd

irrevocably agreed upon. This prevents an adaptive adversary from biasing the derived keys by choosing its own inputs after seeing honest dealers' contributions, and yields an unbiased DKG protocol suitable for establishing the initial signing key.

Summary of contributions. To clarify the relationship with [GS23]: for the $t < n/3$ case, we build on the GoAVSS protocol of [GS23], with two major additions — protocol enhancements and a security proof for the adaptive corruption model (using sender-forward-secure channels with erasures), and a player elimination framework integrated with a signature production pipeline. The $t < n/4$ protocol, the associated star-finding algorithms and ILP optimization, and the unbiased DKG protocol are entirely new. While our presentation focuses on threshold Schnorr signatures, the techniques developed here — particularly the player elimination framework and the simplified $t < n/4$ protocol — should be applicable more broadly to settings that use AVSS as a building block, such as asynchronous MPC.

The case for robustness. Threshold public-key cryptography — including threshold signing, decryption, and more general multi-party computation — is increasingly deployed in operational systems, from blockchain validators to certificate authorities. In such settings, the system must continue to function even when some parties crash, lose key material, or behave maliciously. Protocols that provide only *security with abort* offer no such guarantee: a single unresponsive or malicious party can prevent the system from producing output, causing a liveness failure that may require costly manual intervention to resolve.

A vivid illustration of this risk occurred in 2025, when the IACR's online election system — which used 3-out-of-3 threshold decryption — was rendered permanently unable to decrypt the election results after one of the three trustees irretrievably lost their private key.[3] While the loss was due to an honest mistake rather than adversarial action, the outcome was indistinguishable: a total and irrecoverable loss of functionality. A threshold of 2-out-of-3 would have prevented this failure entirely.

In the context of threshold Schnorr signatures, FROST [KG20a] is a widely deployed protocol that provides security with abort: if any designated signer fails to respond, the signing attempt fails. ROAST [RRJ+22] addresses this by adding a robustness wrapper that retries with different signer subsets, but this adds coordination overhead. Our protocols, by contrast, build robustness in from the start: signatures are produced regardless of adversarial behavior, with no need for retry mechanisms or manual intervention.

Why asynchrony. Synchronous protocols require all parties to complete each round within a fixed time bound. In practice, this bound must be set conservatively to account for worst-case network delays, leading to high latency even when the network is fast. Worse, if the bound is set too aggressively, the protocol's security guarantees may be violated. Asynchronous protocols avoid this dilemma entirely: they make progress as fast as the network allows and remain secure regardless of message delays.

Organization. After reviewing related work (Section 1.1), we present preliminaries in Section 2. In Section 3.1, we review the SMD protocol from [SS23] and show what changes are needed to make

[3] https://www.iacr.org/news/item/27138

it work in the adaptive corruption model. Our new GoAVSS protocol is described in Section 4 and analyzed in Section 5. This GoAVSS protocol incorporates ideas from [SS23] and [GS23] to significantly reduce interaction, as well as the logic of our modified SMD protocol. The signature production pipeline and player elimination framework are presented in Section 6. The unbiased DKG protocol is presented in Section 7. The simplified $t < n/4$ protocol is presented in Section 8. Appendix A presents efficient polynomial extension algorithms based on finite differences, used for share computation during dealing and for the evaluation steps in online error correction. These algorithms may be useful in other applications related to secret sharing. Finally, the new star-finding algorithms, which may be of independent interest, are discussed in Appendix B.

1.1 Related work

The starting point for our work is the AVSS protocol of Shoup and Smart [SS23] and the GoAVSS protocol and threshold Schnorr signing protocol built on top of it by Groth and Shoup [GS23]. The AVSS protocol in [SS23] uses only lightweight cryptographic primitives (hash functions, symmetric encryption) and achieves $O(\kappa n)$ amortized communication per secret on the happy path, with $O(\kappa n^2 \log n)$ additive overhead.[4] When a corrupt dealer forces the protocol off the happy path, the per-secret cost rises to $O(\kappa n^2)$, but this identifies the corrupt dealer, who can then be eliminated. Groth and Shoup [GS23] build on this AVSS protocol to construct a GoAVSS protocol (where the dealer additionally publishes group elements associated with the shared secrets) and use it as the basis for an efficient threshold Schnorr signing protocol, achieving $O(\kappa n)$ amortized communication per signature.

Two recent works — Gossamer [XLL+25] and Bandarupalli et al. [BJJ+25] — address the gap between happy-path and unhappy-path communication complexity in [SS23]. Both build on the techniques of [SS23] and aim to achieve $O(\kappa n)$ per-secret communication even when share recovery is needed. We discuss each in turn and explain how our work relates to and differs from both.

Gossamer. Gossamer [XLL+25] achieves $O(\kappa n)$ per-secret communication on both happy and unhappy paths by introducing a bivariate polynomial structure. Secrets are embedded in (t, t)-degree bivariate polynomials, and a two-phase routing protocol (originating in the batch public reconstruction technique of Choudhury and Patra [CP17]) enables share recovery at linear cost. The key trade-off is that, at the AVSS level, the bivariate structure introduces roughly a factor of 3 increase in the happy-path amortized cost per secret compared to [SS23], and the additive overhead per AVSS instance is $O(\kappa n^3 \log n)$ (a factor of n worse than [SS23]), requiring a batch of size $\gg n^2 \log n$ to amortize. Gossamer does not incorporate any player elimination mechanism.

Our work differs from Gossamer in several respects. First, at the AVSS level, our GoAVSS protocol builds directly on [SS23], inheriting its $O(\kappa n)$ happy-path amortized cost and $O(\kappa n^2 \log n)$ additive overhead without the $3\times$ overhead of the bivariate structure, while our pipeline framework ensures that unhappy-path episodes lead to player elimination within a bounded number of signing requests. Second, our simplified protocol ($t < n/4$) achieves worst-case linear communication without bivariate polynomials, using star-finding at the pipeline level rather than bivariate share recovery within each AVSS instance.

[4] κ is the output length of a collision-resistant hash. Scalars and group elements are assumed to be $O(\kappa)$ bits in length. By *communication complexity*, we mean the total number of bits transmitted by all honest parties in aggregate across the network. By *communication complexity per party*, we mean the number of bits transmitted by any one honest party across the network.

(We note that Gossamer's comparison with [SS23] in [XLL+25] presents the unhappy-path cost $O(\kappa n^2)$ per secret as the bottom-line complexity, without adequately distinguishing it from the happy-path cost of $O(\kappa n)$ per secret. This distinction is fundamental to the design philosophy of [SS23]: the quadratic cost arises only when a corrupt dealer provably misbehaves — an event that identifies and enables removal of the corrupt party.)

Bandarupalli et al. Bandarupalli et al. have two recent papers that are relevant to our work: an AMPC protocol [BJK+24] and a DPSS protocol [BJJ+25]. Both build on the AVSS protocol of [SS23], using it in a mode they call ACSS-Id (ACSS with identifiable abort), which suppresses the unhappy path: parties who do not receive valid shares silently record the dealer as corrupt rather than complaining. A bivariate polynomial layer is added for efficient public reconstruction.

A central feature of both protocols is a **party-elimination framework** in which the reconstruction of secrets (or, in the AMPC setting, the online evaluation phase) is divided into t sequential segments. In each segment, the parties attempt to reconstruct a batch of secrets; if the attempt fails (due to corrupt parties providing bad shares), the parties run a consensus protocol to agree on the identity of a corrupt party, eliminate that party, and retry the segment.

This segmentation is a deliberate design choice: by dividing L secrets into segments of size L/t, each retry only re-exchanges shares for one segment, keeping the total worst-case communication linear in L. Without segmentation, each retry would touch all L secrets, leading to $O(n^2)$ per-secret communication — which is the same cost as the unhappy path in [SS23]. However, the trade-off is that this segmented structure imposes $O(n)$ sequential consensus steps *even in the common case* when no parties are corrupt: the protocol must process $O(n)$ segments sequentially, and each segment requires an agreement step to confirm that reconstruction succeeded. In [BJJ+25], each segment involves two instances of Byzantine agreement; in [BJK+24], a similar agreement mechanism is used per segment. While each BA instance has $O(1)$ expected rounds, the sequential composition means that the total common-case latency grows linearly with n.

Our pipeline makes a different trade-off. We do not segment: instead, we accept that the complaint path is expensive (each corrupt party may cause $O(C)$ additional communication, where C is the happy-path cost of producing a presignature batch and its signatures), but we ensure that all misbehaving parties are eliminated by the time the presignature batch is finalized. This means the system converges to the happy-path rate after at most a few presignature batches, and remains there until further active misbehavior occurs. Moreover, all consensus operations within the signing protocol are confined to presignature batch production and finalization. Signature production itself requires no consensus beyond the external mechanism (e.g., a blockchain) used to order signing requests and pair them with presignatures — which any threshold Schnorr protocol will need.

We note that [BJJ+25] focuses on DPSS (dynamic proactive secret sharing), where the goal is to transfer secrets from one committee to another, while our focus is on threshold signatures with a fixed committee. The DPSS setting introduces additional challenges (such as secret transfer between committees) that are outside the scope of this paper.

Velox. A third paper by largely the same group of authors [BJK+25] presents Velox, an asynchronous MPC protocol that achieves better concrete constants ($\approx 9.33n$ field elements per multiplication gate, $O(\kappa n^3)$ additive overhead) than [BJK+24, BJJ+25]. The key to these improvements is that Velox relaxes the security guarantee from **guaranteed output delivery** (GOD) to **fair-**

ness: the adversary may cause the computation to abort, but if it does, no party (including the adversary) learns the output. This relaxation allows Velox to avoid the party-elimination framework entirely, replacing per-segment consensus with a lightweight hash-comparison mechanism for per-layer agreement that provides only security with abort.

For threshold signing, however, fairness is insufficient: as discussed above, a signing service must provide robustness, and an adversary that can prevent signature production causes an unacceptable liveness failure. Thus, while Velox demonstrates that relaxing the security model can yield significant efficiency gains for general MPC, its techniques do not directly apply to the robust threshold signing setting that is the focus of this paper.

Summary of key differentiators. To summarize the comparison with the most closely related lines of work: achieving $O(\kappa n)$ per-secret communication on the unhappy path requires some structural cost. Gossamer pays this cost in the form of roughly a $3\times$ factor in happy-path amortized cost and an $n\times$ factor in additive overhead (due to the bivariate structure). Bandarupalli et al. pay this cost in the form of $O(n)$ sequential consensus steps during reconstruction, even in the common case with no corrupt parties (due to their segmented party-elimination framework). Our pipeline makes a different trade-off: we accept that active corruption temporarily inflates communication by a bounded multiplicative factor, but all misbehaving parties are eliminated, and the system converges to the happy-path rate. Moreover, signature production itself requires no consensus at all. All of the above works are analyzed only in the static corruption model, whereas our protocol achieves adaptive security.

Comparison of AVSS protocols. Since AVSS is the core primitive underlying all of the protocols discussed above, it is useful to compare the AVSS components directly. Table 1 presents the amortized communication cost per shared secret per party, along with other key properties, for the AVSS protocols in these works.

Table 1 Comparison of AVSS protocols.

Protocol	Scalars/secret/party	Complete	Resilience
Shoup–Smart [SS23]	≈ 6	yes	$t < n/3$
Bandarupalli et al. [BJK+24, BJJ+25]	≈ 6	no	$t < n/3$
Gossamer [XLL+25]	≈ 18	yes	$t < n/3$
Choudhury–Patra [CP17]	≈ 11	yes	$t < n/4$
This paper ($t < n/4$)	≈ 1	no	$t < n/4$

"Scalars/secret/party" is the amortized happy-path communication cost per shared secret, measured in scalar field elements per party, for large batch sizes. "Complete" indicates whether the protocol guarantees that all honest parties eventually obtain valid shares (as opposed to identifiable abort only). All protocols use computational security in the random oracle model, except Choudhury–Patra, which is perfectly secure with no cryptographic assumptions. Our $t < n/3$ protocol (not shown) uses the same AVSS as Shoup–Smart; however, we work out the details for adaptive corruptions, as was suggested but not carried out in [SS23]. Our simplified protocol ($t < n/4$) is incomplete, but this is adequate for our threshold signing application because star-finding at the pipeline level ensures that all receivers in the agreed set hold valid shares.

The AVSS protocol of Choudhury and Patra [CP17] is a natural point of comparison for our $t < n/4$ protocol: it is also a complete, linear-communication AVSS protocol at the $t < n/4$ threshold, and it achieves perfect (rather than computational) security. However, its amortized cost per secret per party is roughly 11 scalars — an order of magnitude worse than the ≈ 1 scalar achieved by our simplified protocol. The gap is partly explained by the fact that the Choudhury–Patra protocol achieves completeness (via a share completion stage based on degree-$2t$ error correction), while our protocol is incomplete. In our threshold signing application, completeness at the AVSS level is not needed because the pipeline-level star-finding mechanism serves the same purpose.

Star-finding and the $t < n/4$ threshold. Star-finding algorithms were introduced by Canetti [Can96] in the context of AVSS with $t < n/4$, and further developed by Choudhury and Patra [CP17]. In both works, star-finding is used *within* each AVSS instance to identify a core set of parties during a verification stage, after which a completion stage ensures that all parties obtain valid shares.

Our approach uses star-finding in a different way: we forgo completeness entirely and instead use star-finding at the pipeline level to agree on which dealers and receivers to use for a given presignature batch. This avoids the communication cost of the completion stage and leads to a significantly simpler protocol with much better concrete efficiency (see Table 1).

SPRINT and reduced resilience thresholds. SPRINT [BHK+24] is a threshold Schnorr signing protocol that, like our work, operates in the asynchronous setting with robustness. SPRINT uses "packed secret sharing" to amortize costs, at the expense of a weaker resilience threshold: $n \geq 3t + 2a - 1$, where a is a packing parameter. As shown in the detailed comparison in [GS23], even with packing, SPRINT's computational cost is significantly higher (roughly $5\times$ slower for $n = 49$), owing to its reliance on polynomial commitments with expensive group operations.

In the context of our $t < n/4$ protocol, SPRINT is relevant as a precedent for using a weaker corruption bound to achieve better amortized performance. However, the starting point and techniques are quite different: SPRINT trades resilience for packing efficiency within a polynomial-commitment-based AVSS, while our $t < n/4$ protocol trades resilience for a simpler pipeline architecture that eliminates the need for the SMD subprotocol and complaint mechanism entirely.

Note that SPRINT is only analyzed under static corruptions.

Adaptive security for threshold Schnorr signatures. Adaptive security has recently become a central concern for threshold Schnorr signatures, motivated in part by NIST's call for threshold schemes [BP23] and by a striking observation of Crites and Stewart [CS25]. They show that for any threshold Schnorr scheme in which public key shares have the form $pk_i = g^{sk_i}$, with all secret key shares lying on a polynomial, full adaptive security cannot be proven without assuming the hardness of a certain search problem P whose hardness is unclear. Moreover, this barrier holds even with secure erasures. This vulnerability applies to many protocols, including the SPRINT and FROST protocols mentioned above.

Another protocol to which the Crites/Stewart vulnerability applies is the HARTS [BLSW24] protocol. Like our protocol, HARTS is a threshold Schnorr protocol that achieves optimal resilience ($t < n/3$) and robustness in the asynchronous communication model. Unlike our protocol, HARTS supports a *dual threshold* mechanism: up to $t < n/3$ corruptions and a reconstruction threshold of $t^\dagger \in [t+1, n-t]$. Such a dual threshold mechanism can be useful in applications (see [Sho00]). Unlike

our protocol, the amortized communication complexity of HARTS is $O(\kappa n^2 \log n)$ per signature, and relies on quite heavyweight cryptographic operations.

The paper [BLSW24] gives a proof that HARTS is adaptively secure (with erasures) in the algebraic group model (AGM) under the one-more discrete logarithm assumption. However, in light of [CS25], this proof has a gap. We note that the Crites/Stewart vulnerability itself does not apply when $t^\dagger - t$ is sufficiently large (but it is unclear whether the proof gap can be closed).

Our protocol sidesteps the Crites/Stewart vulnerability. Indeed, because we use lightweight, hash-based verification rather than polynomial commitments, and rely on online error correction [BOCG93, Can96] during reconstruction, no group-element commitments to individual secret key shares are ever published. The group elements that are published ($\mathcal{S}_\ell = f_\ell(0)\mathcal{G}$ for the shared polynomials) are ephemeral presignature public keys, not commitments to points on the secret key polynomial, and no party is actually holding corresponding secret keys. The design in [GS23] already had this structure; by making a few subtle modifications (including sender-forward-secure channels with erasures), we can prove full adaptive security.

Several recent works achieve adaptive security for threshold Schnorr through different means. Glacius [BDLR24] and Gargos [BDLR25] use equivocable Pedersen commitments ($pk_i = g^{sk_i} h^{r_i}$) to allow the simulator to reprogram secret keys, achieving adaptive security from DDH without erasures. However, these are synchronous, non-robust protocols. Moreover, they rely on heavyweight cryptography and have a communication complexity of $O(\kappa n^2)$ per signature, making them unsuitable for high-throughput signing.

In a different direction, Das and Ren [DR23] and Das, Ren, and Yang [DRY25] achieve adaptive security for threshold BLS signatures and threshold ElGamal decryption, respectively, from DDH in pairing-based settings. These address different primitives and use heavyweight cryptography, but are part of the same broader effort toward provable adaptive security for threshold cryptography.

Our protocol achieves adaptive security (albeit with erasures), optimal resilience ($t < n/3$), and robustness in the asynchronous communication model. Moreover, it does so while achieving extremely high throughput on the happy path, and the player elimination framework ensures rapid convergence back to the happy-path rate after any disruption.

2 Preliminaries

2.1 Schnorr signatures

Let us first recall the Schnorr signature scheme. Let E be a group of prime order q generated by $\mathcal{G} \in E$. As mentioned already, we use additive notation for the group operation of E, and denote the identity element of E by $\mathcal{O}$. The secret key is a random $x \in F := \mathbb{Z}_q$ and the public key is $\mathcal{X} \leftarrow x\mathcal{G} \in E$. To sign a message m, the signer chooses $r \in F$ at random, computes

$$\mathcal{R} \leftarrow r\mathcal{G} \in E, \quad h \leftarrow H_{\text{sign}}(\mathcal{X}, \mathcal{R}, m) \in F, \quad s \leftarrow r + xh \in F,$$

and outputs the signature $(\mathcal{R}, s) \in E \times F$.

2.2 Network model

We assume n parties, $P_1, \ldots, P_n$. Except where otherwise noted, we assume that at most $t < n/3$ parties may be corrupted. Of course, we assume adaptive corruptions, and we allow erasures — so whenever a party is corrupted, the adversary obtains the current state of the corrupted party.

Parties are connected by asynchronous, point-to-point channels. We review here the different types of authenticated and secure channels we will need, and how they might be implemented. We will assume that a given channel is only used to transmit a single message m from a sender S to a receiver R. We will also assume that for each channel used in the protocol, all messages have a fixed, system-defined length.

2.2.1 Decentralized key provisioning

To implement these channels, we may assume some form of **decentralized key provisioning**, as discussed in Section 22.4.1 of [BS23]. This allows each party P_i to generate a public key pk_i so that all parties are provisioned with $(pk_1, \ldots, pk_n)$. This is a fairly weak assumption, in that we assume that corrupt parties may generate their public keys in an arbitrary fashion, possibly depending on the public keys of honest parties. The setting in [BS23] is generally in the static corruption model, but is easily adapted to the adaptive corruption model.

2.2.2 Authenticated channels

An authenticated channel provides integrity but no privacy. In the ideal functionality $\mathfrak{F}_{\text{auth}}$, the sender S may input a message m, which is immediately leaked to the ideal-world adversary. At a later time, that adversary may deliver a message to R. If S was honest at the time the message m was sent and is still honest when the message is delivered to R, then m is the message delivered to R; otherwise, the message delivered to R is chosen by the adversary.

Such a channel can easily be implemented using digital signatures, or a non-interactive hybrid scheme, in which S can send to R an encryption of a symmetric MAC key under R's public encryption key, signed under S's signing key. This symmetric key can be used for all channels from S to R. It is important that the encryption is generated using the ID of the sender as associated data (see Section 12.7 of [BS23]). In fact, one can get by with an AD-only CCA secure encryption scheme (see Section 12.7.1 of [BS23]).

We generally do not want to use any kind of interactive key exchange protocol for implementing this (or another) type of channel, as in the asynchronous communication setting, it would prevent us from using some essential techniques. Specifically, for some protocols, we want one party to be able to erase certain data before transmitting messages to all parties. If the sending party had not completed the interactive step for *all* parties, it could not proceed.

2.2.3 Secure channels

A secure channel provides both integrity and privacy. In the ideal functionality $\mathfrak{F}_{\text{sec}}$, the sender S may input a message m. The message m is immediately leaked to the ideal-world adversary only if R is currently corrupted. However, if *either* S *or* R are corrupted at any later point in time, then m is leaked to the adversary at that time. Authenticity is the same as above: if S was honest at the time the message m was sent and is still honest when the message is delivered to R, then m is the message delivered to R; otherwise, the message delivered to R is explicitly chosen by the adversary.

Such a channel can easily be implemented on top of an authenticated channel using CCA secure public key encryption. It is important that the encryption is generated using the ID of the sender as associated data, and in fact, one can get by with an AD-only CCA secure encryption scheme.

One can also use a non-interactive hybrid scheme, in which S sends to R an encryption of a symmetric encryption key under R's public encryption key signed under S's signing key. This symmetric key can be used for all channels from S to R.

To make the simulation work, we need to employ a type of non-committing encryption, which is easily achieved using the standard technique of encrypting the plaintext m as $m \oplus h$, where h is the output of a hash function that we can model as a global programmable random oracle (as in [CDG+18]).

2.2.4 Sender forward-secure channels

For some settings, we need a stronger type of secure channel, which we call a **sender-forward-secure channel**, which provides both **sender-forward secrecy** and a specific type of **non-malleability**. Unlike the authenticated and plain secure channels, a sender-forward-secure channel requires *erasures*.

In the ideal functionality $\mathfrak{F}_{\text{fsec}}$, the sender S may input a message m. The message m is immediately leaked to the ideal-world adversary only if R is currently corrupted. However, if R is corrupted at any later point in time, then m is leaked to the adversary at that time. Note that m is not leaked to the adversary if S is later corrupted (unlike an ordinary secure channel), providing **sender-forward secrecy**.

Authenticity is also strengthened in the following way to provide a form of **non-malleability**. If S was honest at the time the message m was sent and is still honest when the message is delivered to R, then m is the message delivered to R; otherwise, the adversary can choose to deliver either

(a) the original message m, or

(b) a message that it must explicitly choose (independent of m).

Such a channel can easily be implemented on top of an authenticated channel using CCA secure public key encryption. It is important that the encryption is generated using the session ID of the channel (which includes the ID of the sender) as associated data. To achieve non-malleability, one needs *full* CCA security (not just AD-only CCA security). In addition, to achieve sender-forward secrecy, it is essential that after encrypting the message, *the sender erases the message and the randomness used by the encryption algorithm.*

Designing a hybrid implementation of this is more challenging. One approach would be as follows. We can use a GGM-style tree-based PRF [GGM86] to derive a sequence of keys $F(0), F(1), \ldots, F(2^k - 1)$ from a root secret, where 2^k is a bound on the number of keys that will be generated. Initially, the sender can transmit to the receiver the tree root using public-key encryption as above. The receiver holds the root and can compute any $F(i)$ directly by traversing the tree from root to leaf i. The sender, however, maintains only the minimal state needed to compute $F(i), F(i+1), \ldots$ sequentially, consisting of $O(k)$ nodes: specifically, the siblings along the path from the current leaf to the root that are roots of not-yet-traversed right subtrees. After computing $F(i)$, the sender derives any new sibling nodes needed for subsequent leaves, then erases the nodes that are no longer required. This ensures that compromise of the sender's state at any point reveals only future keys, not past ones, providing forward secrecy.

To actually implement a sender-forward-secure channel, we use this technique as follows. We require that for each sender/receiver pair, the sender can use the indices $i = 0, 1, \ldots$ as channel IDs in that order. This will be the case for all applications in this paper. Note that we do not require

the sender and receiver to stay synchronized in any way, which is required in constructions such as those in [BY01]. To actually encrypt a message m in the ith channel, the sender would derive from $F(i)$ a one-time encryption key k_{enc} and a one-time MAC key k_{mac}, and then compute $c \leftarrow m \oplus h$, where h is the output of a global programmable random oracle on input k_{enc}, and then apply the MAC to c using the key k_{mac} to get a tag t. The entire ciphertext is (c, t).

Now, before the receiver will decrypt any of these ciphertexts, it must wait for the initial ciphertext containing the GGM root, even if there are several channels with messages in flight. If the sender is corrupted before that happens, it may choose to deliver that initial ciphertext unchanged, or deliver a different ciphertext.

- In the former case, for all channels with messages in flight, the adversary must deliver those messages unchanged.
- In the latter case, for all channels with messages in flight, the adversary must deliver messages that it explicitly chooses (independent of the original messages in flight).

For our application, this hybrid approach is perhaps overkill. However, it could be useful in applications where post-quantum secure public-key encryption is used and so minimizing the actual use of such (expensive) encryption is desirable (from an efficiency point of view).

2.3 Reliable broadcast

A **reliable broadcast (RBC)** protocol allows a sender S to broadcast a single message m to $P_1, \ldots, P_n$. Such a protocol should satisfy the following *correctness* property (informally stated):

> *All honest parties that output a message must output the same message. Moreover, if the sender S is honest, that message is the one input by S.*

It should also satisfy the following *completeness* property (informally stated):

> *If any honest party outputs a message or an honest sender S inputs a message, then eventually all honest parties output a message.*

Here, "eventually" means (roughly speaking) if and when there are no undelivered protocol messages that were sent from a party that is currently honest to an honest party that is currently honest.

In the adaptive corruption setting, these notions need to be more precisely defined.

For correctness, our requirements are as follows:

- Consider the first point in time that some honest party P outputs a message m. Then at any subsequent time that an honest party outputs a message, that message must be m, even if P is corrupt at that point in time.
- Suppose that S is honest at the time it inputs a message m. Consider the first point in time that some honest party P outputs a message. If S is still honest at this time, then that message must be m.

Note that one implication of this definition is that the sender cannot just immediately output its own message (which it could do in the static setting).

For completeness, our requirements are as follows:

- If some honest party P outputs a message, then for every currently honest party Q, eventually either Q will output a message or Q will become corrupted.
- If S inputs a message at a time that it is honest, eventually some honest party will output a message or S will become corrupted.

While Bracha's classical 3-round protocol [Bra87] satisfies this definition, other protocols do not. For example, the 2-round protocol Fig. 1 of [ANRX21], does not satisfy the completeness property as we have defined it here. In that protocol, the first honest party P to output a message is supposed to also transmit a quorum certificate to all other parties to make sure that they also output the message. However, P could immediately become corrupted, and that quorum certificate may not be delivered to anyone. We could have defined the completeness property more liberally, replacing the first bullet point by:

- If some honest party P outputs a message, then for every currently honest party Q, eventually either Q will output a message or P *or* Q will become corrupted.

Arguably, this weaker definition of completeness is adequate for most applications, and certainly all of our applications here. However, we find the stronger notion of completeness a bit more natural to work with.

To make our working definition of completeness painfully clear, it can be stated as follows: it is infeasible for the adversary to drive the system into a state where all messages between honest parties have been delivered, some honest parties have not yet output a message, and at least one of the following holds:

- some party previously output a message at a time that it was honest, or
- the sender input a message at a time that it was honest and is still honest.

Bracha's classical RBC protocol, with the usual 3-round *send/echo/vote* logic, has a communication complexity of $O(n^2|m|)$ to broadcast a message m. We will need an RBC protocol with better communication complexity. To this end, we can use the protocol in Section 3.2.2 of [SS23]. This protocol uses only lightweight cryptography (a collision resistant hash), and to broadcast a message m, its communication complexity is

$$\frac{n}{n-2t}(n+1)|m| + O(\kappa n^2 \log n),$$

where κ is the output length of the hash function. Note that for $n > 3t$, this is bounded by

$$3n|m| + O(\kappa n^2 \log n).$$

Moreover, communication is very well balanced, with every party transmitting roughly the same number of bits. However, this balanced property is not essential for our application, as every party will play the role of broadcaster.

The RBC protocol in [SS23] is based on a Merkle tree whose leaves form the code word for an $(n, n-2t)$-erasure encoding of the message m. That is, m is encoded as a sequence of n fragments, each of size $|m|/(n-2t) + O(\log n)$, any $n-2t$ of which may be used to reconstruct m, and these n fragments form the leaves of the Merkle tree. The protocol follows the usual *send/echo/vote* structure of Bracha broadcast, where the root ρ is included in each such message. The sender also

sends each party a fragment and corresponding Merkle path, who in turn broadcasts the fragment and path to all parties. This data is "piggybacked" with the *send* and *echo* messages, respectively. Note that the reconstruction procedure may fail, in which case the message output by all parties will be $\perp$ (this can only happen if the sender is corrupt).

The communication complexity for RBC was improved in [LS24]. For $n > 3t$, the MiniCast protocol in [LS24] achieves a communication complexity of

$$1.5n|m| + O(\kappa n^2 \log n).$$

We will in fact make use of the MiniCast protocol here.

2.4 Reliable agreement

In a **reliable agreement (RA)** protocol, each party P_i may input a message and output a message. The *correctness* property is:

> *All honest parties that output a message must output the same message; moreover, if t' parties are corrupt at the time the first party outputs a message, then $n - t - t'$ honest parties must have input that message.*

The *completeness* property is:

> *If any honest party outputs a message, or if all honest parties input the same message, then eventually all honest parties output a message.*

Bracha's RBC protocol can be easily adapted to implement an RA protocol: each party's input is translated into an echo message in Bracha's protocol, and the logic for *vote* messages is the same as in Bracha's protocol. This adaptation of Bracha's broadcast to RA is quite well known — see, for example, [DDL+24]. As for communication complexity, all of the bounds cited above for reliable broadcast also hold here. In [SS23], a degenerate form of this, where the only possible message is a null message, is used for "one-sided voting".

2.5 Consistent broadcast

A **consistent broadcast (CBC)** protocol is the same as a reliable broadcast protocol, except that the completeness requirement is weakened:

> *If an honest sender S inputs a message, then eventually all honest parties output a message.*

A simple protocol for this is presented in [ST87]. It is the same as Bracha's protocol, but without the third round.

2.6 Asynchronous verifiable information dispersal

An **asynchronous verifiable information dispersal (AVID)** protocol allows a sender S to *disperse* a single message m among $P_1, \ldots, P_n$ so that at a later time, the message can be selectively *delivered* to one or more parties. There are two phases to such a protocol: the **dispersal phase**, where S disperses m (or fragments of m) among $P_1, \ldots, P_n$, and the **delivery phase**, where the message is selectively delivered to individual parties. The *correctness* property for such a protocol is essentially the same as that of a reliable broadcast protocol:

All honest parties that output a message in the delivery phase output the same message. Moreover, if S is honest, that message is m.

The *completeness* property has two parts. First, in the dispersal phase:

If an honest sender S inputs a message or if one honest party completes the dispersal phase, then every honest party eventually completes the dispersal phase.

Second, in the delivery phase:

If all honest parties have completed the dispersal phase and have initiated the delivery of the message to a party P_i, then P_i eventually outputs a message.

Note that one can use any AVID protocol to implement reliable broadcast, by first dispersing the message and then having every party retrieve the message. Indeed, the RBC protocol in Section 3.2.2 of [SS23] can be seen as an optimized version of this implementation applied to the AVID protocol in the DispersedLedger system, called *AVID-M* [YPA+21], which uses exactly the same Merkle tree encoding of the message. In the AVID-M protocol, to disperse a message m, the sender transmits

$$\frac{n}{n-2t}|m| + O(\kappa n \log n)$$

bits and every party transmits $O(\kappa n)$ bits. To deliver m to one party, every party transmits

$$\frac{|m|}{n-2t} + O(\kappa \log n)$$

bits to that party.

Such an AVID protocol can lead to much better communication complexity in situations where either

- a single dispersed message only needs to be delivered to a few parties, or
- many messages are dispersed, but only a few need to be delivered.

2.7 Asynchronous common subset

An **asynchronous common subset (ACS)** protocol allows the parties to agree on a set $S \subseteq [n]$ of at least $n - t$ parties. The protocol is parameterized by an **asynchronous validity predicate** [Sho24]: each party P_i may locally *validate* each other party P_k. Each party inputs k to the ACS protocol as soon as it validates P_k.

The protocol guarantees:

- *Consistency:* all honest parties output the same set S with $|S| \geq n - t$.
- *Validity:* when a party outputs S, it has validated P_k for every $k \in S$.
- *Liveness:* all honest parties eventually output a set.

Here, "eventually" means: once (1) all messages sent from one honest party to another have been delivered and (2) every party that has been validated by some honest party has been validated by all honest parties.

This notion of liveness composes naturally with other protocols. For example, if validation of P_k means that P_k's reliable broadcast has delivered, then the completeness property of RBC guarantees that validation by one honest party implies eventual validation by all honest parties. The same holds if validation is defined in terms of any protocol with a similar completeness property, such as a complete AVSS protocol.

The ACS protocols in [DDL+24, Sho24] have $O(\kappa n^3)$ communication complexity, have expected constant round complexity, do not require any setup assumptions beyond secure channels, and rely only on lightweight cryptography. Although these protocols were analyzed in the static corruption model, this was done mainly for simplicity. It should be straightforward to establish their security under adaptive corruptions (even without erasures). The main issue to consider is the ASKS protocol. For this, one needs the generalized linear hiding assumption we gave above (see Section 3.1), or just work directly in the random oracle model.

3 Secure Message Distribution

Section 4 of [SS23] presents a protocol for a new primitive, called **secure message distribution (SMD)**.

SMD is similar to AVID. Roughly speaking, the main differences are that

- Instead of dispersing a single message, a vector $\vec{m} = (m_1, \ldots, m_n)$ of messages is dispersed. Individual messages can then be delivered to specified parties (in a specific manner described below).
- If the sender is honest, each message remains **private** until it becomes **compromised**. Roughly speaking, a message becomes compromised when it is delivered to a corrupt party. Details are given below, but an essential feature is that corrupting the sender does not compromise any messages.

We shall assume the following, more specific message delivery pattern, which is slightly different than in [SS23], and which better suits our needs here:

- Initially, after the dispersal phase, each message m_i is automatically delivered to P_i.
- A party P_i may **forward** its own message m_i to a party P_j.
- Each party may choose to **open** an arbitrary message m_i for all parties to see.

In the UC model, these properties may be captured by an ideal functionality where the vector $\vec{m}$ is input to the ideal functionality (which models online extractability), and the message m_i is given to the simulator as soon as it becomes compromised (which models the privacy requirement).

With adaptive corruptions, some subtleties arise, especially as regards a sender who starts out as honest but is later corrupted. Section 3.3 gives a detailed ideal functionality, but we sketch the main idea here. Suppose the sender inputs a vector of messages $\vec{m} = (m_1, \ldots, m_n)$. Initially, the ideal-world adversary learns nothing about these inputs. Before any messages get delivered to any party, the ideal-world adversary can choose to either *lock* these inputs, or to corrupt the sender.

In the latter case, it can subsequently choose to *lock* either the sender's original input vector or its own input vector (essentially overwriting the sender's original inputs). We stress that up until the point that the inputs are *locked* (either before or after corrupting the sender), the ideal-world adversary learns nothing about any of the sender's original input messages. If it *locks* the sender's original input, it will only obtain information about those messages that are clearly compromised (messages delivered or forwarded to a corrupt party, or messages that some honest parties open). If it chooses not to *lock* the sender's original input, it will *never* learn anything about any of the sender's messages; in particular, if the ideal-world adversary locks its own input, it must explicitly provide those messages, and they do not depend on the sender's messages.

An SMD protocol should also satisfy a completeness property, which extends that of reliable broadcast. Roughly speaking:

- *If any honest party outputs a message or an honest sender S inputs a message, then eventually all honest parties output a message.*
- *Assuming some honest party outputs a message, any forwarding operation will eventually complete, and any opening operation will complete provided it is initiated by all honest parties.*

3.1 An implementation

We adapt the SMD protocol from [SS23] with just a few changes. At the core of the design of the SMD protocol in [SS23] is a two-level Merkle tree. The top-level tree has root ρ and leaves $\rho_1, \ldots, \rho_n$, where each ρ_i is the root of Merkle tree that encodes an encryption of the message m_i. Specifically, the leaves of this tree are $(k_{i1}, f_{i1}), \ldots, (k_{in}, f_{in})$, where

- $(s_{i1}, \ldots, s_{in})$ is an $(n-2t)$-out-of-n Shamir secret sharing of a random secret s_{i0},
- $k_{ij} = H_{\text{ss}}(j, s_{ij})$ for $j = 0, 1, \ldots, n$,
- c_i is a symmetric encryption of m_i under the secret key k_{i0}, and
- $(f_{i1}, \ldots, f_{in})$ is an $(n-2t, n)$-erasure encoding of c_i.

3.1.1 Key changes

In [SS23], this protocol was analyzed in detail in the static corruption model. To prove security in the adaptive corruption model, several changes are needed.

Non-committing encryption. The symmetric encryption scheme used to encrypt a single message m should be of the form $\mathit{Encrypt}(k, m) = H_{\text{enc}}(k) \oplus m$, where we then model H_{enc} as a global programmable random oracle (as in [CDG+18]). This allows us to program H_{enc} appropriately at the point in time where a message is compromised.

Generalized linear hiding assumption. The linear hiding assumption introduced in [SS23] needs to be generalized to model adaptive corruptions. The new assumption just states exactly what we need: the adversary is given hashes $H_{\text{ss}}(1, s_1), \ldots, H_{\text{ss}}(n, s_n)$, where $(s_1, \ldots, s_n)$ is a d-out-of-n Shamir secret sharing of a random secret s_0. The adversary then adaptively obtains fewer than d of the preimages $s_1, \ldots, s_n$, and attempts to distinguish $H_{\text{ss}}(0, s_0)$ from random. This is a

perfectly reasonable concrete security assumption that is justified by the fact that it is certainly true if we model H_{ss} as a random oracle. In fact, in this work, we really only need to assume that it is hard to compute $H_{\text{ss}}(0, s_0)$, rather than just distinguish it from random.

Preliminary agreement on a Merkle root. We require that the sender consistently broadcasts the Merkle root ρ of the Merkle tree. This CBC protocol can run concurrently with sending the initial *send* message of the SMD protocol in [SS23]. However, each party will process this *send* message only after the consistent broadcast of ρ completes for that party, and will validate all Merkle paths with respect to this root ρ. The *echo* and *vote* messages will include ρ, as in the original protocol.

Intuitively, the messages of an honest sender become locked when the first honest party completes this CBC. After this time, the sender's input messages cannot be changed, even if the sender is subsequently corrupted.

Implementing this step adds just one round of communication to the original protocol in [SS23]. The root ρ can be included in the initial *send* message, and before this message is processed, each party sends a message $(\mathsf{pre\text{-}echo}, \rho)$ to each party, and waits for $n-t$ matching messages $(\mathsf{pre\text{-}echo}, \hat{\rho})$. The *send* message is only processed when this happens and only if $\rho = \hat{\rho}$ (and is ignored otherwise).

Data erasure and secure channels. We have specific requirements for the erasure of data and the security properties of the channels, discussed below in Section 3.2. These requirements will ensure that an honest sender's messages remain private, unless they are delivered to a corrupt party, even if the sender is corrupted at any time after it initiated the protocol.

3.2 Erasures and secure channels

The sender needs to securely transmit to each party P_j the secret shares $s_{1j}, \ldots, s_{nj}$. This needs to be done using a sender-forward-secure channel, as in Section 2.2.4. Moreover, to achieve the required security goals, the message sent to P_j should be of the form $(\rho, s_{1j}, \ldots, s_{nj})$, where ρ is the root of the Merkle tree. Recall that P_j will only process the initial *send* message it receives from the sender after the CBC subprotocol outputs a Merkle root $\hat{\rho}$. Party P_j will first check that $\rho = \hat{\rho}$; if not, P_j will treat the initial *send* message it receives as invalid.

These secret shares also need to be securely transmitted whenever they are relayed by parties in delivering initial messages, forwarding messages, or opening messages. This can be done using ordinary secure channels, as in Section 2.2.3. All other data that is transmitted by all parties can be sent using authenticated channels, as in Section 2.2.2.

Recall from Section 2.2.4 that we need to use erasures to implement these sender-forward-secure channels. In addition, all messages $m_1, \ldots, m_n$ must be erased and all of the secrets s_{i0} and secret shares s_{ij} must be erased. We are assuming here the simple "message passing" model underlying most formulations of secure distributed computation, where control remains with a machine until it explicitly relinquishes control by passing a message to another machine. In this model, we need to ensure that all of the data associated with messages, secrets, secret shares, and sender-forward-secure channels is *erased* before the sender's machine relinquishes control. In particular, an honest sender can only be corrupted after it relinquishes control, and so after all of this data has been erased.

3.2.1 Communication complexity

As for communication complexity, if each message m_i has length ℓ, to disperse $\vec{m} = (m_1, \ldots, m_n)$ the sender transmits a total of

$$\frac{n}{n-2t} n\ell + O(\kappa n^2 \log n)$$

bits and every party transmits $O(\kappa n)$ bits. To deliver one message to one party, each party transmits

$$\frac{1}{n-2t}\ell + O(\kappa \log(n))$$

bits to that party. To forward a message from one party to another, the forwarder transmits

$$\ell + O(\kappa n \log(n))$$

bits to the receiving party.

3.2.2 Other changes from [SS23]

In [SS23], a party P_i could forward a message m_j from P_j to another party P_k. We do not need this functionality here.

A new feature not in [SS23] is the ability for the honest parties, working together, to force the opening of any party's initial message, without the help of that party. Note that this feature exploits the particular construction in [SS23] that Shamir-shares a symmetric encryption key for each message. In [SS23], this technique was motivated mainly by a desire to avoid public-key cryptography. Here, it plays a crucial role in implementing the opening functionality, which we need for our new player elimination framework.

3.3 Ideal functionality

We present here an ideal functionality $\mathfrak{F}_{\text{SMD}}$ for SMD. It differs somewhat from that given in [SS23], in that it

- models adaptive corruptions, and
- models a slightly different message delivery pattern.

As stated, $\mathfrak{F}_{\text{SMD}}$ can really only be realized if we allow erasures. Here is the logic of $\mathfrak{F}_{\text{SMD}}$, expressed as a collection of operations that may be invoked upon it:

- $\mathsf{Input}(m_1, \ldots, m_n)$: this operation may be invoked once by an honest sender S, who inputs messages $m_1, \ldots, m_n$. In response, $\mathfrak{F}_{\text{SMD}}$ sends the message $\mathsf{NotifyInput}()$ to the ideal-world adversary.

  We assume for simplicity that all messages have a fixed, system-defined length, so that we do not have to leak length information to the adversary.

- $\mathsf{Lock}()$: this operation may be invoked by the ideal-world adversary at a point in time that no Lock operation was previously invoked and the Input operation was previously invoked (S was honest when it invoked Input, but may be honest or corrupt at this point in time).

  Note that after S's input is locked, it cannot be changed even if S is later corrupted.

- $\mathsf{Lock}(m'_1, \ldots, m'_n)$: this operation may be invoked by the ideal-world adversary at a point in time that no Lock operation was previously invoked, and that S is currently corrupt. The ideal-world adversary inputs $m'_1, \ldots, m'_n$ to $\mathfrak{F}_{\text{SMD}}$. In response, $\mathfrak{F}_{\text{SMD}}$ sets $m_i := m'_i$ for $i \in [n]$.

  This allows the ideal-world adversary to define the input messages of a corrupt sender. If the sender previously input messages at a time that it was honest, and those inputs were not already locked, this operation overwrites those inputs.

- $\mathsf{ProvideOwn}(i)$: This operation may be invoked once per $i \in [n]$ by the ideal-world adversary at any time after a Lock operation has been invoked. In response, $\mathfrak{F}_{\text{SMD}}$ sends the message $\mathsf{Own}(m_i)$ to party P_i.

- $\mathsf{Forward}(j)$: each party P_i may invoke this operation once per $j \in [n]$ after it has received its Own message. In response, $\mathfrak{F}_{\text{SMD}}$ sends the message $\mathsf{NotifyForward}(i, j)$ to the ideal-world adversary.

- $\mathsf{ProvideForwarded}(i, j)$: this operation may be invoked once per $(i, j) \in [n] \times [n]$ by the ideal-world adversary after P_i has invoked $\mathsf{Forward}(j)$ and after P_j has received its Own message. In response, $\mathfrak{F}_{\text{SMD}}$ sends the message $\mathsf{Forwarded}(i, m_i)$ to P_j.

- $\mathsf{Open}(j)$: each *honest* party P_i may invoke this operation once per $j \in [n]$ after it has received its Own message. In response, $\mathfrak{F}_{\text{SMD}}$ sends the message $\mathsf{NotifyOpen}(i, j)$ to the ideal-world adversary.

- $\mathsf{ProvideOpened}(i, j)$: this operation may be invoked once per $(i, j) \in [n] \times [n]$ by the ideal-world adversary after some honest party invoked $\mathsf{Open}(j)$ and after P_i received its Own message. In response, $\mathfrak{F}_{\text{SMD}}$ sends the message $\mathsf{Opened}(j, m_j)$ to P_i.

- $\mathsf{LeakMessage}(i)$: this operation may be invoked once per $i \in [n]$ by the ideal-world adversary at any point in time after one of the following conditions is satisfied:

  – a Lock operation has been invoked and P_i is currently corrupt,
  – an $\mathsf{Forward}(j)$ operation has been invoked and P_j is currently corrupt, or
  – an $\mathsf{Open}(i)$ operation has been invoked.

  In response, $\mathfrak{F}_{\text{SMD}}$ sends the message $\mathsf{Leakage}(m_i)$ to the ideal-world adversary.

  The preconditions of this operation precisely define when a message is ***compromised****.*

3.4 Security proof sketch

We sketch here the main ideas of the security proof for the protocol in Section 3.1, focusing on those aspects that are specific to adaptive corruptions, and our use of sender-forward-secure channels and erasure.

We first recall the assumptions we are making about various hash functions.

- The Merkle tree hash function is collision resistant.

- The hash function H_{ss} used to hash secret keys and secret shares s_{ij} (as $k_{ij} \leftarrow H_{\text{ss}}(j, s_{ij})$) is collision resistant and satisfies the generalized linear hiding assumption given above in Section 3.1.1.

- The hash function H_{enc} used to encrypt the sender's messages (as $\mathit{Encrypt}(k, m) \leftarrow H_{\text{enc}}(k) \oplus m$) is modeled as a global programmable random oracle.

There may be other hash functions and cryptographic primitives used to implement the various types of channels we need (as in Section 2.2), but we treat those channels as ideal functionalities.

We prove security through a sequence of games, starting with the "real world" and ending with a game that corresponds to the "ideal world" with a simulator.

Each run of the protocol falls into one of the following cases, depending on the outcome of the CBC subprotocol that determines the Merkle root $\hat{\rho}$.

- **Honest lock**: the sender was initially honest and $\hat{\rho} = \rho$, where ρ is the root of the Merkle tree generated by the sender. The sender may be corrupted at any time (before or after the CBC completes), but its original input is locked.
- **Corrupt lock**: either the sender was corrupt from the start (**initially corrupt lock**), or the sender was initially honest but $\hat{\rho} \neq \rho$ (**transitional corrupt lock**).

Note that when the sender is initially honest but later corrupted, the CBC may output either $\hat{\rho} = \rho$ (honest lock) or $\hat{\rho} \neq \rho$ (transitional corrupt lock), depending on whether the adversary interfered with the CBC protocol.

3.4.1 Game 0

This is the real world. All honest parties follow the protocol.

3.4.2 Game 1

In this game, we introduce a **teleportation** rule for message reconstruction in honest locks, analogous to the use of the correctness property of public-key encryption (in which a "correct" ciphertext is processed by directly delivering the underlying plaintext from sender to receiver, rather than by actually decrypting).

In the SMD protocol, messages are reconstructed and delivered to parties in three operations: $\mathsf{Own}(m_i)$ to P_i, $\mathsf{Forwarded}(i, m_i)$ to P_j, and $\mathsf{Opened}(j, m_j)$ to P_i. In each case, the reconstruction involves collecting secret shares s_{ij} from various parties to recover s_{i0}, derive the key $k_{i0} = H_{\text{ss}}(0, s_{i0})$, and decrypt m_i.

In an honest lock, the Merkle tree was generated by the honest sender, and the locked root is $\hat{\rho} = \rho$. By collision resistance of H_{ss} and the Merkle hash, any data that validates against ρ — including shares, Merkle paths, and ciphertext fragments — must be the data originally placed in the tree by the honest sender.

We now describe the teleportation rule for each operation. Recall that in the relay steps, secret shares are transmitted over (plain) secure channels. If the relaying party is honest at delivery time, the secure channel guarantees that the correct data arrives; if the relaying party is corrupt at delivery time, the adversary must explicitly specify the data.

- $\mathsf{Own}(m_i)$ and $\mathsf{Opened}(j, m_j)$: reconstruction collects shares from a mix of parties. Shares relayed by honest parties are correct by the secure channel guarantee — we assume this without verification. Shares from corrupt parties are explicitly specified by the adversary — we check these against the Merkle tree. By collision resistance, any share that passes the

check must be the original share. If all checks pass, we deliver the correct message directly without decrypting.

- $\mathsf{Forwarded}(i, m_i)$ to P_j: all data comes from a single party P_i. If P_i is honest at delivery time, the secure channel guarantees the correct data arrives, and we deliver the correct message directly. If P_i is corrupt, the adversary explicitly specifies the data, which we check against the Merkle tree as above.

In all cases, if any check on adversary-supplied data fails, the reconstruction fails as it would in Game 0.

By the collision resistance argument above, the only way the environment can distinguish Game 1 from Game 0 is if a collision occurs in H_{ss} or the Merkle hash, which happens with negligible probability.

3.4.3 Game 2

In this game, whenever the sender is initially honest, we replace all ciphertexts c_i with independent random values of the same length, where we model H_{enc} as a global programmable random oracle. In the case of an honest lock, when a message m_i subsequently becomes compromised (i.e., the preconditions of $\mathsf{LeakMessage}(i)$ in $\mathfrak{F}_{\text{SMD}}$ are met), we program $H_{\text{enc}}(k_{i0}) \leftarrow c_i \oplus m_i$ to maintain consistency. In the case of a transitional corrupt lock, no programming is performed.

This change is undetectable provided the adversary never explicitly queries H_{enc} at $k_{i0} = H_{\text{ss}}(0, s_{i0})$ before the programming occurs (in the honest lock case) or at any time (in the transitional corrupt lock case). Recall that $(s_{i1}, \ldots, s_{in})$ is an $(n-2t)$-out-of-n Shamir secret sharing of s_{i0}. By the generalized linear hiding assumption with $d = n - 2t$ (Section 3.1.1), an adversary that holds the hashes $k_{ij} = H_{\text{ss}}(j, s_{ij})$ for $j \in [n]$ (visible in the Merkle tree) but obtains fewer than $n - 2t$ of the shares cannot compute k_{i0} except with negligible probability. It therefore suffices to show that the adversary holds at most $t < n - 2t$ shares (since $n > 3t$) at the relevant point in time. The crux of the argument is tracking the adversary's information about the shares. Note that after Game 1, no honest party ever actually decrypts — messages are delivered by teleportation — and only adversary-supplied data is checked against the Merkle tree.

We justify the share count separately for each type of lock.

- In an *honest lock*, the sender transmits shares to each P_j over a sender-forward-secure channel. By sender-forward secrecy, corruption of the sender does not leak these shares. If the sender is corrupted while a message to an honest P_j is in flight, the non-malleability property requires the adversary to either deliver the original message unchanged (in which case P_j receives the shares but the adversary does not learn them) or deliver a message it explicitly chooses, independent of the original (in which case no information about the original shares leaks). During the relay steps in the Own, $\mathsf{Forwarded}$, and Opened operations, shares may be transmitted over plain secure channels, which leak when either endpoint is corrupted. However, one can verify — by examining each of the three compromise conditions for $\mathsf{LeakMessage}(i)$ — that any such leakage either involves a share already held by a corrupt party or coincides with m_i becoming compromised. It follows that before m_i is compromised, the adversary holds at most t shares, and the programming of H_{enc} succeeds except with negligible probability.

- In a *transitional corrupt lock*, the sender was initially honest, but $\hat{\rho} \neq \rho$. When an honest party P_j receives a message on the sender-forward-secure channel, it either receives the sender's

original message (containing ρ and the original shares $s_{1j}, \ldots, s_{nj}$), or a message explicitly chosen by the adversary (independent of the original, by the non-malleability property of the channel). In the first case — and this is the key point — P_j finds that $\rho \neq \hat{\rho}$ and rejects the message outright. In the second case, the adversary's message is independent of the original shares. In either case, the original shares are never relayed by honest parties. The only shares the adversary can obtain are those from directly corrupting parties — at most $t < n - 2t$ of them. By the generalized linear hiding assumption, the adversary cannot query H_{enc} at k_{i0} except with negligible probability, and the replacement of c_i by a random value is undetectable.

3.4.4 The simulator

Game 2 gives us the simulator. The simulator runs the protocol, except that when the sender is initially honest, it does not know the messages m_i; it generates random secrets and secret shares s_{ij} as in the protocol, and produces dummy ciphertexts.

The simulator handles the different lock types as follows.

Honest lock. The simulator invokes the $\mathsf{Lock}()$ operation. From this point, the sender's inputs cannot be changed, even if the sender is subsequently corrupted. The teleportation rule (Game 1) ensures that honest receivers always obtain the correct messages when reconstruction succeeds. When a message becomes compromised, the simulator obtains it from the ideal functionality and programs H_{enc} as described in Game 2. In particular, the sender's inputs cannot be changed and information about input messages leaks to the adversary only when they are compromised. This analysis is quite similar to that in [SS23] in the case where the sender is honest.

Corrupt lock. In both sub-cases of a corrupt lock, all data processed by honest parties is explicitly supplied by the adversary. By collision resistance of the Merkle hash and H_{ss}, the data committed under $\hat{\rho}$ uniquely determines the messages $m'_1, \ldots, m'_n$, ensuring that all honest parties reconstruct the same messages.

In the case of an initially corrupt lock, the simulator has explicit access to all data supplied by the adversary and can directly determine the outcome of all validations. The simulator extracts the adversary's effective inputs $m'_1, \ldots, m'_n$ from this data and invokes $\mathsf{Lock}(m'_1, \ldots, m'_n)$. The analysis runs along the same lines as the static analysis in [SS23] in the case where the sender is corrupt.

In the case of a transitional corrupt lock, the situation is the same: the adversary must explicitly provide any data to honest parties, since the sender's original messages on the sender-forward-secure channels are either rejected (because $\hat{\rho} \neq \rho$) or replaced by messages explicitly chosen by the adversary (by the non-malleability property of the channel). The simulator extracts inputs from the adversary's explicit data and invokes $\mathsf{Lock}(m'_1, \ldots, m'_n)$. Note that the simulator never needs the honest sender's original inputs $m_1, \ldots, m_n$.

4 A GoAVSS protocol

The basic idea of a GoAVSS protocol is that a dealer takes as input polynomials $f_1, \ldots, f_{L+M} \in F[X]_{\leq t}$, distributes shares of these polynomials to n parties at distinct nonzero evaluation points

$e_1, \ldots, e_n \in F$, and publishes the group elements $\mathcal{S}_\ell := f_\ell(0)\mathcal{G}$ for $\ell \in [L]$. For implementation purposes, we typically take $e_i = i$.

The notion of a GoAVSS protocol with $M = 0$ was introduced in [GS23]. Here, we generalize to allow $M > 0$ so that we can use a single protocol instance to share L "GoAVSS polynomials" and M "plain AVSS polynomials". In our application, we will typically have $M \ll L$.

The protocol is parameterized in terms of positive integers K and R, and a subset $\Gamma \subseteq F$. The choice of these parameters will be discussed below. However, we will always have $K \leq n$. Typically, R will be quite small — usually 1 or 2.

The protocol also makes use of a number of hash functions

- Hash functions H_{chall}, H'_{chall}, H''_{chall}. These will be used to generate Fiat-Shamir challenges.
- A hash function for building Merkle trees.
- The hash function H_{ss} used to hash secret shares as in the SMD protocol in Section 3.
- The hash function H_{enc} used to encrypt the sender's messages in the SMD protocol in Section 3.

We will model all of these as (local) random oracles. As such, in an implementation based on a single standard hash function, these should all be properly domain separated by session ID and by individual hash function.

4.1 Dealer logic

1. Dealer inputs polynomials $f_1, \ldots, f_{L+M+K+R} \in F[X]_{\leq t}$
2. Compute shares
$$v_{\ell,i} \leftarrow f_\ell(e_i) \in F \qquad (\text{for } \ell \in [L+M+K+R], i \in [n]). \tag{1}$$
3. Construct the Merkle tree with root ρ_1 used in the implementation of the SMD protocol (as in Section 3) to securely distribute the messages $(m_1, \ldots, m_n)$, where $m_i := \{v_{\ell,i}\}_{\ell \in [L+M+K+R]}$.
4. Pass ρ_1 to H_{chall} to get a challenge
$$\{\theta_{r,\ell}\}_{r \in [R], \ell \in [L+M+K]},$$
where each $\theta_{r,\ell}$ is a random element of F.
5. Compute polynomials
$$h_r \leftarrow f_{L+M+K+r} + \sum_{\ell \in [L+M+K]} \theta_{r,\ell} \cdot f_\ell \in F[X]_{\leq t} \qquad (\text{for } r \in [R]). \tag{2}$$
6. Construct the Merkle tree with root ρ_2 that includes
   (a) ρ_1, and
   (b) the Merkle tree used in the implementation of an RBC protocol (as in Section 2.3) to reliably broadcast $\{h_r\}_{r \in [R]}$.

7. Compute

   (a) group elements
   $$\mathcal{S}_\ell \leftarrow f_\ell(0)\mathcal{G} \in E \qquad (\text{for } \ell \in [L]); \tag{3}$$

   (b) group elements
   $$\mathcal{T}_k \leftarrow f_{L+M+k}(0)\mathcal{G} \in E \qquad (\text{for } k \in [K]). \tag{4}$$

8. Construct the Merkle tree with root ρ_3 that includes

   (a) ρ_2, and

   (b) the Merkle tree used in the implementation of an RBC protocol (as in Section 2.3) to reliably broadcast $(\{\mathcal{S}_\ell\}_{\ell\in[L]}, \{\mathcal{T}_k\}_{k\in[K]})$.

9. Pass ρ_3 to H'_{chall} to get a challenge
   $$\{\gamma_{k,\ell}\}_{k\in[K],\ell\in[L]},$$
   where each $\gamma_{k,\ell}$ is a random element of $\Gamma \subseteq F$.

10. Compute scalars $c_k \leftarrow h'_k(0) \in F$ for $k \in [K]$, where
    $$h'_k \leftarrow f_{L+M+k} + \sum_{\ell\in[L]} \gamma_{k,\ell} \cdot f_\ell \in F[X]_{\le t} \quad (\text{for } k \in [K]). \tag{5}$$

11. Construct the Merkle tree with root ρ_4 that includes

    (a) ρ_3, and

    (b) the Merkle tree used in the implementation of an RBC protocol (as in Section 2.3) to reliably broadcast $\{c_k\}_{k\in[K]}$.

12. Pass ρ_4 to H''_{chall} to get a challenge
    $$\{\delta_k\}_{k\in[K]},$$
    where each δ_k is a random element of F.

13. Compute the polynomial $h^* \leftarrow \sum_{k\in[K]} \delta_k h'_k \in F[X]_{\le t}$.

14. Construct the Merkle tree with root $\rho = \rho_5$ that includes

    (a) ρ_4, and

    (b) the Merkle tree used in the implementation of an RBC protocol (as in Section 2.3) to reliably broadcast h^*.

15. Send each party the data associated with the initial *send* messages associated with all of the SMD and RBC subprotocols, along with all of the other Merkle roots $\rho_1, \ldots, \rho_4$, which allows for the Merkle path verification of all Merkle subtrees with respect to the root ρ.

    Just as we did in Section 3, the sender also consistently broadcasts the Merkle root ρ of the Merkle tree. This CBC runs concurrently with the dissemination of the initial *send* messages.

Crucially, just as discussed in Section 3.2, the secret shares associated with the SMD must be transmitted as messages over sender-forward-secure channels, with the root ρ included in those messages — *however, we include the top-level root ρ, rather than the root associated with the SMD protocol.* In addition, the dealer should also erase all other data generated above. This means that if the dealer is subsequently corrupted, no information about this dealing will leak to the adversary.

Discussion. Steps 1–6 of the dealer's protocol correspond to the AVSS protocol Π_{avss1} in Section 5 of [SS23], implemented using a random oracle as discussed in Section 7.1 of [SS23]. Steps 7–14 of the dealer's protocol correspond to the GoAVSS protocol Π_{GoAVSS2} in Section 5.3 of [GS23], implemented using a random oracle as discussed in Section 5.5 of [GS23] and using the probabilistic check as discussed in Section 5.6.2 of [GS23].

4.2 Receiver logic

Parties $P_1, \ldots, P_n$, acting in their role as *receivers* proceed as follows.

1. They first participate in the protocol for consistently broadcasting the Merkle root ρ.

2. Once each party completes the consistent broadcast of ρ, it processes the *send* messages from the dealer, performing the *echo* and *vote* logic associated with the SMD and RBC subprotocols. All Merkle path validations are done with respect to the root ρ output from the CBC protocol.

   As discussed in Section 3.2, the secret shares relayed in this step need to be sent over (ordinary) secure channels. No data needs to be erased.

3. When each party P_i obtains the messages from the SMD and RBC subprotocols (some of which may be $\perp$), it performs the local checks associated with protocols Π_{avss1} and Π_{GoAVSS2}. Note that at this point, P_i has the Merkle roots $\rho_1, \ldots, \rho$.

   First, P_i attempts to reconstruct the message $m_i = \{v_{\ell,i}\}_{\ell \in [L+M+K+R]}$, using the logic of the SMD subprotocol, and the message $\{h_r\}_{r \in [R]}$ using the logic of the RBC subprotocol, with each $v_{\ell,i} \in F$ and each $h_r \in F[X]_{\leq t}$.

   Second, if these reconstructions succeed, P_i checks that

   $$h_r(e_i) = v_{L+M+K+r,i} + \sum_{\ell \in [L+M+K]} \theta_{r,\ell} \cdot v_{\ell,i} \qquad (r \in [R]), \tag{6}$$

   where $\{\theta_{r,\ell}\}_{r \in [R], \ell \in [L+M+K]}$ is obtained from H_{chall} on input ρ_1.

   These checks correspond to the checks in Π_{avss1}.

   Third, if the above checks succeed, P_i attempts to reconstruct the messages $(\{\mathcal{S}_\ell\}_{\ell \in [L]}, \{\mathcal{T}_k\}_{k \in [K]})$, $\{c_k\}_{k \in [K]}$, and $h^* \in F[X]_{\leq t}$ using the logic of the RBC subprotocol.

   Fourth, if these reconstructions succeed, P_i checks that

   $$h^*(e_i) = \sum_{k \in [K]} \delta_k \cdot \Big(v_{L+M+k,i} + \sum_{\ell \in [L]} \gamma_{k,\ell} \cdot v_{\ell,i} \Big), \tag{7}$$

$$h^*(0) = \sum_{k \in [K]} \delta_k \cdot c_k, \tag{8}$$

and

$$c_{\kappa(i)}\mathcal{G} = \mathcal{T}_{\kappa(i)} + \sum_{\ell \in [L]} \gamma_{\kappa(i),\ell}\mathcal{S}_\ell, \tag{9}$$

where $\kappa(i) \coloneqq (i \bmod K) + 1 \in [K]$, $\{\gamma_{k,\ell}\}_{k\in[K],\ell\in[L]}$ is obtained from H'_{chall} on input ρ_3, and $\{\delta_k\}_{k\in[K]}$, is obtained from H''_{chall} on input ρ_4.

These checks correspond to the checks in Π_{GoAVSS2} (using the probabilistic check as discussed in Section 5.6.2 of [GS23]).

If all of the above checks pass, P_i inputs a null message into an RA protocol to perform a "one-sided vote".

4. Each party waits for the one-sided vote to return a result, and if that happens, it outputs $\mathsf{Ready}(\{\mathcal{S}_\ell\}_{\ell\in[L]})$.

Consider the point in time at which an honest party P_i outputs $\mathsf{Ready}(\{\mathcal{S}_\ell\}_{\ell\in[L]})$. As we will argue, we can be sure that (with overwhelming probability) the dealer has properly shared polynomials $f_1, \ldots, f_{L+M}$ of degree at most t among at least $n - t - t'$ honest parties, where t' is the number of currently corrupt parties, and that $\mathcal{S}_\ell = f_\ell(0)\mathcal{G}$ for $\ell \in [L]$. Moreover, we can be sure that all honest parties will eventually output $\mathsf{Ready}(\{\mathcal{S}_\ell\}_{\ell\in[L]})$. However, P_i may not itself hold valid shares of the polynomials $f_1, \ldots, f_{L+M}$. The criterion for determining if the P_i's shares are valid is the condition that $m_i \neq \perp$ and that equation (6) holds. In [SS23], a particular **complaint resolution mechanism** is given that will allow every party to obtain valid shares. The basic idea is this: P_i can forward its message m_i to all parties, so as to prove that the dealer is corrupt. In response, any party P_j that did receive valid shares can itself verify that the dealer is corrupt, and forward its message m_j to P_i. Then P_i itself can verify which such messages m_j are valid, and after receiving $t + 1$ valid messages, can interpolate to compute its correct shares.

The complaint resolution mechanism in [SS23] can significantly increase the communication complexity. However, when this happens, all parties will eventually learn the identity of the corrupt party that caused the damage, and when sufficiently many parties learn the identity of this party, it will be effectively eliminated and cause no more damage. While this property of the mechanism will *eventually* stop the corrupt party from causing further damage to the protocol, there is no a priori bound on *when* this will happen, *or on how many signing requests will be processed while the protocol's performance is degraded.*

Below, in Section 6, we propose a more refined complaint resolution mechanism that is tightly integrated with a novel **player elimination framework** coupled with a **signature generation pipeline**. As we will see, with these more refined techniques, a corrupt party can only degrade the protocol's performance for a bounded amount of time, and, more specifically, a bounded number of signing requests will be performed before the corrupt party is eliminated. These new techniques apply not only to our threshold signature protocol, but to a broad range of applications that use AVSS (such as asynchronous MPC).

4.3 GoAVSS Ideal functionality

We present here an ideal functionality $\mathfrak{F}_{\text{GoAVSS}}$ for GoAVSS. Compared to the ideal functionality in [GS23], the ideal functionality here does four additional things:

- first, it models the additional parameter $M > 0$ for "plain AVSS polynomials";
- second, it models adaptive corruptions;
- third, it separates out the notification to a party of a successful dealing and the delivery of that party's shares (which may come later);
- fourth, it allows individual shares to leak to the adversary before the dealer's input is locked.

As stated, $\mathfrak{F}_{\text{GoAVSS}}$ can really only be realized if we allow erasures. Here is the logic of $\mathfrak{F}_{\text{GoAVSS}}$, expressed as a collection of operations that may be invoked upon it:

- $\mathsf{Input}(f_1, \ldots, f_{L+M})$: this operation may be invoked once by an honest dealer D, who inputs polynomials $f_1, \ldots, f_{L+M} \in F[X]_{\leq t}$ to $\mathfrak{F}_{\text{GoAVSS}}$. In response, $\mathfrak{F}_{\text{GoAVSS}}$ sends the message $\mathsf{NotifyInput}(\{\mathcal{S}_\ell\}_{\ell \in [L]})$ to the ideal-world adversary, where $\mathcal{S}_\ell \coloneqq f_\ell(0)\mathcal{G}$ for $\ell \in [L]$.
- $\mathsf{Lock}()$: this operation may be invoked by the ideal-world adversary at a point in time that no Lock operation was previously invoked, the Input operation was previously invoked (D was honest when it invoked Input, but may be honest or corrupt at this point in time).

  Note that after D's input is locked, it cannot be changed even if D is later corrupted.
- $\mathsf{Lock}(f'_1, \ldots, f'_{L+M})$: this operation may be invoked by the ideal-world adversary at a point in time that no Lock operation was previously invoked, and that D is currently corrupt. The ideal-world adversary inputs $f'_1, \ldots, f'_{L+M} \in F[X]_{\leq t}$ to $\mathfrak{F}_{\text{GoAVSS}}$. In response, $\mathfrak{F}_{\text{GoAVSS}}$ sets $f_\ell \coloneqq f'_\ell$ for $\ell \in [L+M]$ and $\mathcal{S}_\ell \coloneqq f_\ell(0)\mathcal{G}$ for $\ell \in [L]$.

  This allows the ideal-world adversary to define the input polynomials of a corrupt dealer. If the dealer previously input polynomials at a time that it was honest, and those inputs were not already locked, this operation overwrites those inputs for purposes of the $\mathsf{ProvideShares}$ and $\mathsf{ProvideReady}$ operations. However, the original polynomials input by D are retained internally by $\mathfrak{F}_{\text{GoAVSS}}$ for use by the $\mathsf{LeakShares}$ operation.
- $\mathsf{ProvideReady}(i)$: This operation may be invoked once per $i \in [n]$ by the ideal-world adversary at any time after a Lock operation has been invoked. In response, $\mathfrak{F}_{\text{GoAVSS}}$ sends the message $\mathsf{Ready}(\{\mathcal{S}_\ell\}_{\ell \in [L]})$ to party P_i.

  Party P_i may be corrupt or honest at this time.
- $\mathsf{GetShares}()$: each party P_i may invoke this operation once after it has received its Ready message. In response, $\mathfrak{F}_{\text{GoAVSS}}$ sends the message $\mathsf{NotifyGetShares}(i)$ to the ideal-world adversary.

  Party P_i may be corrupt or honest at this time.
- $\mathsf{ProvideShares}(i)$: This operation may be invoked once per $i \in [n]$ by the ideal-world adversary at any time after receiving $\mathsf{NotifyGetShares}(i)$ from $\mathfrak{F}_{\text{GoAVSS}}$. In response, $\mathfrak{F}_{\text{GoAVSS}}$ sends the message $\mathsf{Shares}(\{f_\ell(e_i)\}_{\ell \in [L+M]})$ to party P_i.

  Party P_i may be corrupt or honest at this time — if P_i is corrupt, the output is effectively sent to the ideal-world adversary.

- LeakShares(i): this operation may be invoked once per $i \in [n]$ by the ideal-world adversary, at a point in time that P_i is currently corrupt and Input was previously invoked by D. In response, $\mathfrak{F}_{\text{GoAVSS}}$ sends the message Leakage($\{f_\ell(e_i)\}_{\ell \in [L+M]}$) to the ideal-world adversary, where $f_1, \ldots, f_{L+M}$ are the original polynomials input by D.

  This models the fact that we allow shares of an honest dealer's original polynomials to leak to corrupt parties. This may occur before or after the dealer's input is locked, and even if the adversary has overwritten the dealer's input via Lock($f'_1, \ldots, f'_{L+M}$). *The GoAVSS protocol in this section does not require this operation, but others might (such as the simplified protocol below in Section 8.2).*

Completeness. We do not attempt to define completeness in terms of an ideal functionality. Rather, it is property of a concrete protocol. For a GoAVSS protocol, we require the following (informally stated):

- If any honest party outputs Ready, or an honest dealer initiates the protocol, then eventually all honest parties output Ready.
- If all honest parties invoke GetShares, then eventually all honest parties obtain their shares.

5 Analysis of our GoAVSS protocol

5.1 The high level security proof

We want to prove that the protocol emulates $\mathfrak{F}_{\text{GoAVSS}}$. We do this by a series of games, starting with the "real world" and ending with the "ideal world".

Game 0

This is the "real world". All honest parties follow the protocol.

Game 1

Just as for the SMD protocol (Section 3.4), each run of the GoAVSS protocol falls into one of two cases, depending on the outcome of the CBC subprotocol that determines the Merkle root $\hat{\rho}$. Following the terminology introduced in Section 3.4, we call $\hat{\rho} = \rho$ an **honest dealing** and all other cases a **corrupt dealing** (encompassing both initially corrupt and transitional corrupt sub-cases).

In this game, we introduce the teleportation rule from Game 1 of Section 3.4 for honest dealings. That is, in the SMD subprotocol underlying the GoAVSS protocol, messages are delivered by teleportation rather than by actual decryption, exactly as described there.

In addition, the GoAVSS protocol includes several RBC subprotocols whose data is also committed under the top-level Merkle root ρ (the polynomials $\{h_r\}_{r \in [R]}$, the group elements $\{\mathcal{S}_\ell\}_{\ell \in [L]}$ and $\{\mathcal{T}_k\}_{k \in [K]}$, the scalars $\{c_k\}_{k \in [K]}$, and the polynomial h^*). This data is all public, so no privacy issues arise. However, by collision resistance of the Merkle hash, in an honest dealing, any such data that validates against ρ must be the data originally committed by the honest dealer. This is a straightforward correctness property, not requiring the teleportation machinery.

As in the SMD proof, Game 0 and Game 1 are indistinguishable except with negligible probability, bounded by the collision resistance of H_{ss} and the Merkle hash.

Game 2

Now consider a corrupt dealing. Suppose that some honest party receives $n-t$ votes in the one-sided vote subprotocol. This means at least $n-2t$ honest parties voted in that subprotocol.

In a corrupt dealing, all data processed by honest parties is either explicitly supplied by the adversary or, in the transitional corrupt sub-case, rejected outright because $\hat{\rho} \neq \rho$ (as discussed in Section 3.4). So the data used by honest parties is always explicit. By collision resistance of the Merkle hash and H_{ss}, this data is uniquely determined by $\hat{\rho}$, ensuring that the extraction of messages and the subsequent interpolation are well-defined.

In [SS23], it was shown how to extract the messages $(m_1, \ldots, m_n)$ of the underlying SMD protocol. We can do that here as well. Also as in [SS23], we identify a set of at least $n-2t$ "happy" parties. These are parties P_i (honest and corrupt) for which (6) holds. Let $f_1, \ldots, f_{L+M}$ be polynomials obtained by interpolation so that $f_\ell(e_i) = v_{\ell,i}$ for $\ell \in [L+M]$ for all happy parties P_i. We can also reconstruct the group elements $\mathcal{S}_\ell$ for $\ell \in [L]$ from the available data.

In this game, we abort execution of the protocol if any of the interpolated polynomials f_ℓ has degree $> t$ or if $f_\ell(0)\mathcal{G} \neq \mathcal{S}_\ell$ for any ℓ. We will argue separately below that we abort with negligible probability.

Game 3

This last step essentially gives us the simulator that we need in the ideal world. While Game 2 established soundness for corrupt dealings, this game establishes zero knowledge for initially honest dealings.

As in Game 2 of the SMD proof (Section 3.4.3), whenever the dealer is initially honest, we replace the real ciphertexts with random dummy values, and program the random oracle H_{enc} as there. The same argument shows that this change is undetectable.

Beyond the SMD-level changes, we also need to simulate the additional data in the GoAVSS protocol. We are assuming the dealer starts out as honest and we need to generate a Merkle tree without knowing the polynomials $f_1, \ldots, f_{L+M}$. To this end, we generate all of the secret keys and secret shares at random, and use dummy ciphertexts as above.

Looking at the dealer logic in Section 4.1, this gets us through Step 4. For Step 5, we just compute the h_r polynomials as random polynomials.

This brings us to Step 7.

- We set the group elements $\mathcal{S}_\ell = f_\ell(0)\mathcal{G}$ for $\ell \in [L]$ (as in the protocol).
- As for the group elements $\mathcal{T}_k$, here we need to do some extra work.

1. First, we compute random polynomials h'_k $(k \in [K])$.
2. Second, we generate the challenge $\{\gamma_{k,\ell}\}_{k\in[K],\ell\in[L]}$ at random.
3. Third, since we have:
$$h'_k = f_{L+M+k} + \sum_\ell \gamma_{k,\ell} f_\ell$$
We also have:
$$h'_k(0)\mathcal{G} = \mathcal{T}_k + \sum_\ell \gamma_{k,\ell} \mathcal{S}_\ell$$
So we solve this for $\mathcal{T}_k$ $(k \in [K])$.

This gets us to Step 9. Here, we program $H'_{\text{chall}}(\rho_3)$ to output $\{\gamma_{k,\ell}\}$.

This programming fails only if H'_{chall} was already queried on input ρ_3; since the Merkle tree rooted at ρ_3 contains a number of random leaves, ρ_3 is effectively a fresh random oracle output, so this happens with negligible probability.

The remaining steps can be carried out as in the protocol.

Note that whenever we need to leak the shares $\{v_{\ell,i}\}_{\ell\in[L+M]}$ to a corrupt party P_i, we also need to compute the shares $\{v_{\ell,i}\}$ for $\ell = L+M+1,\ldots,L+M+K+R$.

This is done in the usual way (as was already done in [SS23] and [GS23]). We have:

$$h'_k = f_{L+M+k} + \sum_{\ell\in[L]} \gamma_{k,\ell} f_\ell$$

So from $f_\ell(e_i)$ for $\ell \in [L]$ and h'_k, we can compute $f_{L+M+k}(e_i)$ for $k \in [K]$.

We also have:

$$h_r = f_{L+M+K+r} + \sum_{\ell\in[L+M+K]} \theta_{r,\ell} f_\ell$$

So from $f_\ell(e_i)$, where $\ell \in [L+M+K]$ and h_r, we can compute $f_{L+M+K+r}(e_i)$ for $r \in [R]$.

So now we have all of the shares we need, and we can program H_{enc} as needed.

The simulator

We now essentially have a simulator: it delivers messages by teleportation in honest dealings as in Game 1, extracts inputs for corrupt dealings as in Game 2, and simulates the initially honest dealer's data as in Game 3. In particular, for initially honest dealings, the simulator obtains the group elements $\mathcal{S}_\ell$, as well as shares belonging to corrupt parties, from the ideal functionality $\mathfrak{F}_{\text{GoAVSS}}$ and processes these as discussed in Game 3. If an initially honest dealer is subsequently corrupted, the simulator has nothing to produce, since the dealer has erased all data associated with the dealing (as discussed in Section 4.1).

We note that the LeakShares operation in $\mathfrak{F}_{\text{GoAVSS}}$ is never invoked by the simulator for this protocol. The SMD-based share distribution ensures that no individual shares leak before the dealer's input is locked: the sender-forward-secure channels protect the shares from leaking upon corruption of the sender, and each party's shares are only delivered after the lock point.

5.2 Soundness

In Game 2 of the security proof in Section 5.1, we aborted the execution of the protocol if we were unable to extract correct inputs to the protocol in a corrupt dealing. We argue here that this happens with negligible probability.

We stress that we are making this argument in a game that is just one step removed from the real-world execution. The only difference from this game and the real world is the introduction of the teleportation rule in Game 1. In particular, all random oracles used in the protocol are "real" random oracles — they are not being simulated in any way (as is eventually done in Game 3 of the security proof in Section 5.1). We also note that in making this soundness argument, we are considering only corrupt dealings, in which all data processed by honest parties is explicitly supplied by the adversary.

5.2.1 An interactive protocol

We start by considering the interactive protocol which is at the heart of our GoAVSS protocol. This interactive protocol is modeled as a protocol between a prover and a verifier. It consists of several rounds.

1. Prover sends to verifier $(m_1, \ldots, m_n)$, where each m_i is either $\perp$ or $\{v_{\ell,i}\}_{\ell \in [L+M+K+R]}$, with each $v_{\ell,i} \in F$.
2. Verifier sends to prover the challenge
$$\{\theta_{r,\ell}\}_{r \in [R], \ell \in [L+M+K]},$$
where each $\theta_{r,\ell}$ is a random element of F.
3. Prover sends to verifier $\{h_r\}_{r \in [R]}$, $\{\mathcal{S}_\ell\}_{\ell \in [L]}$, and $\{\mathcal{T}_k\}_{k \in [K]}$, with each $h_r \in F[X]_{\leq t}$, each $\mathcal{S}_\ell \in E$, and each $\mathcal{T}_k \in E$.
4. Verifier sends to prover
$$\{\gamma_{k,\ell}\}_{k \in [K], \ell \in [L]},$$
where each $\gamma_{k,\ell}$ is a random element of Γ.
5. Prover sends to verifier $\{c_k\}_{k \in [K]}$, with each $c_k \in F$.
6. Verifier sends to prover
$$\{\delta_k\}_{k \in [K]},$$
where each δ_k is a random element of F.
7. Prover sends $h^* \in F[X]_{\leq t}$ to verifier.

Let S be the set of indices $i \in [n]$ such that (6) holds. Let $f_1, \ldots, f_{L+M}$ be polynomials obtained by interpolation so that $f_\ell(e_i) = v_{\ell,i}$ for $\ell \in [L+M], i \in S$. Let S' be the subset of indices $i \in S$ such that (7), (8), and (9) hold. We say the prover *wins* if $|S'| \geq n - 2t$ and either

- $\deg(f_\ell) > t$ for some $\ell \in [L+M]$, or
- $\mathcal{S}_\ell \neq f_\ell(0)\mathcal{G}$ for some $\ell \in [L]$.

Using arguments from [SS23] and [GS23], we obtain a bound on the winning probability of

$$\frac{2^n}{q^R} + \epsilon + \frac{1}{q}, \tag{10}$$

where

$$\epsilon \leq \mathrm{Tail}(K, J, 1/|\Gamma|), \tag{11}$$

$$J \coloneqq \max\left(\left\lceil \frac{n-2t}{\lceil n/K \rceil} \right\rceil,\ K - \left\lfloor \frac{2t}{\lfloor n/K \rfloor} \right\rfloor\right), \tag{12}$$

and $\mathrm{Tail}(K, J, p)$ is the probability of getting at least J successes out of K Bernoulli trials with success probability p.

To see this, let us first extend the above definition of f_ℓ to all $\ell \in [L+M+K]$, using the same interpolation rule. Fix all of the values $v_{\ell,i}$ in Step 1. Let the challenge $\{\theta_{r,\ell}\}_{r,\ell}$ be randomly generated in Step 2, and let $\{h_r\}_r$ be the prover's response in Step 3. So now the set S and the polynomials f_ℓ are defined. As was argued in the proof of Theorem 5.1 in [SS23], the probability that $|S| \geq n-2t$ and some f_ℓ has degree greater than t is bounded by $2^n/q^R$. The term 2^n comes from taking a union bound over subsets $S \subseteq [n]$. As was discussed in [SS23], such an exponential factor cannot be avoided (at least without extra assumptions). Indeed, [SS23] give an explicit, simple example that illustrates this. This is in contrast to the analysis of a similar construction in [DN07]. However, [DN07] only considers the synchronous communication model and static corruption model. While [SS23] generally worked in the asynchronous/static model, the argument is general enough to apply in the asynchronous/adaptive model. Indeed, our formulation here does not take into account communication or corruption models; however, the fact that we are existentially quantifying over all sets S of size $n-2t$ will allow us to apply this to the asynchronous/adaptive model.

So if $|S| < n-2t$, the prover cannot possibly win, and if some f_ℓ has degree greater than t, we will generously credit the prover with the win. Otherwise, the prover will have to continue to the interaction to win. So now consider the values $\{\theta_{r,\ell}\}_{r,\ell}$, $\{h_r\}_r$, $\{\mathcal{S}_\ell\}_\ell$, $\{\mathcal{T}_k\}_k$ to be fixed. Assume that $\mathcal{S}_\ell \neq f_\ell(0)\mathcal{G}$ for some $\ell \in [L]$ (otherwise, the prover cannot win). The set S and the polynomials f_ℓ for $\ell \in [L+M+K]$ are fixed, and we are assuming that $|S| \geq n-2t$ and each f_ℓ has degree at most t. Now let the challenge $\{\gamma_{k,\ell}\}_{k,\ell}$ be randomly generated in Step 4. Let $\hat{S} \subseteq S$ be the set of indices i such that

$$h'_{\kappa(i)}(0)\mathcal{G} = \mathcal{T}_{\kappa(i)} + \sum_{\ell \in [L]} \gamma_{\kappa(i),\ell}\mathcal{S}_\ell, \tag{13}$$

where the polynomials h'_k are defined as in (5), and $\kappa(i)$ is defined just as we did right after (9). If $|\hat{S}| \geq n-2t$, we credit the prover with the win.

Note that the condition (13) depends on i only through $\kappa(i)$, and for distinct values of k, the checks are independent, each passing with probability at most $1/|\Gamma|$. Thus, $|\hat{S}| \geq n-2t$ requires that sufficiently many of the K independent checks pass, and the probability ϵ of this is bounded by $\mathrm{Tail}(K, J, 1/|\Gamma|)$, where J is the minimum number of passing checks needed.

In Theorem 5.2 of [GS23], it is argued that (11) holds with

$$J \coloneqq \left\lceil \frac{n-2t}{\lceil n/K \rceil} \right\rceil. \tag{14}$$

This bound is actually a bit suboptimal. It is easy to show that we can take J as in (12). To prove this, let $X_k \coloneqq \{i \in [n] : \kappa(i) = k\}$ for $k \in [K]$. Then J is the minimum number of these sets whose union has at least $n-2t$ elements. Let $Q \coloneqq \lceil n/K \rceil$ and $Q' \coloneqq \lfloor n/K \rfloor$. The sets $X_1, \ldots, X_K$ partition $[n]$ into K classes, each of size Q or Q'. Each class has at most Q elements, so J classes cover at most JQ, giving $J \geq \lceil (n-2t)/Q \rceil$. Each omitted class removes at least Q' elements, so we can omit at most $\lfloor 2t/Q' \rfloor$ classes, giving $J \geq K - \lfloor 2t/Q' \rfloor$.

If $|\hat{S}| < n-2t$, there is only one other way the prover can win. Note that if $|S'| \geq n-2t$, then since $n-2t > t$ (recall that $n > 3t$), conditions (7) and the fact that h^* and $\sum_{k\in[K]} \delta_k h'_k$ both have degree at most t force $h^*(0) = \sum_{k\in[K]} \delta_k h'_k(0)$ (since two polynomials of degree at most t that agree at more than t points must be identical); combined with (8), this gives $\sum_{k\in[K]} \delta_k(c_k - h'_k(0)) = 0$. Moreover, if $c_k = h'_k(0)$ for all k, then (9) reduces to (13), which would imply $S' \subseteq \hat{S}$, contradicting

$|\hat{S}| < n - 2t$. So the prover must choose $\{c_k\}_k$ in Step 5 with some $c_k \neq h'_k(0)$, and for a randomly chosen challenge $\{\delta_k\}_k$ in Step 6, the above equation can hold with probability at most $1/q$.

5.2.2 Moving to the real world

We move from the interactive protocol in Section 5.2.1 to the real world (or, more precisely, Game 1 in Section 5.1) as follows.

First, we replace random challenges generated by the verifier by random oracle outputs. Of course, this opens up a "grinding" attack vector. This changes the bound on the adversary's winning probability from (10) to

$$Q_{\text{ch}} \cdot \max\left(2^n/q^R, \epsilon, 1/q\right), \tag{15}$$

where Q_{ch} is a bound on the number of queries to the challenge oracle.

Second, we replace the messages sent to the verifier by the Merkle roots $\rho_1, \ldots, \rho_5$. Here, we use the fact that we are modeling the hash function used to build the Merkle tree as a random oracle, which means that we can effectively "extract" the messages needed in the interactive protocol from the adversary by looking at all of the queries to this random oracle. This is an "online" extraction — that is, it does not require any type of rewinding. We remind the reader that at this point in the security proof, we are not simulating any of the random oracles nor do we need to hide any random oracle queries made by honest parties from the adversary. Thus, to extract the messages, we can indeed look at *all* random oracle queries. In extracting these messages, we also need to take into account the erasure coding and encryption logic of the underlying SMD and RBC protocols. If the relevant preimage queries have not been made, we can safely assume they will never be made during the protocol execution. Thus, some messages extracted can end up being the "error" message $\perp$, either because some Merkle hash preimages have not been queried, or because the erasure coding and/or encryption logic fails for some other reason.

This online extraction procedure adds an error term of $O(Q^2_{\text{mrkl}}/N_{\text{mrkl}})$ to the adversary's winning probability (15), where Q_{mrkl} is a bound on the number of Merkle hash queries made and N_{mrkl} is the size of the output space of the Merkle hash.

Putting it all together, the probability that the abort condition in Game 2 in Section 5.1 is triggered is bounded by

$$Q_{\text{ch}} \cdot \max\left(2^n/q^R, \epsilon, 1/q\right) + O(Q^2_{\text{mrkl}}/N_{\text{mrkl}}). \tag{16}$$

5.3 Communication complexity

We estimate here the communication complexity of our GoAVSS protocol on the "happy path", where there are no complaints. Later, in Section 6.10, we will examine the complexity on the "unhappy path". We also assume $t \approx n/3$.

Let λ be the bit length of q. So elements of F can be encoded as bit strings of length λ. We denote by λ' the bit length of a compressed element of the group E and by λ'' the bit length of an uncompressed element of E. If E is an elliptic curve of order q (and cofactor 1), we can take $\lambda' = \lambda + 1$ and $\lambda'' = 2\lambda$.

Recall that κ is the output length of a collision resistant hash. In practice, with a curve such as secp256k1 and a hash function like SHA256, we can take $\kappa = \lambda = 256$, $\lambda' = 257$, and $\lambda'' = 512$. Below, in discussing amortized costs, we shall assume $\kappa, \lambda, \lambda', \lambda''$ are all within constant multiples of each other.

5.3.1 The SMD subprotocol

Consider the SMD subprotocol corresponding to Step 3 of our dealer logic Section 4.1. Each message m_i sent by the dealer through the SMD subprotocol has size

$$(L + M + K + R)\lambda.$$

The value L is the number of "GoAVSS polynomials" to be shared. We shall assume $L \gg n \log n$ to get good amortization. In practice, for n up to 100 or so, we would likely choose $L \approx 10\,000$. The other parameters M, K, R will be much smaller than L in our main application:

- M counts the number of "plain AVSS polynomials" to be shared. In our application (see Section 6.4 for details), M is bounded in terms of the maximum number of signature batches to be produced per presignature batch. In theory, we may set M as high as L, if we want to allow for signature batches of size 1. In practice, M will likely be a small constant ($M \approx$ 10–20).
- K will be at most n. As we are assuming $L \gg n \log n$, we can ignore this.
- R controls the soundness error of the underlying secret sharing protocol, as in (15). Certainly, R is well below n, and for most parameter settings it will be 1 or 2.

Because L dominates the other parameters M, K, R, we will simply estimate the size of each message m_i as $|m_i| \approx L\lambda$.

Based on the estimates in Section 3.2.1, the total communication complexity (among all parties) for the SMD subprotocol (to disseminate each message m_i to each P_i) is $6n\ell + O(\kappa n^2 \log n)$, where ℓ is a bound on the length of each individual message m_i. Using our estimate $\ell \approx L\lambda$ above, the contribution of the SMD subprotocol to the total communication complexity is

$$\approx 6nL\lambda + O(\kappa n^2 \log n) \tag{17}$$

5.3.2 Reliably broadcasting the group elements

Besides the dissemination of secret shares via the SMD subprotocol, the other major contributor to the communication complexity is the reliable broadcast of the group elements $(\{\mathcal{S}_\ell\}_{\ell\in[L]}, \{\mathcal{T}_k\}_{k\in[K]})$ corresponding to Step 8 of the dealer logic Section 4.1.

As above, we will assume $L \gg K$, so we can approximate the size of the message to be reliably broadcast as $L\lambda''$. Here, we are using the uncompressed group element size λ''. The reason for this (as we will discuss in more detail below in Example 5.1) is that otherwise, the computational cost of decompressing those points would dominate the computational cost of the receiver.

We can use the MiniCast protocol from [LS24] here to our advantage. As discussed in Section 2.3, the total communication complexity is

$$1.5n|m| + O(\kappa n^2 \log n).$$

Note that the constant 1.5 improves on the usual constant of 3 appearing in most earlier RBC protocols. Using $|m| \approx L\lambda''$, we get a total communication complexity for this step of

$$1.5nL\lambda'' + O(\kappa n^2 \log n).$$

A few details about incorporating MiniCast into our GoAVSS protocol are worth mentioning. The upshot is that incorporating MiniCast does not increase the "good case" latency of our GoAVSS protocol at all. The basic data flow of MiniCast is as follows. The message m is encoded into n fragments using an $(n, n-t)$ erasure code, and then each of these fragments is encoded into n "minifragments" using an $(n, n-2t)$ erasure code. A two-level Merkle tree is then constructed: the ith leaf of the top-level tree is the root of the Merkle tree whose leaves are the minifragments of the ith fragment. Party P_i receives the ith fragment and a top-level Merkle path, and then computes the minifragments and Merkle subtree, and validates the Merkle path it received relative to the computed Merkle subtree root. To reconstruct the message, each must receive $n-t$ fragments with appropriate Merkle paths. Once party P_i reconstructs the message, it can send each party P_j the ith minifragment of the jth fragment along with an appropriate Merkle path.

As presented in [LS24], MiniCast has four rounds: a "dissemination" round in which the leader sends each party a fragment plus Merkle path, an "echo" round in which each party broadcasts the Merkle root to agree upon a Merkle root; a "vote" round in which each party broadcasts their fragment plus Merkle path (waiting until this step ensures that parties need only broadcast a fragment that belongs to the agreed upon root); a "confirm" round in which each party constructs the message from $n-t$ fragments and sends minifragments with Merkle paths as described above. A party delivers the message upon receiving $n-t$ "confirm" messages. This ensures that each party can recover their own fragment from minifragments (if necessary) and then participate in the "vote" and "confirm" rounds (if they did not do so before).

The GoAVSS CBC subprotocol on the top-level Merkle root ρ can replace the "echo" round of MiniCast. Thus, the remaining three rounds fit within the 3-round budget we have allocated for all of the SMD and RBC protocols.

5.3.3 Other costs

The only other costs are the RBC of $\{h_r\}_{r\in[R]}$, the RBC of $\{c_k\}_{k\in[K]}$, and the RBC of h^*. Treating R as a small constant (typically 1 or 2, as discussed above), the size of each RBC message is at most $O(n\lambda)$, and so the total communication cost for these is $O(\kappa n^2 \log n)$.

5.3.4 Total communication cost

We conclude that the total communication cost for GoAVSS on the "happy path" is

$$\approx 6nL\lambda + 1.5nL\lambda'' + O(\kappa n^2 \log n). \tag{18}$$

Assuming $L \gg n^2 \log n$, the amortized communication cost per shared polynomial is

$$\approx 6n\lambda + 1.5n\lambda''. \tag{19}$$

5.3.5 Communication cost per presignature per party

Looking forward, we will generate a batch of presignatures by having each party run the GoAVSS protocol as a dealer, and we will "harvest" a total of $\approx \frac{n}{3}L$ presignatures using a batch randomness extraction technique. This leads to an amortized communication cost *per presignature per party* of

$$\approx 18\lambda + 4.5\lambda''. \tag{20}$$

With λ equal to 32 bytes and λ'' equal to 64 bytes (as is typical), this is a total of 864 bytes per presignature per party, or roughly 7K bits. However, for actually producing signatures, there are other communication costs to account for, as we will discuss below Section 6.8.

5.4 Computation cost

We estimate here the computational complexity of our GoAVSS protocol on the "happy path", where there are no complaints. As in our estimates for communication cost, we assume $t \approx n/3$, and that the parameters M, K, R are much smaller than L.

5.4.1 Algebraic costs per shared polynomial

We first consider algebraic costs: operations over the field F and the group E. We will compute these as amortized costs per shared polynomial.

Recall the set Γ of coefficients from our GoAVSS protocol. We shall assume that

$$\Gamma = \{0, \ldots, 2^g - 1\}. \tag{21}$$

Typically, g will be much smaller than the bit length of q.

To measure the cost of various algebraic computations, we use the terminology introduced in [GS23].

Full scalar/group multiplication: This is a scalar/group multiplication, where the scalar is a secret, full-sized element of F, and the group element is the generator $\mathcal{G}$. This means that (a) we can use precomputation on $\mathcal{G}$ to make the algorithm faster, but (b) we have to be careful to use a constant-time algorithm. For typical parameter settings and implementations, we can estimate the cost of a full scalar/group multiplication to be about 64 group additions (see Appendix A of [GS23]).

Tiny scalar/group multiplication: This is a scalar/group multiplication where the scalar is a public, random element of Γ, and the group element is variable and public.

Moreover, the resulting products are only used as terms in a long summation (of length $\approx L$), so special optimized algorithms may be used. Using Pippenger's simple "bucket method" for typical parameter settings, the amortized cost of each tiny/scalar group multiplication is just 1 or 2 group additions (see Appendix B of [GS23]).

Tiny scalar/scalar multiplication: This is a scalar/scalar multiplication where one scalar input is a public, random element of Γ, and the other scalar input is a secret, full-sized element of F.

Moreover, the resulting products are only used as terms in a long summation (of length $\approx L$), so special optimized algorithms may be used. In particular, the cost of such an operation is essentially that of multiplying a g-bit integer by a λ-bit integer (without a modular reduction). For typical parameter settings, the amortized cost of each tiny scalar/scalar multiplication is just a very small fraction (less than 1/40) of the cost of a group addition.

In addition to this, we count:

Full scalar/scalar multiplication: The resulting products are only used as terms in a long summation (of length $\approx L$). In particular, we can ignore the cost of reducing mod q (which can be done once at the end of the summation).

Lazy scalar addition: This operation is an addition (or subtraction) that takes two numbers in the range $[-q, q)$ and produces their sum (or difference) mod q in the range $[-q, q)$. This is only used in the finite differences technique discussed in Section A (see, in particular, Section A.8) for polynomial extension.

Dealer costs. The amortized cost per shared polynomial for a party in the role of dealer is as follows:

- $\approx n^2/3$ lazy scalar additions to compute all of the shares $v_{\ell,i}$ as in (1), as in Section A.
- $\approx Rn/3$ full scalar/scalar multiplications to compute all of the polynomials h_r as in (2).
- ≈ 1 full scalar/group multiplication to compute all of the group elements $\mathcal{S}_\ell$ as (3).
- $\approx Kn/3$ tiny scalar/scalar multiplications to compute the polynomials h'_k as in (5).

Receiver costs. The amortized cost per shared polynomial for a party in the role of receiver is as follows:

- $\approx R$ full scalar/scalar multiplications for the verification in (6).
- $\approx K$ tiny scalar/scalar multiplications for the verification in (7).
- ≈ 1 tiny scalar/group multiplication for the verification in (9).

5.4.2 Algebraic costs per presignature

Again, looking forward, we will generate a batch of presignatures by having each party run the GoAVSS protocol as a dealer, and we will "harvest" a total of $\approx \frac{n}{3}L$ presignatures using a batch randomness extraction technique. This leads to an amortized algebraic cost *per presignature per party* as follows.

- $\approx 4K$ tiny scalar/scalar multiplications.
- $\approx n$ lazy scalar additions.
- $\approx 4R$ full scalar/scalar multiplications.
- ≈ 3 tiny scalar/group multiplications.
- $\approx 3/n$ full scalar/group multiplications.

Example 5.1. *We take as an example $t = 16$, $n = 3t + 1 = 49$, $K = n$. We will use* secp256k1 *as the group E, so $\lambda = 256$. For these settings, $R = 1$ is sufficient. We also set $L = 10\,000$ in running relevant benchmarks.*

All benchmarks in this paper are run on a single performance core of a MacBook Pro (Mac17,2):W with an Apple M5 chip (ARM, 4.6 GHz), unless otherwise noted.

We can set $g = 14$ or $g = 16$. Using just the soundness error bound (11), even with the simple bound (14), gives us an error bound of 2^{-195} (with $g = 14$) or 2^{-229} (with $g = 16$). Even the larger error bound of 2^{-195} is small enough to account for grinding attacks and still leave plenty of residual security.

With either of these values of g, we can use Pippenger's "bucket method" with a window of size 7 or 8, so that the amortized cost of 1 tiny scalar/group multiplication is just 2 group additions.

Note that to reduce this cost to a single group addition, and to still reach 128-bit security, we would need to set $g = 11$ and use a window of size 11, which may not work as well in practice as two windows of smaller size.

For arithmetic in secp256k1, we used the libsecp256k1 library (see https://github.com/bitcoin-core/secp256k1). On our Apple M5, we have benchmarked the various algebraic costs as follows.

- *1 tiny scalar/scalar multiplication:* 0.6 ns.
- *1 lazy scalar addition:* 2 ns.
  - *A tiny scalar/scalar multiplication is actually cheaper on our machine than a lazy scalar addition, because the latter performs a "lazy reduction" in each step.*
- *1 full scalar/scalar multiplication:* 5 ns.
- *1 tiny scalar/group multiplication:* 130 ns.
  - *We estimate this as twice the time of group_add_affine_var in libsecp256k1.*
- *1 full scalar/group multiplication:* 5200 ns.
  - *We estimate this as the time of ecmult_gen in libsecp256k1.*

So then we have:

- 118 ns *for* $\approx 4K$ *tiny scalar/scalar multiplications.*
- 98 ns *for* $\approx n$ *lazy scalar additions.*
- 20 ns *for* $\approx 4R$ *full scalar/scalar multiplications.*
- 390 ns *for* ≈ 3 *tiny scalar/group multiplications.*
- 318 ns *for* $\approx 3/n$ *full scalar/group multiplications.*

So the total time is ≈ 1 µs. *This is the total amortized time for algebraic computations per presignature per party.*

We also note that the time to compute a square root in the field (operation field_sqrt in libsecp256k1) is roughly 2 µs. *Compare this to the time of one tiny scalar/group multiplication, which is* 130 ns. *If we were to use compressed group elements and needed to decompress them, this decompression cost would dominate.*

Finally, the cost of one group addition is between 65 ns *and* 90 ns, *so at* 0.6 ns, *a tiny scalar/scalar multiplication is less than* $1/100$ *of a group addition — well within the* $1/40$ *bound claimed above.*

5.4.3 Other computational costs

Erasure coding costs. Consider one run of the GoAVSS protocol.

To transmit the shares through the SMD subprotocol, the leader has to take n messages each of length $\approx L\lambda$, and encode each using an $(n, n-2t)$-erasure code. Each receiver will have to run the corresponding decoding and encoding algorithms once.

To transmit the group elements in the RBC subprotocol of [LS24], the sender will have to encode one message of length $\approx L\lambda''$ using an $(n, n-t)$-erasure code into n fragments, and then encode each such fragment using an $(n, n-2t)$-erasure code into n minifragments. Each receiver will run the $(n, n-t)$ decoding algorithm once, and then run the $(n, n-t)$ encoding algorithm once and the $(n, n-2t)$ encoding algorithm n times.[5] The receiver may also run the $(n, n-2t)$ decoding algorithm once as well.

Example 5.2. *We continue here with Example 5.1. For erasure coding, we used the* reed-solomon-simd *library (see* `https://github.com/AndersTrier/reed-solomon-simd`*).*

- *Consider the coding associated with the SMD.*

  For the $(n, n-2t)$ code, encoding a message of length $L\lambda$ ($= 320\,000$ bytes) takes under $E = 95\,\mu s$, while decoding takes under $D = 380\,\mu s$.

  So per L polynomials, the dealer cost is $\approx nE$ and the receiver cost is $\approx D+E$. The amortized cost per L presignatures is roughly the dealer cost times $3/n$ plus the receiver cost times 3, so $\approx 3E + 3D + 3E = 6E + 3D = 1710\,\mu s$.

  Dividing by $L = 10\,000$, we get $0.17\,\mu s$ per presignature per party for the SMD encoding and decoding costs.

- *Consider the cost associated with the RBC.*

  For the $(n, n-t)$ code, encoding a message of length $L\lambda''$ ($= 640\,000$ bytes) takes under $E = 95\,\mu s$, while decoding takes under $D = 420\,\mu s$.

  For the $(n, n-2t)$ code, encoding a message of length $L\lambda''/(n-t)$ ($\approx 20\,000$ bytes) takes under $E' = 6\,\mu s$, while decoding takes under $D' = 180\,\mu s$.

  So per L polynomials, the dealer cost is $\approx E + nE'$ and the receiver cost is $D + E + nE' + D'$. The amortized cost per L presignatures is roughly the dealer cost times $3/n$ plus the receiver cost times 3, so $\approx 3E + 3(n+1)E' + 3D + 3D' = 2985\,\mu s$.

  Dividing by $L = 10\,000$, we get $0.30\,\mu s$ per presignature per party for the RBC encoding and decoding costs.

Hashing costs. Consider one run of the GoAVSS protocol.

To transmit the shares through the SMD subprotocol, the leader has to hash n objects, each $\approx 3L\lambda$ bits in length. Each such object is composed of n equal sized strings which are hashed separately (generally, the smaller the strings, the greater the hashing cost per bit, as fixed overheads start

[5] Note that naively, the receiver may have to run the $(n, n-2t)$ encoder a total of $2n$ times: n times when it is downloading and verifying Merkle paths, and n times when it is re-encoding the decoded result to validate the Merkle root. However, there is no need to run the encoder on the same fragment in each of these two phases, unless we detect that the downloaded Merkle path in the first phase was invalid, in which case we know the sender of that path is corrupt and can ignore them in the future. So in a steady state, these numbers are correct.

to increase that cost). Each receiver will effectively hash the equivalent of 2 such objects — once while validating and relaying Merkle paths from the dealer, and once while collecting/validating Merkle paths to obtain its shares.[6]

To transmit the group elements in the RBC subprotocol of [LS24], the sender will have to hash an object $\approx 4.5L\lambda''$ bits in length. Each such object is composed of n^2 equal sized strings which are hashed separately (these are the mini-fragments that make up the leaves of the 2-level Merkle tree in [LS24]). Each receiver will effectively have to hash the equivalent of (roughly) one such object.[7]

Example 5.3. *We continue here with Example 5.2. For hashing, we benchmarked* sha256 *using* openssl.

- *Consider the hashing associated with the SMD.*

  For these parameters, we measured the time to hash one object as above as $H = 273\,\mu s$.

  So per L *polynomials, the dealer cost is* $\approx nH$ *and the receiver cost is* $\approx 2H$. *The amortized cost per* L *presignatures is roughly the dealer cost times* $3/n$ *plus the receiver cost times* 3, *so* $\approx 3H + 6H = 9H = 2457\,\mu s$.

  Dividing by $L = 10\,000$, *we get* $0.25\,\mu s$ *per presignature per party for the SMD hashing costs.*

- *Consider the cost associated with the RBC.*

  For these parameters, we measured the time to hash one object as above as $H' = 950\,\mu s$.

  So per L *polynomials, the dealer cost is* $\approx H'$ *and the receiver cost is* $\approx H'$. *The amortized cost per* L *presignatures is roughly the dealer cost times* $3/n$ *plus the receiver cost times* 3, *so* $\approx 3H' = 2850\,\mu s$.

  Dividing by $L = 10\,000$, *we get* $0.29\,\mu s$ *per presignature per party for the RBC hashing costs.*

At this point, combining all the algebraic, coding, and hashing costs, we are at $1 + 0.17 + 0.30 + 0.25 + 0.29 \approx 2\,\mu s$ *per presignature per party.*

PRG costs. One cost we have ignored in the above analysis is that of a PRG, which is needed in a few places. This PRG can be implemented using AES counter mode. Many modern processors have fast hardware support for AES which make these costs negligible relative to other computational costs, and so it is safe to ignore them (as we shall demonstrate by looking at some benchmarks).

One place is the encryption operation $Encrypt(k, m) = H_{\text{enc}}(k) \oplus m$ used in the SMD protocol. Here, we would use AES counter mode to implement $H_{\text{enc}}(k)$. Note that in this construction, we need to model $H_{\text{enc}}(k)$ as a random oracle. This can be justified by viewing AES as an ideal cipher.

Another place is in the derivation of challenges in the GoAVSS protocol itself. These challenges are vectors of scalars — either pseudorandom elements of F or elements of $\Gamma \subseteq F$. In typical applications, a dealer will also generate pseudorandom elements of F to form its polynomial inputs to GoAVSS.

[6] Just as we did above, we can assume that in the typical case, the same fragment is not hashed both in the downloading and re-encoding phases.

[7] Again, we can assume that in the typical case, the same fragment is not hashed both in the downloading and re-encoding phases.

Example 5.4. *For example, on our Apple M5, we benchmarked AES-256 to run at roughly* 1.4 ns *per 16-byte block.*

We implemented and benchmarked the computation of $\sum_{\ell\in[L]} \gamma_{k,\ell} \cdot v_{\ell,i}$ *for all* $k \in [K]$ *that appears in the check (7). For each* k*, this computation requires the derivation of the* L *"tiny" (16 bit, in our running example) coefficients* $\gamma_{k,\ell} \in \Gamma$ *for* $\ell \in [L]$ *and* L *tiny scalar/scalar multiplications. This computation is also one of the points in the computation that makes the heaviest use of the PRG, especially when* $K = n$ *as in our running example. Our implementation computes four independent linear combinations in parallel, sharing the same vector* $\{v_{\ell,i}\}$ *of 256-bit scalars, but using independent coefficient streams generated by AES-256 in counter mode, each under an independent 256-bit key. Each AES block yields eight 16-bit coefficients and the inner loop is software-pipelined: the next batch of AES blocks is encrypted while the current batch of coefficients is consumed by the multiply-accumulate chain. Since the AES operations use the cryptographic/NEON pipeline while the multiply-accumulate uses the integer multiply units, the out-of-order engine overlaps the two phases effectively. For* $L = 10\,000$*, the per-sum cost of the combined PRG generation and linear combination is approximately* 0.66 ns *per term, compared to* 0.59 ns *for the multiply-accumulate alone — the AES-256 PRG adds less than 12% overhead. Note that the software pipeline reduces the effective cost of generating 16 pseudorandom bits from* $\approx 1.4\,\text{ns}/8 = 0.175\,\text{ns}$ *to* $0.66\,\text{ns} - 0.59\,\text{ns} = 0.07\,\text{ns}$.

6 Signature production pipeline and player elimination

In this section, we describe a **signature production pipeline** that is tightly integrated with a **player elimination framework**. The goal is to ensure that if any corrupt party causes the protocol to deviate from its "happy path" (where the amortized communication complexity per signature is linear), that party will be identified and eliminated, and moreover, this will happen after only a bounded amount of extra communication and a bounded number of signing requests have been processed.

6.1 Overview

We begin with a high-level overview of the pipeline, which consists of several stages. This overview serves as a roadmap that will help orient the reader as the details are developed in subsequent subsections.

Stage 1: Key setup. The parties use a complete AVSS protocol (such as that in [SS23]) together with a variation of the GoAVSS DKG protocol from [GS23] to establish a sharing of the secret key x along with the corresponding public key $\mathcal{X} = x\mathcal{G}$. This setup produces an *unbiased* key, which is essential for certain applications (such as ECDSA). The details of this setup are discussed in Section 6.2.

Stage 2: Presignature batch production. Presignatures are produced in batches. To produce a batch, each party P_i acts as a dealer and runs an instance of the GoAVSS protocol to share a collection of random polynomials. An ACS protocol instance (see Section 2.7), denoted DealerACS, is then used to agree on a set S of $n-t$ dealers whose GoAVSS instances have successfully completed.

Using the batch randomness extraction technique from [GS23], the parties then locally compute $(n-2t)L$ presignatures from the polynomials shared by the dealers in S.

Importantly, during presignature batch production, parties do *not* yet process complaints against dealers. Complaints may be registered (via RBC instances described below), but their resolution is deferred. This design ensures that any complaint processing that does occur will lead to player elimination, as we explain below.

Stage 3: Signature batch production. Once presignatures are available, they may be consumed to produce signatures. We assume that signing requests are ordered by some consensus mechanism (e.g., a blockchain), which also pairs each signing request with a presignature from an available presignature batch. Signatures are produced in batches, where each signature batch uses a subset of presignatures from a single presignature batch.

To produce a signature batch, we employ:

- the technique of **batch presignature re-randomization** from [Sho23a], which requires opening a single random sharing not associated with any group element;
- **online error correction** to handle potentially incorrect shares contributed by corrupt parties; and
- **batch public reconstruction** to reconstruct the scalar components of the signatures.

Stage 4: Presignature batch finalization. When a party completes the construction of all signatures belonging to a presignature batch, it encodes the full collection of signatures using an erasure-coded RBC-like mechanism and inputs the resulting commitment to an RA protocol instance (denoted PresigBatchResultsRA). When this RA instance outputs for a party, that party knows that the signatures are available: any party can reconstruct them by requesting fragments from other parties. At this point, the party **abandons** the presignature batch and proceeds to **finalize** it. This is where complaints are adjudicated and corrupt parties are identified and eliminated.

Each party P_i reliably broadcasts a set Supp_i of parties whose complaints it supported during the lifetime of the presignature batch. An ACS protocol instance (denoted FinishACS) is then used to agree on a set of parties, where a party is considered valid for this ACS if it has reliably broadcast its Supp set and each party in that set has reliably broadcast a complaint.

Once this ACS completes, the parties process the complaints that were registered by parties in the agreed-upon set. For each such complaint, the complainer's message from the original dealing is opened for all to see, allowing all parties to determine whether the complaint was valid (the dealer was corrupt) or invalid (the complainer was corrupt). The corrupt parties thus identified are eliminated.

Finally, an RA protocol instance (denoted PresigBatchFinalityRA) is used as a "finality gadget," after which parties need only retain:

- the set of eliminated parties,
- the protocol messages associated with PresigBatchFinalityRA,
- the signatures produced,

- the protocol messages associated with the PresigBatchResultsRA instance, as well as messages associated with the corresponding RBC-like mechanism to deliver signatures.

All other resources, including the Complain RBC instances, may be garbage collected.

Communication cost structure. Before describing the complaint mechanism, it is helpful to understand the cost structure of the protocol. Certain operations contribute only an **additive overhead** that is independent of the batch size L: the various RBC instances (for complaints, support sets, etc.), the RA instances (for presignature batch finalization), and the ACS instances (for agreeing on dealers and finishing batches). The only operations that are truly expensive (scaling with L) are those that require messages from the **secure message distribution** subprotocol to be delivered. This includes both complaint *processing* (delivering messages to help a complainer reconstruct shares) and complaint *adjudication* (delivering messages to determine whether a complaint was valid). The central design goal is to ensure that whenever these expensive message-delivery operations occur, they lead to the identification and elimination of all the corrupt parties that caused them to occur.

The complaint mechanism. Threading through the above stages is a complaint mechanism that allows parties who did not receive valid shares from a dealer to eventually obtain them. This mechanism is critical to ensuring liveness while also enabling player elimination.

For each presignature batch, an RBC instance $\mathsf{Complain}_i$ is associated with each party P_i acting in its role as a receiver. Once the dealer set S is agreed upon via DealerACS, party P_i may reliably broadcast (using $\mathsf{Complain}_i$) a subset $S_i \subseteq S$ of dealers against which it wishes to complain.

When a party P_j outputs S_i from the $\mathsf{Complain}_i$ instance, it sends a $\mathsf{support}_i$ message to all parties, *provided* P_j has not yet abandoned the presignature batch. By sending this message, P_j commits to including P_i in its Supp_j set at the presignature batch finalization stage.

A complaint is processed (i.e., messages from the secure message distribution subprotocol are delivered to validate the complaint and to help the complainer reconstruct valid shares) only when it has received $n - t$ support messages. This gating mechanism ensures two properties:

- **Liveness:** If an honest party genuinely needs complaint resolution to participate in signature production, its complaint will receive $n - t$ support (since honest parties who have not abandoned the batch will support it), and the complaint will be processed.
- **Accountability:** If a complaint triggers expensive message delivery (whether for processing or later adjudication), then by a quorum intersection argument, FinishACS will include a party whose Supp set includes the complainer, ensuring that the complaint is adjudicated at presignature batch finalization, leading to identification and elimination of a corrupt party.

Note that a complaint might *not* receive $n - t$ support messages, in which case it is not processed. However, this can only happen if all honest parties have already abandoned the presignature batch, which in turn means that the PresigBatchResultsRA instance has already output for those parties. At that point, the signatures are available via the data recovery mechanism, so the complainer no longer *needs* to reconstruct its shares to participate in signature production. Such a complaint may still be *adjudicated* at presignature batch finalization (if it appears in some party's Supp set), which does incur the expensive message-delivery cost, but this cost is justified by the resulting player elimination.

Rate limiting. The system should not allow presignature production to run too far ahead of presignature consumption. Specifically, the protocol enforces a configurable bound on the number of presignature batches that have not yet been finalized. If this bound is reached, parties pause presignature production until extant batches are finalized. Conversely, if signing requests arrive slowly, signature batches may be made smaller (even of size 1 if necessary). This ensures that a corrupt party can "poison" at most a bounded number of presignature batches before being eliminated.

Key invariant. The central property of our design is this: *any corrupt behavior that causes communication complexity to exceed the happy-path bound will result in the identification and elimination of all corrupt parties that contributed to the extra communication, and this will happen within a bounded number of signing requests.* The argument hinges on the role of support messages as a gate for expensive operations. Complaint processing — the only operation whose communication cost scales with L — is triggered only when a complaint receives $n-t$ support messages. Since at most t of these come from corrupt parties, at least $n-2t$ honest parties must have supported the complaint, and each commits to including the complainer in its Supp set. Since FinishACS outputs $n-t$ parties, a standard quorum intersection argument (using $n > 3t$) guarantees that at least one party in the output includes the complainer in its Supp set. Therefore, the complaint will be adjudicated at presignature batch finalization, and a corrupt party (either the dealer or the complainer) will be identified and eliminated. This is in contrast to the complaint mechanism in [SS23], which guarantees eventual elimination but without any a priori bound on when elimination occurs or (in the application to threshold signing in [GS23]) how many signing requests are affected.

6.2 Key setup

The key setup stage establishes a t-out-of-n sharing of the secret key $x \in F$ along with the public key $\mathcal{X} = x\mathcal{G} \in E$. For threshold Schnorr signatures, a biased key suffices. Indeed, the analyses in [Sho23a] that give a reduction from forging to attacking an interactive identification scheme all carry over immediately to biased keys. However, the same cannot be said for the generic group model analyses in [Sho23a] (although we believe these analyses could be modified to deal with biased keys). Moreover, for other applications (such as threshold ECDSA), an *unbiased* key is required (see Section 3.6 of [GS22]).

A biased key can already be generated efficiently using the techniques in [GS23]. However, we shall assume the key setup uses a complete AVSS protocol together with a DKG construction that produces an unbiased key. This will allow us to directly use the security analyses in [Sho23a]. The details of this construction are deferred to Section 7.

6.3 Presignature batch production

We now describe the production of presignature batches in detail.

Dealer phase. Each party P_i acts as a dealer and runs an instance of the GoAVSS protocol from Section 4. Party P_i shares:

- L GoAVSS polynomials, whose secrets will become (after batch randomness extraction) the random scalars $\bar{r}$ in presignatures $([\bar{r}], \bar{\mathcal{R}})$, with $\bar{\mathcal{R}} = \bar{r}\mathcal{G}$ published; and

- M plain AVSS polynomials, whose secrets will be used (again, after batch randomness extraction) for presignature re-randomization.

The specific values of L and M are discussed below.

Agreement on dealers. An ACS protocol instance DealerACS is used to agree on a set $S \subseteq [n]$ of $n - t$ dealers whose GoAVSS instances have successfully completed (i.e., for which all honest receivers have output Ready). This ACS uses an **asynchronous validity predicate** as in [Sho24]: a dealer is valid if its GoAVSS instance has successfully completed. While this validity predicate is evaluated locally by each party, it has the property that if the predicate becomes true for one honest party, it will eventually become true for all honest parties.

Complaint registration. Once DealerACS outputs the dealer set S, each party P_i may register complaints against dealers in S from whom it did not receive valid shares. Specifically, P_i reliably broadcasts (via the RBC instance $\mathsf{Complain}_i$) the set $S_i \subseteq S$ of dealers against which it complains.

At this point, complaints are only *registered*, not processed. Processing (which involves delivering additional messages associated with secure message distribution protocol instances) occurs only when a complaint receives sufficient support, as described in Section 6.5.

Batch randomness extraction. Using the batch randomness extraction technique from [GS23], the parties locally compute presignatures from the shared polynomials. Let W be an $(n-2t)\times(n-t)$ super-invertible matrix over F (e.g., a Pascal matrix). Writing $S = \{i_1, \ldots, i_{n-t}\}$, the parties compute $n - 2t$ biased presignatures $([\bar{r}_1], \bar{\mathcal{R}}_1), \ldots, ([\bar{r}_{n-2t}], \bar{\mathcal{R}}_{n-2t})$ where

$$\begin{pmatrix} \bar{r}_1 \\ \vdots \\ \bar{r}_{n-2t} \end{pmatrix} = W \cdot \begin{pmatrix} r_{i_1} \\ \vdots \\ r_{i_{n-t}} \end{pmatrix}$$

and similarly for the group elements $\bar{\mathcal{R}}_j = \bar{r}_j \mathcal{G}$. Here, r_i denotes the secret contributed by dealer P_i. This extraction is performed independently for each of the L GoAVSS polynomials, yielding a total of $(n - 2t)L$ biased presignatures per batch.

The shares $[\bar{r}_j]$ are computed locally using linear operations on the shares $[r_i]$ for $i \in S$, and the group elements $\bar{\mathcal{R}}_j$ are computed directly from the published group elements $\mathcal{R}_i$ for $i \in S$.

A similar extraction is performed on the plain AVSS polynomials to obtain sharings of random scalars for re-randomization.

6.4 Signature batch production

Once presignatures are available, the parties may produce signatures in batches.

Consensus on signing requests. In a typical application in the blockchain setting, smart contracts would decide what messages are to be signed and when. So we assume an external consensus mechanism orders signing requests. Moreover, we assume that this mechanism also pairs each request with a presignature from an available batch. This pairing is essential for security: using the same presignature to sign two different messages would allow signature forgery.

Batch presignature re-randomization. For a signature batch, let $([\bar{r}_1], \bar{\mathcal{R}}_1), \ldots, ([\bar{r}_B], \bar{\mathcal{R}}_B)$ be the biased presignatures assigned to the B signing requests in the batch. The parties open a single shared random value $\delta \in F$ (from the plain AVSS polynomials in the presignature batch) and compute re-randomized presignatures:

$$\mathcal{R}'_k \leftarrow \bar{\mathcal{R}}_k + \delta\mathcal{G} \quad \text{and} \quad [r'_k] \leftarrow [\bar{r}_k] + [\delta] \quad \text{for } k \in [B].$$

This re-randomization, performed using a value δ that is revealed only after the signing requests are fixed, ensures security even though the presignatures $\mathcal{R}_k$ were revealed to the adversary before the messages were known. See [Sho23a] for the security analysis.

Online error correction and reconstruction. For each signing request m_k with re-randomized presignature $([r'_k], \mathcal{R}'_k)$ and public key $\mathcal{X}$, the parties compute the hash $h_k = H_{\text{sign}}(\mathcal{X}, \mathcal{R}'_k, m_k)$ and then compute shares of the signature scalar:

$$[s_k] \leftarrow [r'_k] + h_k \cdot [x].$$

These shares are then publicly reconstructed using online error correction to handle potentially incorrect shares from corrupt parties. The resulting signatures are $(\mathcal{R}'_k, s_k)$ for $k \in [B]$. See [BOCG93, Can96] for online error correction and [DN07, CP17] for batch public reconstruction.

Additive key derivation. The protocol can also be easily adapted to support **additive key derivation**. Here, an individual signing request m_k may include an additive "tweak" $\epsilon_k \in F$, so that the effective secret key is $x'_k \coloneqq x + \epsilon_k$ and the effective public key is $\mathcal{X}'_k \coloneqq \mathcal{X} + \epsilon_k\mathcal{G}$. As this is a purely linear operation, the shares $[s_k]$ are easily computed. Of course, the public key $\mathcal{X}'_k$ also needs to be computed and passed to H_{sign}, which adds to the signing cost (although these can perhaps be precomputed).

This type of additive key derivation was used in [GS22] in the context of threshold ECDSA signatures, and analyzed in [Sho23a] in the context of Schnorr signatures. It effectively allows a single signing key to be used to cheaply derive many signing keys, in such a way that obtaining signatures under one derived key does not allow an attacker to forge signatures under another derived key.

Batch size. Note that the size of a signature batch may vary dynamically. Under heavy load, the size can be large. Under light load, the size can be small (down to size 1 if necessary). Thus, processing signing requests in batches does not need to add any extra delay.

Optional per-signature-batch RA. We note that on the normal path (where there are no complaints), each honest party computes signatures locally as signing requests arrive, so signatures are available to honest parties without waiting for any RA protocol. The PresigBatchResultsRA instance serves primarily as a data recovery mechanism for parties that fall behind or need to catch up. If the surrounding application requires earlier certification of individual signature batches — for example, to post a data availability certificate to a blockchain for each batch as it completes — one could additionally run a separate RA instance for each signature batch. However, this is not needed for the core protocol and we do not pursue it further here.

6.5 Complaint mechanism details

We now describe the complaint mechanism in detail.

Complaint registration. As described in Section 6.3, once DealerACS outputs the dealer set S, each party P_i may reliably broadcast its complaint set $S_i \subseteq S$ via the RBC instance $\mathsf{Complain}_i$. We assume that a complaint set S_i is always nonempty — empty complaint sets need not and should not be broadcast.

Support messages. When a party P_j outputs a nonempty set S_i from $\mathsf{Complain}_i$, it sends a $\mathsf{support}_i$ message to all parties, *provided* P_j has not already abandoned the presignature batch. By sending this message, P_j commits to including P_i in its Supp_j set at presignature batch finalization.

A party only abandons a presignature batch when the PresigBatchResultsRA instance for that presignature batch has output. This ensures that support messages are available whenever an honest party needs complaint resolution to produce signatures.

Complaint processing. A party processes the complaints in S_i only after the following precondition is satisfied:

- it has output S_i from $\mathsf{Complain}_i$; and
- it has received $n - t$ corresponding $\mathsf{support}_i$ messages.

To process a complaint by receiver P_i against dealer $P_j \in S_i$, party P_i forwards message m_i (the message that P_j sent to P_i in P_j's GoAVSS dealing) to all parties.

Upon obtaining m_i, each party can verify whether the complaint is valid by checking:

- $m_i = \perp$; or
- equation (6) does not hold for m_i.

If either condition holds, the complaint is valid, indicating that dealer P_j sent invalid shares to P_i.

Assisting the complainer. If a party P_k sees a valid complaint by P_i against P_j, then P_k forwards its message m_k from P_j's dealing to P_i. Note that party P_k only does this if the precondition for processing complaints in S_i is satisfied and its own message m_k is valid. Once P_i receives $t + 1$ such valid messages, it can interpolate to reconstruct its correct shares, allowing it to fully participate in the signing protocol.

Unprocessed complaints. If a complaint does not receive $n - t$ support messages, it is not processed (no messages from the secure message distribution subprotocol are delivered for this purpose). This situation arises when honest parties have already abandoned the presignature batch before outputting the complaint from the corresponding RBC instance. But abandonment only occurs after the PresigBatchResultsRA instance has output, which means the signatures are already available and complaint processing is no longer needed for liveness. The complaint may still be adjudicated at presignature batch finalization (if some party included the complainer in its Supp set before abandoning). Adjudication does incur the expensive message-delivery cost (since the complainer's message must be opened for all to see), but this is justified because adjudication leads to player elimination.

6.6 Presignature batch finalization

When a party completes the construction of all signatures belonging to a presignature batch, it proceeds to finalize the presignature batch.

Signature availability. The party encodes the full collection of signatures from all signature batches belonging to this presignature batch using an erasure-coded RBC-like mechanism, and inputs the resulting commitment to the RA protocol instance PresigBatchResultsRA for this presignature batch. When PresigBatchResultsRA outputs, the party knows that:

- sufficiently many honest parties have computed the signatures and hold fragments; and
- any party that needs the signatures can request fragments from other parties and reconstruct them.

At this point, the party abandons the presignature batch, ceasing to send new support messages for complaints, and proceeds to the remaining finalization steps.

More precisely, abandoning a presignature batch means that the party stops sending new support messages for complaints associated with that batch and inputs its current Supp set to the finalization protocol. This ensures two properties. First, once PresigBatchResultsRA outputs, all honest parties can eventually obtain signatures via data recovery, so complaint processing is no longer needed for liveness. Second, if any complaint was processed (having received $n - t$ support messages), then at least $n - 2t$ honest parties supported it and committed to including the complainer in their Supp sets; by quorum intersection with the $n - t$ parties in the FinishACS output, the complaint will be adjudicated at presignature batch finalization, leading to the identification and elimination of a corrupt party.

A party that has abandoned may also stop participating in signature production messages, though in practice it may continue doing so for a nominal grace period to allow slower parties to obtain their signatures on the normal path rather than resorting to data recovery.

Broadcasting support sets. Each party P_i reliably broadcasts its set Supp_i: the set of parties whose complaints P_i supported during the lifetime of this presignature batch (before P_i abandoned the batch). Every party broadcasts its support set, even if it is empty.

Agreement on parties. An ACS protocol instance FinishACS is used to agree on a set of parties. Again, this ACS uses an *asynchronous validity predicate*: a party P_i is valid if:

- P_i has successfully reliably broadcast its Supp_i set; and
- each party $P_k \in \mathsf{Supp}_i$ has successfully reliably broadcast a complaint (via $\mathsf{Complain}_k$).

Complaint adjudication. Once FinishACS outputs a set T of parties, the complaints are adjudicated. For each party $P_i \in T$ and each complainer $P_k \in \mathsf{Supp}_i$, the parties open the complainer's message m_k (as received from the dealer being complained against). This is an expensive operation (scaling with L), but it is justified because it leads to player elimination.

By examining m_k, all parties can determine:

- if the complaint was valid ($m_k = \perp$ or fails the share validity check), then the *dealer* is corrupt; or
- if the complaint was invalid (m_k passes all checks), then the *complainer* is corrupt.

Since all parties see the same messages and perform the same checks, they all agree on the set of corrupt parties to eliminate.

Finality gadget. An RA protocol instance PresigBatchFinalityRA is used as a finality gadget for the presignature batch. The parties input the set of eliminated parties to this RA instance.

Once PresigBatchFinalityRA outputs, parties need only retain:

- the set of eliminated parties;
- the protocol messages for PresigBatchFinalityRA;
- the signatures produced;
- the protocol messages for the PresigBatchResultsRA instance.

All other resources, including the Complain RBC instances, may be garbage collected.

Key observation. As argued above in the discussion of abandonment (in the "Signature availability" paragraph) and the key invariant (in Section 6.1), any time the expensive message-delivery operation is triggered by complaint processing, the quorum intersection argument guarantees that the complaint will be adjudicated when FinishACS completes. Therefore, *any time the expensive message-delivery operation occurs, the corrupt party causing that operation will be identified and eliminated.*

6.7 Rate limiting and bounded damage

The protocol enforces a configurable bound C on the number of presignature batches that have not yet been finalized. If a party observes that C presignature batches are currently active, it will not participate in producing new presignature batches until some of the active batches are finalized.

This rate limiting, together with the player elimination mechanism, ensures the following property:

> *A corrupt party can poison at most C presignature batches before being eliminated.*

More precisely, if a corrupt party causes extra communication in some presignature batch, it will be eliminated when that batch finalizes. Since at most C batches can be active simultaneously, the corrupt party can affect at most C batches before elimination.

6.8 Communication complexity on the happy path

We consider here the communication complexity added by the signature production pipeline on the happy path. We do not include here the costs associated with the GoAVSS protocol itself — these were already discussed in Section 5.3. By the "happy path", we mean no honest party actually *processes* any complaints as discussed in Section 6.5.

6.8.1 Overhead costs

We first consider "overhead costs" per presignature batch, which do not depend on the parameter L (the number of polynomials shared in the GoAVSS protocol), and only depend on n and κ. Note that the overhead costs of running n GoAVSS protocols are $O(\kappa n^3 \log n)$, so our goal is just to argue that the additional overhead costs are bounded by this.

- *Consensus.* We need to run two ACS protocol instances: one in Stage 2 of the pipeline, DealerACS, to agree on a set of dealers, and another one in Stage 4, FinishACS, to agree on a set of support sets. Almost any ACS protocol will suffice here – for example, the protocol in [DDL+24, Sho24], which does not rely on any setup assumptions beyond what we are relying on here, and has a communication complexity of $O(\kappa n^3)$. In practice, any blockchain protocol may also be used here, although these typically rely on partial synchrony and other setup assumptions.

  Besides these two ACS protocol instances, in Stage 3 of the pipeline, we are relying on a consensus mechanism to order signing requests and pair them with presignatures. We can use any of the consensus protocols discussed above for this. Note that we need to run this protocol once for every signature batch, and there may be several of these within a presignature batch. We are assuming that in practice there will be a small, constant number of these. The actual process of disseminating and agreeing on actual signing requests is outside of the scope of this analysis, and so we will not estimate those costs here.

- *Complaint mechanism bookkeeping.* Here we count the overhead of running n RBC instances for reliably broadcasting complaint sets, running n RBC instances for reliably broadcasting support sets, and for each party to broadcast up to n support messages. Using the protocol of [DXR21] for RBC, the RBC cost is $O(\kappa n^3)$, and the cost to send all the support messages is $O(n^3 \log n)$.

6.8.2 Amortized costs

We now estimate the amortized communication cost per signature.

- *Public reconstruction.* In public reconstruction, used in Stage 3 of the pipeline, secrets are processed in groups of size $t + 1 \approx n/3$. In each such group, each party broadcasts a secret share twice. So the total communication cost per group is $2n^2\lambda$; therefore, the amortized cost per signature is $6n\lambda$.

- *Signature availability.* In Stage 4 of the pipeline, an RBC-like mechanism is used to provide signatures to parties who may not have received them in the normal protocol execution. We assume signatures are represented as a compressed point plus a scalar, which is compatible with the BIP-340 and RFC 8032 standards, so a signature is encoded as $\lambda + \lambda'$ bits.

  For the RBC-like mechanism, we may use the general approach of [LS24], where a large message is first encoded into fragments using an $(n, n - t)$ code, and then each fragment is encoded into mini-fragments using an $(n, n - 2t)$ code. A two-level Merkle tree is formed with n^2 mini-fragments at the leaves, and the root of this tree is the commitment used in the RA protocol instance PresigBatchResultsRA.

In this mechanism, each party who obtained signatures in the normal run of the protocol will broadcast their fragment and also send to each party a mini-fragment that will enable them to receive their own fragment. Each party that does not obtain signatures in the normal run of the protocol will reconstruct their fragment from mini-fragments and broadcast that. Eventually, all honest parties will broadcast their fragment, and all honest parties will reconstruct the message.

This leads to an amortized cost per signature of $1.5n(\lambda + \lambda')$.

Note that we could alternatively use a simpler RBC-like mechanism based on an $(n, n - 2t)$ erasure code. This would increase the amortized communication cost per signature from $1.5n(\lambda + \lambda')$ to $3n(\lambda + \lambda')$, while simplifying the protocol and reducing the computation somewhat.

We conclude that the total amortized cost per signature in the signature production pipeline — not including GoAVSS — is

$$7.5n\lambda + 1.5n\lambda'. \tag{22}$$

If we scale by the number of parties and add to the amortized GoAVSS cost per signature per party (20), we get an amortized communication cost *per signature per party* of

$$25.5\lambda + 1.5\lambda' + 4.5\lambda''. \tag{23}$$

With λ and λ' (roughly) equal to 32 bytes and λ'' equal to 64 bytes (as is typical), this is a total of 1152 bytes per signature per party, or roughly 9K bits.

6.9 Computational complexity on the happy path

We consider here the computational complexity added by the signature production pipeline on the happy path. Again, we do not include here the costs associated with the GoAVSS protocol itself. We present here amortized costs per signature per party.

6.9.1 Algebraic costs

The main algebraic costs are the application of the Pascal matrix in the batch randomness extraction step in Stage 2 of the signature production pipeline and the public reconstruction step in Stage 3 of the pipeline.

Application of Pascal matrix. In [GS23], it is shown in Section 6.2 that the amortized cost per presignature is $\approx n/2$ group additions, using a construction based on super-invertible matrices. In addition, in Section 6.3, they show that $\approx n/3$ group additions suffice for t and n not too large, using a construction based on hyper-invertible matrices.

Public reconstruction. Recall that in public reconstruction, used in Stage 3 of the pipeline, secrets are processed in groups of size $t + 1 \approx n/3$. For each such group, each party does the following:

- Treating the corresponding $t+1$ secret shares as shares of the *values* of a polynomial $f \in F[X]$ of degree t at the points $0, \ldots, t$, it extends these to values at $0, \ldots, n - 1$. This can be done

using the algorithms in Section A at a cost of $\approx n^2/3$ lazy scalar additions. So the amortized cost per signature is $\approx n$ lazy scalar additions.

Note that the suggested implementation is a bit different than what is usually presented in the literature, which is to treat the $t+1$ secret shares as shares of the *coefficients* of a polynomial f. For moderate size n, the approach here is more efficient. Moreover, as discussed in Section A.8, it is even more efficient to treat the $t+1$ secret shares as shares of the *differences* $\Delta^0 f(0), \ldots, \Delta^t f(0)$.

- What follows next is two rounds of online error correction. Now, while in the worst case, each such round may require running a Reed-Solomon error corrector, in the *common case*, this will not be necessary.

  Indeed, in the common case, each party will collect $n-t$ shares, interpolate through $t+1$ points, and then evaluate to get the values of a polynomial f of degree t at the points $0, \ldots, n-1$. If these values are consistent with the $n-t$ collected shares, we are done. Otherwise, one of the shares was from a corrupt party and the protocol proceeds with the full online error correction procedure. However, at the end of this procedure, we will have identified any parties who provided incorrect shares and can ignore them going forward (certainly, in processing signing requests associated with any subsequent presignature batches).

  So in the common case, the cost of the online error correction amounts to just computing a matrix-vector product (with a $(t+1) \times (t+1)$ square interpolation matrix) to interpolate and then running a polynomial extension algorithm. The interpolation matrix is computed once per signature batch, and we can typically ignore the cost of construction.

  So per group of $t+1$ secrets, the online error correction step (in the common case) costs $\approx (n/3)^2$ full scalar/scalar multiplications (for the matrix-vector multiplication) and $n^2/3$ lazy scalar additions (for the polynomial extension). (See Section A.12 for algorithmic details and experimental results.) So per signature, we divide by $n/3$ (because we process $\approx n/3$ secrets at a time) and multiply by 2 (because we have to do two online error correction steps) to get $2n/3$ full scalar/scalar multiplications and $2n$ lazy scalar additions.

Total algebraic costs. The total algebraic cost is therefore:

- $\approx 3n$ lazy scalar additions.
- $\approx 2n/3$ full scalar/scalar multiplications.
- $\approx n/2$ (or sometimes $\approx n/3$) group additions.

Example 6.1. *We continue now with our running example (last seen in Example 5.3).*

For the group additions that appear in the application of the Pascal matrix, we estimate the time for each to be 91 ns *(which is the time for* group_add_var *in* libsecp256k1*). As noted in [GS23], for these parameters, we can use the construction based on hyper-invertible matrices in Section 6.3 of that paper. Other algebraic costs are estimated as in Example 5.1.*

So we have:

- 294 ns *for* $\approx 3n$ *lazy scalar additions.*
- 163 ns *for* $\approx 2n/3$ *full scalar/scalar multiplications.*

- 1486 ns *for* $\approx n/3$ *group additions.*

So the total amortized time per signature is $\approx 2\,\mu s$.

6.9.2 Other computational costs

The only other computational costs we need to consider are the coding and hashing costs associated with the RBC-like mechanism used in the signature availability mechanism in Stage 4 of the signature production pipeline. We can safely estimate the amortized cost per signature per party as roughly $\frac{1}{3}$ of the corresponding cost (see Section 5.4.3) associated with the RBC of group elements in the RBCs used in Stage 2.

Example 6.2. *We continue now with our running example (last seen in Example 6.1). Based on our estimates in Example 5.2, the coding cost is* $\approx 0.1\,\mu s$ *per signature per party (taking* $\frac{1}{3}$ *of the cost reported there). Based on our estimates in Example 5.3, the hashing cost per signature per party is* $\approx 0.1\,\mu s$ *(again, taking* $\frac{1}{3}$ *of the cost reported there).*

We can now add up all of the computational costs in all stages of the signature production pipeline, including the GoAVSS costs, computed as amortized cost per signature per party:

- $2\,\mu s$ *for GoAVSS (see Example 5.3).*
- $2.2\,\mu s$ *all other costs (see above).*

So that makes $4.2\,\mu s$ *compute time per signature. Let us round that up to* $5\,\mu s$ *to account for other overheads.*

As we saw in Section 6.8.2, the amortized communication complexity per signature per party is about 9K bits. With a 1 Gbps network connection, that translates to a transmission delay of $9\,\mu s$ *per signature. Let us round that up to* $10\,\mu s$ *to account for other overheads.*

Hopefully, by interleaving compute, transmission, and propagation, we can maintain a pipelined steady state where the network link is fully saturated. If we can do that, we should be able to sustain a throughput of 100K signatures per second.

6.10 Communication and computational complexity on the unhappy path

Let C be the total communication complexity for the process of generating, consuming, and finalizing a batch of presignatures on the happy path. Let $L' \coloneqq L + M + K + R$ (where L, M, K, R are the GoAVSS parameters from Section 4) and (for simplicity) assume that $\lambda, \lambda', \lambda''$ are proportional to κ. Then we have $C = \Theta(\kappa n^2 L' + \kappa n^3 \log(n))$.

Now suppose that during this process, one party is corrupt. That party may be the dealer on one of the GoAVSS protocols that is included in the agreed upon set of dealers for that batch, in which case it may cause honest parties to register complaints against it. On the one hand, if none of those complaints are actually processed, no extra communication occurs. On the other hand, if some of those complaints are processed, that party is sure to be eliminated when the presignature batch is finalized, and the processing and adjudicating of those complaints may cause an additional communication complexity of $O(\kappa n^2 L' + \kappa n^3 \log(n))$. This corrupt party will also be a receiver in the GoAVSS protocols of honest dealers. Now, during the production of signatures for this batch of presignatures, that party may register bogus complaints against these honest dealers. If such a complaint is processed, the complainer forwards its message m_i, which all parties can check: since

the dealer is honest, m_i will pass all validity checks, so the complaint is found to be invalid and no honest party forwards messages to assist the complainer. Thus, complaint processing does not cause honest parties to send any additional messages (recall that the communication complexity counts only bits sent by honest parties, so the forwarding of m_i by the corrupt complainer does not contribute). However, these complaints may still be adjudicated during presignature batch finalization, and this will cause extra communication, but this communication is also bounded by $O(\kappa n^2 L' + \kappa n^3 \log(n))$.

Thus, the following property holds:

> *Suppose C is the communication complexity on the happy path. If a corrupt party causes any extra communication during this process, that additional communication is bounded by $O(C)$ and that party will be eliminated at the end of the process.*

Of course, there may be any number $t^* \leq t$ of corrupt parties that cause extra communication during this process — the additional communication that they cause is bounded by $O(t^*C)$ and they will all be eliminated at the end of the process.

One can also verify that an analogous property holds for computational complexity:

> *Suppose C' is the computational complexity on the happy path. If a corrupt party causes any extra computation during this process, that additional computation is bounded by $O(C')$ and that party will be eliminated at the end of the process.*

6.11 Online signing latency

As we already mentioned above, we assume that signing requests originate from a blockchain or some other type of consensus mechanism. This mechanism also pairs each signing request with a presignature. Such signing requests are processed in batches. These batches can be of any size — there is no need to wait for a specific number of signing requests to fill a batch. In the blockchain setting, it would be natural to process all signing requests generated by the execution of transactions in a single block as a batch. There may be just one signing request in a signature batch in periods of low demand.

After a batch of signing requests has been generated, the online signing latency is as follows:

1. one round of communication to open a random shared value $\delta \in F$ to implement batch re-randomization of the corresponding presignatures (this is essential to maintain security);

2. two rounds of communication to implement the two-phase public reconstruction (the two-phase structure is essential to maintain linear communication complexity).

So three rounds of communication in all.

Note that in some settings, we can completely eliminate the round to open δ. Specifically, in a typical blockchain consensus protocol (such as [Sho23b]), a quorum certificate is produced to finalize a block; moreover, if that certificate is itself a unique threshold signature (like BLS [BLS01, Bol03]), we can use a hash of that signature to derive δ non-interactively. See Appendix C.3.1 of [Sho23a] for additional details.

Also note that in periods of low demand, where the size of a signing request batch is small, the protocol can afford to use a one-phase opening protocol. Combined with the above optimization exploiting a quorum certificate, this reduces the online latency to a single round of communication (at least in periods where we do not need extremely high throughput).

We emphasize that signature batches do not correspond to presignature batches—the latter are produced in an offline, background process, and presignature batch sizes will generally be very large so as to optimize overall throughput. In much of the prior work on offline/online threshold signing (e.g., [KG20b, CGG+20]), the primary justification for an offline presigning phase is to reduce the *latency* of signature production: because the presignature is independent of the message to be signed, it can be computed ahead of time, so that when a signing request arrives, the online phase requires minimal interaction. The offline phase in our work (as well as in [GS23]) serves this purpose as well, but it serves an additional (and arguably more important) purpose: by producing presignatures in large batches, we can exploit amortization techniques (such as batch randomness extraction) that are simply not available when presignatures are generated one at a time. This means that the *per-presignature cost* decreases as the batch size grows, and consequently the protocol can sustain a higher throughput than any protocol that generates presignatures individually—*even if demand is constant and the system is always fully loaded.*

6.12 Security considerations

To analyze the security of the entire threshold signature scheme, we utilize the formal security model in Appendix C.7 of [Sho23a], which presents an ideal functionality $\mathfrak{F}_{\text{rrp}}$ that models a complete signing protocol using re-randomized presignatures. That model does not explicitly model batch randomness extraction or batch re-randomization of presignatures, but it is easy to add these features to get an ideal functionality $\mathcal{F}$ that does so (Appendix C.7 of [Sho23a] actually does discuss in some detail how to add batch randomness extraction to the model, and adding batch re-randomization of presignatures — where a single shared random value δ is used to re-randomize all presignatures in a signature batch — is quite straightforward). The essence of $\mathcal{F}$ is as follows:

- it generates a standard Schnorr public key, which is given to the adversary;
- upon request from the environment, it generates a batch of presignatures, which are given to the adversary;
- to model batch randomness extraction, a matrix/vector pair is supplied by the adversary, which converts a batch of presignatures into a batch of derived presignatures — these derived presignatures are what are actually used to sign messages;
- upon request from the environment, a collection of derived presignatures is specified, which are then paired with messages and signed using batch presignature re-randomization (i.e., a single random shift δ is generated and applied to all presignatures in the batch).

By design, the unforgeability of any threshold signature scheme that emulates $\mathcal{F}$ follows directly from the analysis in Sections 4.1–4.3 of [Sho23a]. It is important to note that the body of [Sho23a] does not involve any multi-party computation: the analysis there is entirely in terms of enhanced single-party attack games, in which an adversary interacts with a challenger. The connection to the multi-party setting is made in the appendices of that paper, which show that any protocol that emulates $\mathcal{F}$ inherits the unforgeability guarantees established by those single-party analyses. These guarantees are based on either a random oracle analysis, a generic group model analysis, or a combination of both.

We now argue that our threshold signing protocol emulates $\mathcal{F}$. Assuming we use an unbiased DKG protocol for generating the Schnorr signing key (as in Section 6.2), we get the "MPC engine"

discussed in Appendix C of [Sho23a], which includes the RandomKeyGen, LinearOp, and Open operations. Our $\mathfrak{F}_{\text{GoAVSS}}$ ideal functionality essentially gives us the InputKey operation discussed in Appendix C.5 of [Sho23a], and from there, the analysis in Appendix C.6 of [Sho23a] shows how the BatchedBiasedKeyGen operation is securely implemented.

Handling adaptive corruptions — the crux of the argument. Appendix C of [Sho23a] assumes static corruptions. However, the only place where the distinction between static and adaptive corruptions is relevant is at the level of the InputKey operation, and our $\mathfrak{F}_{\text{GoAVSS}}$ ideal functionality is designed to handle this gracefully.

In the InputKey operation as defined in Appendix C.5 of [Sho23a], a party P_i inputs a secret $r \in F$ and the group element $\mathcal{R} \coloneqq r\mathcal{G}$ is made public. Under static corruptions, P_i is either honest (in which case r remains hidden) or corrupt from the start (in which case the adversary chose r). In our setting, $\mathfrak{F}_{\text{GoAVSS}}$ permits adaptive corruption: after the group element $\mathcal{R}$ is revealed to the adversary, the adversary may corrupt the dealer. However, $\mathfrak{F}_{\text{GoAVSS}}$ employs a locking mechanism that governs what happens in this case. If the ideal-world adversary has already locked the dealer's input (via a Lock operation with no arguments), then the original input remains fixed and cannot be changed even after corruption — so the dealer's contribution is effectively treated as honest for the purposes of the simulation. Alternatively, if the dealer is corrupted before its input is locked, the adversary may either lock the original input without change, or lock a new input of its choosing (via a Lock operation with explicit arguments) — so the dealer's contribution is treated as corrupt, and the adversary must explicitly provide the replacement values without ever learning the original secret r.[8]

This is precisely the behavior needed for the simulation in Appendix C.6 of [Sho23a] to go through. That simulation constructs the bias matrix U by selecting columns of the super-invertible matrix W corresponding to "honest" contributions (those whose inputs were locked without modification) and derives the bias vector u' from the "corrupt" contributions (those whose inputs were explicitly provided by the adversary). The super-invertibility of W guarantees that U has full rank as long as the number of honest contributions is at least $n - 2t$, which is ensured by the corruption bound. Since the locking mechanism is the only point at which the static vs. adaptive distinction arises, and it provides exactly the same information to the simulator in both cases, the analysis in Appendix C.6 of [Sho23a] carries over directly to our adaptive setting.

At this stage, the proof that our threshold signing protocol emulates $\mathcal{F}$ is essentially complete.

7 Unbiased key generation

As discussed in Section 6.2, we require an unbiased distributed key generation (DKG) protocol to establish the signing key. In a standard GoAVSS-based DKG, each dealer publishes group elements $\{\mathcal{S}_\ell\}_{\ell \in [L]}$ as part of its dealing, and these are visible to the adversary as soon as the dealing is initiated. An ACS protocol then agrees on a set S of $n - t$ dealers, and batch randomness extraction using a super-invertible matrix produces the derived keys. The problem is that the

[8] The $\mathfrak{F}_{\text{GoAVSS}}$ ideal functionality includes a LeakShares operation that allows the ideal-world adversary to learn individual shares before or after the dealer's input is locked. Our GoAVSS protocol in Section 4 does not require this operation, but looking forward, the simplified protocol in Section 8 does. In any case, this does not affect the above argument: the adversary learns at most t random shares that are independent of r.

adversary sees the honest dealers' group elements before the dealer set is fixed, and can therefore choose its own inputs adaptively to bias the resulting keys.

To prevent this, we introduce a **hiding** variant of GoAVSS in which the group elements remain hidden until the dealer set has been irrevocably determined. The group elements are committed to during the dealing phase (so that the soundness guarantees of GoAVSS are preserved), but are only revealed in a subsequent opening phase that takes place after the ACS completes. This approach is analogous to the "secret bulletin board" idea in Section 22.4.2 of [BS23].

7.1 Hiding-GoAVSS ideal functionality

We present here an ideal functionality $\mathfrak{F}_{\text{HiGoAVSS}}$ for hiding-GoAVSS. This is a modification of the $\mathfrak{F}_{\text{GoAVSS}}$ functionality from Section 4.3 in which the group elements $\{\mathcal{S}_\ell\}_{\ell\in[L]}$ are not leaked to the adversary until an honest party explicitly requests their opening. We highlight the differences from $\mathfrak{F}_{\text{GoAVSS}}$:

- $\mathsf{Input}(f_1, \ldots, f_{L+M})$: this operation may be invoked once by an honest dealer D, who inputs polynomials $f_1, \ldots, f_{L+M} \in F[X]_{\leq t}$ to $\mathfrak{F}_{\text{HiGoAVSS}}$. In response, $\mathfrak{F}_{\text{HiGoAVSS}}$ sends the message $\mathsf{NotifyInput}()$ to the ideal-world adversary.

  Unlike $\mathfrak{F}_{\text{GoAVSS}}$, the group elements $\{\mathcal{S}_\ell\}_{\ell\in[L]}$ are not included in this notification.

- $\mathsf{Lock}()$: same as in $\mathfrak{F}_{\text{GoAVSS}}$.

- $\mathsf{Lock}(f'_1, \ldots, f'_{L+M}, err)$: this operation may be invoked by the ideal-world adversary at a point in time that no Lock operation was previously invoked, and that D is currently corrupt. The ideal-world adversary inputs $f'_1, \ldots, f'_{L+M} \in F[X]_{\leq t}$ and an optional flag $err \in \{\mathsf{true}, \mathsf{false}\}$ to $\mathfrak{F}_{\text{HiGoAVSS}}$. In response, $\mathfrak{F}_{\text{HiGoAVSS}}$ sets $f_\ell := f'_\ell$ for $\ell \in [L+M]$ and $\mathcal{S}_\ell := f_\ell(0)\mathcal{G}$ for $\ell \in [L]$. If $err = \mathsf{true}$, the functionality records that this dealer's group elements are erroneous.

  This allows the ideal-world adversary to define the input polynomials of a corrupt dealer, and to indicate that the group elements will not pass validation. In the intended application, the adversary must commit to this error condition before seeing any honest dealer's group elements.

- $\mathsf{ProvideReady}(i)$: this operation may be invoked once per $i \in [n]$ by the ideal-world adversary at any time after a Lock operation has been invoked. In response, $\mathfrak{F}_{\text{HiGoAVSS}}$ sends the message Ready to party P_i.

  Unlike $\mathfrak{F}_{\text{GoAVSS}}$, the group elements are not included in the Ready message.

- $\mathsf{GetShares}()$: same as in $\mathfrak{F}_{\text{GoAVSS}}$.

- $\mathsf{ProvideShares}(i)$: same as in $\mathfrak{F}_{\text{GoAVSS}}$.

- $\mathsf{LeakShares}(i)$: same as in $\mathfrak{F}_{\text{GoAVSS}}$.

- $\mathsf{Open}(i)$: this operation may be invoked once per $i \in [n]$ by an honest party P_i at any time after it has received its Ready message. In response, $\mathfrak{F}_{\text{HiGoAVSS}}$ sends the message $\mathsf{NotifyOpen}(i, \{\mathcal{S}_\ell\}_{\ell\in[L]})$ to the ideal-world adversary.

This is where the group elements are first leaked. In the intended application, honest parties invoke Open only after the ACS has fixed the dealer set.

- ProvideOpened(i): this operation may be invoked once per $i \in [n]$ by the ideal-world adversary at any time after party P_i has invoked Open. If $err = \mathsf{true}$ was set during the Lock operation, $\mathfrak{F}_{\text{HiGoAVSS}}$ sends the message Opened(ERROR) to party P_i. Otherwise, $\mathfrak{F}_{\text{HiGoAVSS}}$ sends the message Opened($\{\mathcal{S}_\ell\}_{\ell \in [L]}$) to party P_i.

Completeness. The completeness property from Section 4.3 is also expanded to include the following:

- If all honest parties invoke the Open operation, then eventually all honest parties output Opened.

7.2 The unbiased DKG protocol

Using the hiding-GoAVSS functionality, we construct an unbiased DKG protocol as follows.

Dealing phase. Each party P_i acts as a dealer and runs an instance of hiding-GoAVSS to share L GoAVSS polynomials and M plain AVSS polynomials, just as in the presignature batch production of Section 6.3.

Agreement on dealers. An ACS protocol instance is used to agree on a set $S \subseteq [n]$ of $n - t$ dealers. The validity condition for each dealer is the Ready message from the corresponding hiding-GoAVSS instance, which — crucially — does not involve the group elements.

Opening phase. After the ACS outputs the dealer set S, each honest party invokes Open on the hiding-GoAVSS instance of every dealer in S. By the completeness property, all honest parties eventually receive Opened for each dealer in S.

Error filtering. Some dealers in S may return Opened(ERROR), indicating that their group elements are invalid. These dealers are identified as corrupt and discarded, yielding a filtered set $S' \subseteq S$. Say t^* dealers are discarded. Then $|S'| = n - t - t^*$, and among the dealers in S', at most $t - t^*$ can be corrupt (since at most t parties are corrupt in total and t^* have already been identified). In particular, S' contains at least $(n - t - t^*) - (t - t^*) = n - 2t$ honest dealers.

Batch randomness extraction. The parties perform batch randomness extraction on the dealers in S', exactly as described in Section 6.3. Let W be an $(n - 2t) \times |S'|$ super-invertible matrix over F. Writing $S' = \{i_1, \ldots, i_{|S'|}\}$, the parties compute $n-2t$ key pairs $(\bar{x}_1, \bar{\mathcal{X}}_1), \ldots, (\bar{x}_{n-2t}, \bar{\mathcal{X}}_{n-2t})$ where

$$\begin{pmatrix} \bar{x}_1 \\ \vdots \\ \bar{x}_{n-2t} \end{pmatrix} = W \cdot \begin{pmatrix} x_{i_1} \\ \vdots \\ x_{i_{|S'|}} \end{pmatrix}$$

and $\bar{\mathcal{X}}_j = \bar{x}_j \mathcal{G}$. The shares $[\bar{x}_j]$ are computed locally from the shares $[x_i]$ for $i \in S'$, and the group elements $\bar{\mathcal{X}}_j$ are computed from the (now revealed) group elements $\mathcal{X}_i$ for $i \in S'$.

Unbiasedness. The extracted keys are unbiased. In the ideal world, the adversary must Lock all of its dealer contributions — including the *err* flags — before any honest party invokes Open, which is before the adversary sees any honest dealer's group elements. Thus the adversary's contributions to S' (both the non-erroneous inputs and the choice of which dealers to mark as erroneous) are determined independently of the honest contributions. As argued above, at least $n - 2t$ of the columns of W correspond to honestly generated inputs. The super-invertibility of W then guarantees that the extracted values are uniformly random, conditioned on the adversary's view, by the same argument as in Appendix C.6 of [Sho23a].

More precisely, the $\mathfrak{F}_{\text{HiGoAVSS}}$ Lock mechanism provides the interface needed for the RandomKeyGen operation discussed in Appendix C of [Sho23a]; see Section 6.12 for further discussion.

Application to signing key setup. We have presented the above construction in full generality, as it may be useful in applications that require multiple unbiased keys. However, for the signing key setup, only a single unbiased key is needed, and a simpler approach suffices. We can set $L = 1$ and use an ACS that agrees on a set S of only $t + 1$ dealers (rather than $n - t$). After filtering out erroneous dealers, at least one honest dealer remains, and the signing key can be taken as the sum of the non-erroneous contributions. Since at least one summand is honestly generated and hidden from the adversary, the resulting key is uniformly random.

7.3 Implementing hiding-GoAVSS

We now describe how to modify the GoAVSS protocol of Section 4 to realize $\mathfrak{F}_{\text{HiGoAVSS}}$. The key idea is that the group elements, instead of being reliably broadcast in the clear, are encrypted under a key that is secret-shared among the parties. The encrypted group elements are committed to during dealing (so that soundness is preserved), but the key is only reconstructed in the opening phase.

7.3.1 Modifications to the dealer logic

In addition to the polynomials $f_1, \ldots, f_{L+M+1+R}$ of the standard GoAVSS dealer logic, the dealer generates a random secret $s_0 \in F$ and computes an $(n - 2t)$-out-of-n Shamir secret sharing $(s_1, \ldots, s_n)$ of s_0. It then derives

$$k_j = H_{\text{ss}}(j, s_j) \quad \text{for } j = 0, 1, \ldots, n.$$

This is exactly the same key derivation mechanism used in the SMD protocol (Section 3.1), except that here there is just a single key k_0 rather than n separate keys.

We set $K = 1$ for hiding-GoAVSS. In the standard GoAVSS protocol, the group element validity check (9) is distributed across parties: with $K > 1$, each party P_i checks the condition corresponding to $\kappa(i) = (i \bmod K) + 1$, and distinct parties may check different conditions. This works because the check happens during the one-sided vote, and the probabilistic soundness argument ensures that if enough parties vote, the group elements are valid. In hiding-GoAVSS, however, the group element check is deferred to the opening phase, where all parties must reach the *same* accept/reject decision — since they must uniformly agree on whether to output ERROR or the group elements. With $K = 1$, there is a single condition that all parties evaluate identically, ensuring unanimous agreement. The tradeoff is a higher computational cost: we use $\Gamma = F$ (rather than a small subset),

so the optimized "tiny scalar/group multiplication" technique no longer applies. However, since key setup is performed infrequently, this additional cost is acceptable. Avoiding it would require a more complex ideal functionality and a separate interactive protocol for group validation.

Moreover, with $K = 1$, the batching optimization in Steps 10–14 of the dealer logic in Section 4.1 (the computation of scalars c_k, challenges δ_k, and the combined polynomial h^*) is unnecessary: it suffices to directly broadcast the polynomial h'_1.

The dealer logic proceeds as follows. Steps 1–6 are unchanged. Steps 7 and 8 are replaced by:

7′. Compute group elements $\mathcal{S}_\ell \leftarrow f_\ell(0)\mathcal{G}$ for $\ell \in [L]$ and $\mathcal{T}_1 \leftarrow f_{L+M+1}(0)\mathcal{G}$. Compute the ciphertext

$$c_{\text{ge}} \coloneqq H_{\text{enc}}(k_0) \oplus (\mathcal{S}_1, \ldots, \mathcal{S}_L, \mathcal{T}_1).$$

8′. Construct the Merkle tree with root ρ_3 that includes

(a) ρ_2,
(b) the Merkle tree used in the implementation of an RBC protocol to reliably broadcast c_{ge}, and
(c) a Merkle tree with leaves $k_1, \ldots, k_n$ (the hashed Shamir shares of s_0, used for verification both in the initial send phase and during the opening phase).

The remaining steps simplify as follows. Step 9 is unchanged: H'_{chall} is applied to ρ_3 to obtain a challenge $\{\gamma_{1,\ell}\}_{\ell\in[L]}$ (just a single set of challenges, since $K = 1$). Steps 10–14 are then replaced by:

10′. Compute the polynomial

$$h'_1 \leftarrow f_{L+M+1} + \sum_{\ell\in[L]} \gamma_{1,\ell} \cdot f_\ell \in F[X]_{\le t}.$$

11′. Construct the Merkle tree with root $\rho = \rho_4$ that includes

(a) ρ_3, and
(b) the Merkle tree used in the implementation of an RBC protocol to reliably broadcast h'_1.

Note that the Merkle chain structure $\rho_1, \rho_2, \rho_3, \rho_4 = \rho$ is preserved, and the only substantive change is in the contents of ρ_3: it now contains the encrypted group elements and the hashed key shares, rather than the plaintext group elements.

The shares $s_1, \ldots, s_n$ are transmitted over sender-forward-secure channels, together with the other secret shares, with the top-level root ρ embedded in those messages, exactly as described in Section 3.2. After transmission, the dealer erases s_0, the shares s_j, and all associated keying material.

The use of sender-forward-secure channels for the s_j shares is essential for the hiding property. If the channels were not forward-secure, an adversary could corrupt a dealer after it has completed the dealing and recover s_0 from the unerased channel state, thereby decrypting the group elements before the ACS has fixed the dealer set — exactly the bias attack that hiding-GoAVSS is designed to prevent. With forward security and erasure, corrupting the dealer after transmission yields no information about s_0. In an honest dealing (where $\hat{\rho} = \rho$), the group elements are revealed only upon explicit Open; in a transitional corrupt dealing (where $\hat{\rho} \neq \rho$), honest parties reject the send messages and the group elements are never revealed.

7.3.2 Modifications to the receiver logic

The receiver logic proceeds similarly to the standard GoAVSS protocol. Each party P_i participates in the consistent broadcast of ρ, then processes the *send* messages from the dealer and performs the *echo* and *vote* logic for the SMD and RBC subprotocols.

When P_i obtains its outputs from these subprotocols, it performs the following checks:

1. Reconstruct the message $m_i = \{v_{\ell,i}\}_{\ell \in [L+M+1+R]}$ from the SMD subprotocol and the polynomials $\{h_r\}_{r \in [R]}$ and h'_1 from the RBC subprotocols. Also reconstruct the ciphertext c_{ge}.
2. Check the share validity condition (6), just as in the standard protocol.
3. Check that
$$h'_1(e_i) = v_{L+M+1,i} + \sum_{\ell \in [L]} \gamma_{1,\ell} \cdot v_{\ell,i}, \tag{24}$$
where $\{\gamma_{1,\ell}\}_{\ell \in [L]}$ is obtained from H'_{chall} on input ρ_3.
4. Validate that P_i's share s_i (received on the sender-forward-secure channel as part of the initial send message) satisfies $H_{\mathrm{ss}}(i, s_i) = k_i$, where k_i is authenticated by the Merkle tree under ρ_3 (and hence ρ).

If all of these checks pass, P_i inputs a null message into the RA protocol to perform a one-sided vote, and upon completion outputs Ready.

Note that no group element validity check is performed at this stage, since the group elements are encrypted and P_i does not yet have the key to decrypt them. This check is deferred to the opening phase.

7.3.3 The opening protocol

When an honest party P_i initiates the opening (after the ACS has fixed the dealer set), it broadcasts its share s_i of the key, along with the Merkle proof that authenticates it with respect to ρ.

Each party collects such broadcasts. A broadcast share s_j from party P_j is accepted if the Merkle proof is valid with respect to ρ and $H_{\mathrm{ss}}(j, s_j)$ matches the committed value k_j (which is itself authenticated by the Merkle tree under ρ). Once a party has collected $n - 2t$ valid shares, it reconstructs s_0 via Shamir interpolation, derives $k_0 = H_{\mathrm{ss}}(0, s_0)$, and decrypts the group elements as $H_{\mathrm{enc}}(k_0) \oplus c_{\mathrm{ge}}$.

The party then runs the group element validity check that was deferred from the receiver logic. Specifically, the party checks that

$$h'_1(0)\mathcal{G} = \mathcal{T}_1 + \sum_{\ell \in [L]} \gamma_{1,\ell} \mathcal{S}_\ell, \tag{25}$$

where h'_1 and $\{\gamma_{1,\ell}\}_{\ell \in [L]}$ are the same values used in (24). With $K = 1$, this check is deterministic: all honest parties evaluate the same condition and reach the same accept/reject decision. With $\Gamma = F$, the soundness error for this check is $1/q$, which is negligible.

If the check passes, the party outputs the group elements $\{\mathcal{S}_\ell\}_{\ell \in [L]}$. If the check fails, the party outputs ERROR.

Completeness. If all honest parties initiate the opening, they will all broadcast their shares. Since there are at least $n-t > n-2t$ honest parties, and each honest party's share is authenticated by the Merkle tree, all honest parties will eventually collect $n-2t$ valid shares and reconstruct the key. Thus, all honest parties eventually receive the opened group elements (or ERROR).

Simulation. In the security proof, the simulator can extract the dealer's polynomials from the one-sided vote (using the same extraction technique as in the standard GoAVSS analysis). At that point, the simulator also extracts s_0 from the AVID-like sharing, derives $k_0 = H_{\text{ss}}(0, s_0)$, decrypts the group elements, and tests validity. If the group elements are invalid, the simulator invokes Lock with $err = \text{true}$; otherwise, it invokes Lock normally. This all occurs before any honest party invokes Open, so the simulator commits to the error condition before the group elements are revealed — exactly as required by $\mathfrak{F}_{\text{HiGoAVSS}}$.

For honest dealers, the simulator programs H_{enc} at the point when the group elements are first leaked (upon NotifyOpen), making the previously generated ciphertext c_{ge} consistent with the actual group elements. This is possible because $k_0 = H_{\text{ss}}(0, s_0)$ is not known to the adversary at the time the ciphertext is produced (by the generalized linear hiding assumption on H_{ss}, as in Section 3.1.1), so $H_{\text{enc}}(k_0)$ has not yet been queried.

8 A simplified protocol for $n > 4t$

In this section, we present a simplified variant of the signature production pipeline that achieves linear communication complexity in the *worst case*, at the cost of a weaker resilience threshold: $n > 4t$ rather than $n > 3t$.

This simplification is made possible by removing the requirement of *completeness* from the GoAVSS protocol. That is, some parties may receive invalid shares or no shares at all from a given dealer, and there is no complaint mechanism to remedy this. As a consequence, the protocol does not require the SMD subprotocol, and the only RBC instances within GoAVSS are those used to disseminate group elements. The complaint mechanism, support messages, and complaint adjudication described in Section 6 are all eliminated.

8.1 Overview

The protocol proceeds through the following stages.

Stage 1: Key setup. Key setup is identical to that described in Section 6.2. It is important that the key setup uses a complete AVSS protocol, since the set of receivers may vary from one presignature batch to the next, and every honest party must hold a valid share of the secret key.

Stage 2: Presignature batch production. Each party P_i acts as a dealer and runs an instance of a *simplified GoAVSS protocol* (described in Section 8.2). This simplified protocol differs from the GoAVSS protocol in Section 4 in that it does not use the SMD subprotocol; instead, the dealer encrypts shares and sends them directly to each receiver over sender-forward-secure channels. The group elements $\{\mathcal{S}_\ell\}_{\ell \in [L]}$ are still reliably broadcast, so that all parties — even those who do not receive valid shares — can compute the public components of presignatures and hence the group elements needed for signature verification without delay.

Once the dealings are distributed, the parties execute a **dealer/receiver agreement protocol** to agree on sets *Dealers* and *Receivers* satisfying:

(i) every party in *Receivers* approves every dealer in *Dealers*;

(ii) $|Receivers| \geq n - t$;

(iii) $|Dealers| \geq n - 2t$.

The precise definition of "approval" is given in Section 8.2. Intuitively, P_j approves P_i if P_j received from P_i a dealing that passes all local validity checks. The dealer/receiver agreement protocol is presented abstractly here as a subprotocol with the above interface; several implementations are discussed in Section 8.4.

The agreement protocol uses the star-finding algorithm of Section B to find sets of the required sizes. Since the honest parties mutually approve each other, they form a clique of size at least $n-t$ in the approval graph. As shown in Section B, the star-finding algorithm is guaranteed to succeed whenever the graph contains a clique of this size. Ideally, $|Dealers|$ should be as large as possible — up to $n - t$ — to maximize the yield of presignatures per batch. In fact, as we demonstrate in Section B.6, for moderate sized n (up to around 100, at least), we can profitably use integer linear programming (ILP) to find dealer sets very close to size $n - t$ almost all the time.

Using the batch randomness extraction technique from [GS23], the parties in *Receivers* compute $(|Dealers| - t)L$ presignatures from the polynomials shared by the dealers in *Dealers*. This is done exactly as in Section 6.3, except that the super-invertible matrix W has dimensions $(|Dealers| - t) \times |Dealers|$.

Stage 3: Signature batch production. Signatures are produced in batches, as in Section 6.4, with two important differences in the public reconstruction step.

First, only the parties in *Receivers* hold valid shares of the presignature polynomials, and so only they can compute shares of the signature scalars $[s_k]$. Second, the public reconstruction uses a two-phase protocol that allows all parties (not just the receivers) to learn the final signature scalars. This protocol processes shared secrets in groups of size $n-2t$ (rather than groups of size $t+1$ as in Section 6.4), which leads to a modest improvement in communication complexity. Details below.

Stage 4: Presignature batch finalization. Since there are no complaints to adjudicate, the finalization stage is greatly simplified compared to Section 6. The signature availability mechanism using an RBC-like encoding and the RA protocol instance PresigBatchResultsRA may still be used for practical purposes:

- gating presignature production so that it does not run too far ahead of signing, and
- enabling garbage collection of protocol state.

However, unlike the original pipeline, this mechanism is not theoretically necessary: all honest parties may eventually obtain the signatures directly through the two-phase public reconstruction.

Remark 8.1. *Star-finding algorithms were introduced by Canetti [Can96] in the context of AVSS with $n > 4t$, where they are employed* within *each AVSS instance to identify a core set of parties during a verification stage. A subsequent completion stage then ensures that all parties — even those*

outside the core — obtain valid shares, achieving a complete *AVSS protocol. This approach has been further developed. For example, more recently, Choudhury and Patra [CP17], gave a more efficient AVSS protocol and built an MPC framework on top of it with $O(n)$ amortized communication per multiplication gate. The $n > 4t$ threshold is intrinsic to both approaches, as the completion stage requires reconstructing degree-$2t$ polynomials via error correction with up to t corrupt parties. Our approach operates at the same resilience threshold, but uses star-finding in a different way: we forgo completeness entirely and instead use star-finding at the signature-production pipeline level to agree on which dealers and receivers to use for a given presignature batch. This avoids the communication cost of the completion stage and leads to a simpler protocol overall.*

Remark 8.2. *It is instructive to compare our approach with the complaint mechanism used in SPRINT [BHK+24]. SPRINT uses a party-elimination framework based on complaints, but its AVSS protocol guarantees only that $n - t$ parties hold valid shares of each presignature polynomial. With $n = 3t + 1$, this means that the parties agree on a set of $2t + 1$ receivers who claim to hold shares for an agreed-upon set of dealers. However, up to t of these $2t + 1$ receivers may be corrupt, leaving only $t + 1$ guaranteed-honest shares — far short of the $2t + 1$ correct values needed for online error correction (OEC) to correct up to t errors. As a result, OEC simply cannot be used in this regime. SPRINT addresses this by using polynomial commitments to validate individual share openings, but this reliance on polynomial commitments introduces significant computational overhead (each verification involves group exponentiations) and increases the communication cost per secret.*

In our $n > 4t$ protocol, the situation is fundamentally different. The star-finding mechanism ensures that the agreed receiver set has size at least $n - t$, and every receiver in this set has approved every dealer in the agreed dealer set. Among $n - t$ receivers, at least $n - 2t$ are honest, each holding valid shares of every selected dealer. Since $n - 2t > 2t$ (using $n > 4t$), OEC has access to more than $2t$ correct values, allowing it to correct up to t errors. Thus, OEC works directly without any need for polynomial commitments or individual share validation, and we retain the lightweight cryptographic framework of [SS23]. In effect, the stronger resilience assumption $n > 4t$ eliminates the need for both the complaint mechanism and *the heavyweight verification machinery that would otherwise be required to compensate for incomplete sharing.*

8.2 Simplified GoAVSS

We describe the modifications to the GoAVSS protocol of Section 4 for the simplified setting.

8.2.1 Dealer logic

The dealer proceeds as in Section 4.1, with the following modifications.

Share encryption. In place of the SMD subprotocol, the dealer encrypts shares directly for each receiver. For each receiver P_i, the dealer chooses a random secret s_i and sends s_i to P_i. Crucially, as in the original GoAVSS protocol, each such secret must be transmitted as a message over a sender-forward-secure channel. The shares $m_i \coloneqq \{v_{\ell,i}\}_{\ell \in [L+M+K+R]}$ are encrypted as

$$c_i \coloneqq H_{\mathrm{enc}}(k_i) \oplus m_i, \qquad \text{where} \quad k_i \coloneqq H_{\mathrm{ss}}(0, s_i).$$

Each ciphertext c_i may be sent directly to P_i over an authenticated channel.

Merkle tree under ρ_1. The Merkle tree with root ρ_1 is constructed with leaves $(H_{\text{ss}}(1, s_i), c_i)$ for $i \in [n]$, replacing the SMD-based structure in Step 3 of Section 4.1. The Fiat-Shamir challenge is then derived from ρ_1 as before.

Merkle tree structure. The remainder of the Fiat-Shamir structure (the trees with roots $\rho_2, \ldots, \rho_5$) is the same as in Section 4.1, except that the group elements $\{\mathcal{T}_k\}$, the polynomials $\{h_r\}$, the scalars $\{c_k\}$, and the polynomial h^* are sent point-to-point rather than via RBC. Only the group elements $\{\mathcal{S}_\ell\}$ are reliably broadcast.

This means that we just need simple, non-hiding, hash-based commitments to $\{\mathcal{T}_k\}$, $\{h_r\}$, $\{c_k\}$, and h^*, rather than tree-based commitments of erasure encodings.

The reason for reliably broadcasting $\{\mathcal{S}_\ell\}$ is to ensure that all parties, even those outside of the receiver set, are able to compute signatures.

Consistent broadcast of ρ. As before, the dealer consistently broadcasts the top-level Merkle root $\rho = \rho_5$. When transmitting the secret s_i, both s_i and ρ should be sent together over the sender-forward-secure channel. After transmitting all messages, the dealer erases all share data and randomness to ensure sender-forward secrecy.

8.2.2 Receiver logic and approval

When a receiver P_j processes a dealing from a dealer P_i, it proceeds as follows.

1. P_j participates in the consistent broadcast of the Merkle root ρ.
2. Once the consistent broadcast completes with output $\hat{\rho}$, P_j checks that the value ρ received on the sender-forward-secure channel matches $\hat{\rho}$. If $\rho \neq \hat{\rho}$, P_j discards (s_j, ρ) without using s_j in any way and does not approve this dealer. Otherwise, P_j uses the secret s_j to derive $k_j = H_{\text{ss}}(0, s_j)$ and decrypt $m_j = H_{\text{enc}}(k_j) \oplus c_j$. P_j verifies the Merkle path for $(H_{\text{ss}}(1, s_j), c_j)$ in the tree under ρ_1, with ρ_1 verified against ρ.
3. P_j processes all other messages by verifying all Merkle paths against the root ρ. The messages corresponding to the reliable broadcast of $\{\mathcal{S}_\ell\}$ are also processed using the logic of the underlying RBC protocol.
4. P_j performs the local validity checks (6), (7), (8), and (9), exactly as in Section 4.

If all of the above checks pass, we say that P_j **approves** P_i. There is no one-sided vote: approval is a purely local decision.

Remark 8.3. *Unlike the GoAVSS protocol in Section 4, there is no mechanism for P_j to obtain valid shares if the checks fail or if P_j does not receive a message from the dealer at all. In such cases, P_j simply does not approve P_i.*

8.2.3 Security considerations

The Fiat-Shamir structure of the dealing is preserved: the Merkle tree under ρ_1 commits the dealer to all n encrypted share messages before the challenge is derived. The soundness analysis from Section 5.1 therefore applies, with the following modification to the extraction argument.

In the original GoAVSS protocol, the simulator extracts all n messages $(m_1, \ldots, m_n)$ via the SMD subprotocol. Here, consider a corrupt dealing. If the dealer was corrupt from the start, the adversary explicitly supplies all data to honest parties. In the transitional corrupt case (where the dealer was initially honest but later corrupted), the $\rho \neq \hat{\rho}$ check on the sender-forward-secure channel ensures that every honest receiver rejects the dealer's original message and discards s_j without using it. Thus, the only data an honest receiver can accept must be explicitly supplied by the adversary. In either case, when a dealer is included in the agreed dealer set, there are at least $n-t$ receivers who approved the dealing, of which at least $n-2t$ are honest. The simulator extracts shares from these honest receivers. Since $n - 2t > 2t > t$ (using $n > 4t$), this provides more than enough shares to interpolate and recover the underlying degree-t polynomials $f_1, \ldots, f_{L+M}$.

One must also argue that any shares subsequently received by an honest party P_i (who was not among the parties used for extraction) are consistent with the extracted polynomials, provided P_i's local validity checks pass. It is straightforward to adapt the arguments in Section 5.1 to this setting, and we omit the details.

For an initially honest dealing, the simulation proceeds along the same lines as Game 3 of the GoAVSS security proof in Section 5.1. The simulator generates random secrets s_i, replaces the real ciphertexts c_i with independent random values, and simulates the Fiat-Shamir structure (programming H'_{chall} as described there). The dealer erases all share data and randomness after sending, so corruption of the dealer reveals nothing. When a receiver P_i is corrupted, the simulator obtains P_i's shares from the ideal functionality (via the LeakShares operation) and programs H_{enc} to maintain consistency. The generalized linear hiding assumption with $d = 1$ ensures that the adversary cannot compute $H_{\text{ss}}(0, s_i)$ from the committed hash $H_{\text{ss}}(1, s_i)$ before this programming occurs.

Unlike the original GoAVSS protocol, the simplified protocol requires the LeakShares operation in $\mathfrak{F}_{\text{GoAVSS}}$. In the original protocol, each party's shares are protected by Shamir sharing of the encryption key within the SMD subprotocol, so no shares leak before the dealer's input is locked. Here, each s_i is sent directly over a sender-forward-secure channel, and if receiver P_i is corrupt, the adversary obtains s_i and can decrypt P_i's shares immediately. This can occur before or after the lock point. This leakage is harmless for the overall security of the pipeline, as discussed in Section 6.12.

8.3 Simplified signature production pipeline

We now describe the full signature production pipeline for the $n > 4t$ setting. Stages 1, 3, and 4 are largely as described in Section 6, with the modifications noted in Section 8.1. We focus here on Stage 2 (presignature batch production) and the modified public reconstruction in Stage 3.

8.3.1 Presignature batch production

Dealer phase. Each party P_i acts as a dealer and runs the simplified GoAVSS protocol of Section 8.2, sharing L GoAVSS polynomials and M plain AVSS polynomials, exactly as in Section 6.3.

Agreement on dealers and receivers. The parties execute a dealer/receiver agreement protocol (see Section 8.4) to obtain agreed-upon sets *Dealers* and *Receivers*. Each party locally builds an *approval graph*: an undirected graph G on vertex set $[n]$, where $\{i, j\}$ is an edge if P_j approves

P_i and P_i approves P_j. The star-finding algorithm of Section B (or the ILP optimization of Section B.6) is applied to this graph to obtain the sets *Dealers* and *Receivers*. Note that a party may appear in both sets, and no self-approval is required: the star definition (Definition B.1) requires adjacency only for distinct nodes.

Batch randomness extraction. Writing $Dealers = \{i_1, \ldots, i_d\}$ with $d = |Dealers|$, the parties in *Receivers* compute $(d-t)L$ biased presignatures using a $(d-t) \times d$ super-invertible matrix W, exactly as in Section 6.3. Since $d \geq n - 2t$, the yield is at least $(n-3t)L$ presignatures per batch. When d is larger (up to $n-t$), the yield improves correspondingly, up to a maximum of $(n-2t)L$.

8.3.2 Two-phase public reconstruction

We describe here the two-phase public reconstruction protocol used in Stage 3. It proceeds as follows.

- **Phase 1:** Take a group of $n-2t$ shared signature scalars. Treating the corresponding $n-2t$ secret shares as shares of the *values* of a polynomial $f \in F[X]$ of degree $n-2t-1$ at the points $0, \ldots, n-2t-1$, each party $P_j \in Receivers$ extends these to values at $0, \ldots, n-1$. This can be done using the algorithms in Section A. Each extended value $f(k)$ is a linear combination of $f(0), \ldots, f(n-2t-1)$, and so $[f(k)]$ is a sharing on a polynomial of degree t. Each $P_j \in Receivers$ sends its share of $f(k-1)$ to party P_k (for $k \in [n]$, including parties outside *Receivers*). Each party P_k then applies online error correction to recover $f(k-1)$: it receives shares from $|Receivers| \geq n-t$ parties, of which at most t may be corrupt, for a polynomial of degree t. This works since $n - t > t + 2t = 3t$, which holds because $n > 4t$.
- **Phase 2:** All n parties broadcast their value $f(k-1)$. The vector $(f(0), \ldots, f(n-1))$ lies on a polynomial of degree at most $n-2t-1$, and at most t of the n broadcast values may be incorrect. Online error correction works since $n > (n-2t-1)+2t = n-1$. After this step, all parties can recover the polynomial f and hence the signature scalars $f(0), \ldots, f(n-2t-1)$.

This two-phase protocol is a generalization of the public reconstruction protocol of [DN07, CP17] (see also the discussion in Section A).

8.4 Implementations of dealer/receiver agreement

The dealer/receiver agreement protocol takes as input the local approval decisions of all parties and produces agreed-upon sets *Dealers* and *Receivers* satisfying the conditions in Section 8.1. We present three implementations with different trade-offs in communication complexity, computational cost, and cryptographic assumptions.

In all three implementations, the high-level structure is the same. Each party P_k locally constructs an undirected approval graph G_k and runs the star-finding algorithm (or the ILP optimization) to obtain a candidate pair $(Dealers_k, Receivers_k)$. The implementations differ in how the approval graph is built, how candidates are proposed, and how agreement is reached.

In all cases, after the ACS completes, proposals are processed in some agreed ordering and the first proposal that passes all verification checks is adopted. As part of this verification, parties must check that the proposed sets satisfy the size requirements $|Dealers| \geq n-2t$ and $|Receivers| \geq n-t$. A proposal from a corrupt party may have undersized sets or fail other verification checks; such a

proposal is simply skipped and the proposing party is identified as corrupt and eliminated. Since the ACS output includes at least $n - 2t$ honest parties, each of whose proposals is guaranteed to be valid, a valid proposal will always be found. An implementation may also choose to process proposals in decreasing order of $|\mathit{Dealers}|$ to maximize the yield of presignatures per batch.

8.4.1 Implementation 1: RBC-based

This is the simplest implementation.

Approval dissemination. For each pair of parties (P_i, P_j) with $i \neq j$, there is an RBC instance on channel (i, j). When P_j approves P_i, party P_j reliably broadcasts a YES message on channel (i, j). (Since there is only one possible value for this channel, the standard Bracha protocol can be simplified a bit.)

Graph construction. When party P_k receives YES on both channels (i, j) and (j, i), it adds the edge $\{i, j\}$ to its local graph G_k.

Proposal. P_k waits until it can find an (n, t)-star $(\mathit{Dealers}_k, \mathit{Receivers}_k)$ in G_k, and then reliably broadcasts $(\mathit{Dealers}_k, \mathit{Receivers}_k)$.

Agreement. An ACS protocol instance is used to agree on $n - t$ parties whose proposals have been reliably broadcast. The ACS uses an asynchronous validity predicate: a party P_k's proposal is valid if the RBC of $(\mathit{Dealers}_k, \mathit{Receivers}_k)$ has completed and, for every $P_i \in \mathit{Dealers}_k$ and $P_j \in \mathit{Receivers}_k$ with $i \neq j$, the RBC on channel (i, j) has delivered YES. Note that only one direction is checked: every receiver must approve every dealer, but not vice versa. The parties adopt the $(\mathit{Dealers}, \mathit{Receivers})$ pair of the first valid party in some agreed ordering of the ACS output.

Communication complexity. There are $\approx n^2$ RBC instances, each carrying a single-bit payload. Each RBC instance has communication complexity $O(n^2)$, leading to an overall communication complexity of $O(n^4)$. In fact, the *message* complexity is also $O(n^4)$. In practice, many of the short messages can be coalesced into larger messages between each pair of parties, mitigating the overhead.

8.4.2 Implementation 2: BLS multisignatures

This implementation reduces the number of messages by replacing the n^2 RBC instances with transferable BLS signatures, at the cost of introducing a pairing-based cryptographic assumption.

Approval dissemination. When P_j approves P_i, it sends a BLS signature $\sigma_j^{(i)}$ on the message $(\mathsf{approve}, i)$ to all parties. Since BLS signatures are transferable, no RBC is needed for this step.

Graph construction. Party P_k builds its local graph G_k as in Implementation 1, adding edge $\{i, j\}$ when it holds both $\sigma_j^{(i)}$ and $\sigma_i^{(j)}$. For computational efficiency, P_k does *not* verify individual signatures at this stage; it proceeds optimistically.

Proposal. P_k runs star-finding on G_k to obtain $(\mathit{Dealers}_k, \mathit{Receivers}_k)$. For each dealer $P_i \in \mathit{Dealers}_k$, P_k aggregates the individual signatures $\{\sigma_j^{(i)}\}_{j \in \mathit{Receivers}_k}$ into a BLS multisignature $\mu_k^{(i)}$. P_k then batch-verifies all $|\mathit{Dealers}_k|$ multisignatures in a single operation. If verification passes, P_k reliably broadcasts its proposal $(\mathit{Dealers}_k, \mathit{Receivers}_k, \{\mu_k^{(i)}\}_{i \in \mathit{Dealers}_k})$. If verification fails, P_k falls back to checking individual signatures, removes edges corresponding to invalid signatures from G_k, and reruns star-finding. This fallback is guaranteed to identify at least one corrupt party (which sent an invalid signature), so in steady state the optimistic path is taken.

Agreement. An ACS protocol instance agrees on $n - t$ parties whose proposals have been reliably broadcast, *without* verifying any multisignatures. After the ACS completes, the parties process proposals in an agreed ordering. For each proposal, the parties batch-verify the included multisignatures. The first proposal that passes verification is adopted. A party whose multisignatures fail verification is identified as corrupt.

Communication complexity. The approval phase consists of point-to-point messages: each party sends up to $n - 1$ BLS signatures, each to all n parties. This is $O(n^3)$ messages of $O(\kappa)$ bits each, for $O(\kappa n^3)$ bits total. Each proposal RBC carries $O(n)$ group elements, i.e., $O(\kappa n)$ bits. Using the RBC protocol of [DXR21], each RBC costs $O(\kappa n^2)$ bits. With n proposals, the total is $O(\kappa n^3)$ bits.

8.4.3 Implementation 3: Hash-committed approvals with AVID

This implementation achieves $O(\kappa n^3)$ communication without requiring pairings or public-key signatures, using only hash functions in combination with AVID.

Commitment setup. At the start of each presignature batch, each party P_j chooses random secrets $r_{j,1}, \ldots, r_{j,n}$ and reliably broadcasts the commitment vector $(H(r_{j,1}), \ldots, H(r_{j,n}))$, where H is a hash function (assumed to be preimage resistant). This is a separate RBC instance for each party, carrying $O(\kappa n)$ bits.

Approval dissemination. When P_j approves P_i, it sends the preimage $r_{j,i}$ to all parties. Any party can verify the approval by checking $H(r_{j,i})$ against the ith entry of P_j's committed vector (obtained from P_j's commitment RBC).

Graph construction. Party P_k builds its local graph G_k as in Implementation 1, adding edge $\{i, j\}$ when it holds valid preimages $r_{j,i}$ and $r_{i,j}$ (verified against the respective commitment vectors).

Proposal. P_k's proposal includes the sets $(\mathit{Dealers}_k, \mathit{Receivers}_k)$ (encoded as bitmasks) together with the $O(n^2)$ preimages that serve as approval evidence. This makes the proposal size $O(\kappa n^2)$ bits, which is too large for direct RBC. Instead, P_k disseminates $(\mathit{Dealers}_k, \mathit{Receivers}_k)$ via RBC and its approval evidence using an AVID protocol (as in Section 2.6). With a message of size $|m| = O(\kappa n^2)$, the AVID-M protocol has dispersal cost $O(\kappa n^2)$ bits per instance (the data term dominates the $O(\kappa n \log n)$ Merkle overhead). With n instances, the total dispersal cost is $O(\kappa n^3)$ bits.

Agreement. An ACS protocol instance agrees on $n - t$ parties whose AVID dispersals have completed. The ACS uses an asynchronous validity predicate requiring that the commitment RBCs have completed for all parties in the proposer's *Receivers* set. This is a valid asynchronous predicate: if one honest party has seen these RBC outputs, all honest parties eventually will. The use of AVID to disseminate proposals, followed by ACS to select a proposal and subsequent retrieval, follows the disperse-agree-retrieve paradigm used in multi-valued Byzantine agreement protocols (see, e.g., [LLTW20]).

After the ACS completes, the parties process proposals in an agreed ordering. For each proposal, each party retrieves the full proposal from the AVID (costing $O(\kappa n^2)$ bits per retrieval), then verifies the included preimages against the committed hash vectors. The first valid proposal is adopted.

Each invalid proposal identifies a corrupt party, so at most t retrievals are needed before an honest party's proposal is reached. All parties reach the same conclusion since they process proposals in the same order and verification is deterministic.

Communication complexity. The commitment RBCs cost $O(\kappa n^3)$ bits total (n instances, each carrying $O(\kappa n)$ bits). The approval phase (point-to-point preimage dissemination) costs $O(\kappa n^3)$ bits. AVID dispersal costs $O(\kappa n^3)$ bits total. Proposal retrieval costs $O(\kappa n^3)$ for all parties to retrieve a single proposal. In the worst case, there could be $O(n)$ retrieved proposals — some number of invalid proposals from corrupt parties followed by one valid proposal from an honest party. However, each invalid proposal identifies a corrupt party, which can be excluded from all future executions of this stage. So in a steady state, the communication complexity per presignature batch is $O(\kappa n^3)$.

Remark 8.4. *One could use ordinary digital signatures (e.g., Schnorr signatures) in place of the hash-commitment scheme, eliminating the need for the commitment RBCs. In this variant, P_j signs $(\mathsf{approve}, i)$ and sends the signature to all parties; the rest of the protocol proceeds identically. However, the hash-based approach seems preferable overall: verifying a proposal requires $O(n^2)$ hash evaluations rather than $O(n^2)$ signature verifications, which is orders of magnitude faster, while the additional commitment RBCs are dominated by other communication costs.*

8.5 Communication complexity

We now analyze the amortized communication complexity per signature per party in the simplified protocol. We assume $n = 4t + 1$, so $t \approx n/4$.

For a given batch, this cost depends on the size of the dealer set, which determines the "yield" of the batch randomness extraction step. The "minimum yield" is $t+1 \approx n/4$, which is guaranteed by the star finding algorithm in Section B. However, as suggested by the experiments in Section B.6, in practice we can expect to get a "high yield" of $\approx 2t + 1 \approx n/2$.

8.5.1 GoAVSS costs

We first consider the amortized cost per shared polynomial for a single instance of the simplified GoAVSS protocol.

- It does not make use of the SMD subprotocol. Rather, the dealer sends encrypted shares directly to each receiver. So this step contributes $\approx n\lambda$ to this cost. (Contrast this with $\approx 6n\lambda$ for the SMD subprotocol.)

- The only other significant cost is that associated with the RBC of the group elements. Using [LS24], this is $\frac{4}{3}n\lambda''$. (This is somewhat better than the $\frac{3}{2}n\lambda''$ cost in the original GoAVSS protocol, simply because we are using an $(n, n-t)$ code and t is smaller relative to n.)

So this gives a total of

$$\approx n\lambda + \tfrac{4}{3}n\lambda''$$

for a single GoAVSS.

To translate this to the amortized cost per presignature per party, we divide by n and multiply by

$$\theta := \begin{cases} 4, & \text{in the minimum-yield case;} \\ 2, & \text{in the high-yield case.} \end{cases}$$

So the amortized communication cost per presignature per party is

$$\approx \theta\lambda + \theta\tfrac{4}{3}\lambda''$$

8.5.2 Public reconstruction costs

In the two-phase public reconstruction, secrets are processed in groups of $n - 2t \approx n/2$. In Phase 1, each receiver sends one share per secret to the designated party, and in Phase 2, all n parties broadcast one share per secret. We upper bound the cost for any individual party, which may or may not be in the receiver set; in the worst case, each party sends $\approx 2n\lambda$ bits per group across both phases. So the amortized communication cost per signature per party is $\approx 4\lambda$. This is a modest improvement over the 6λ cost in the original pipeline (where groups of size $t + 1 \approx n/3$ are used).

8.5.3 Signature availability

If the signature availability mechanism of Section 6 is retained, its amortized cost per signature per party is $\approx \frac{4}{3}(\lambda + \lambda')$, which is a modest improvement over the $1.5(\lambda + \lambda')$ cost in the original pipeline.

8.5.4 Putting it all together

So the total amortized communication cost per signature per party is

$$\approx (\theta + 5\tfrac{1}{3})\lambda + \tfrac{4}{3}\lambda' + \theta\tfrac{4}{3}\lambda''$$

With λ and λ' equal to 32 bytes and λ'' equal to 64 bytes (as is typical), this gives a total of

$$213 + 117\theta$$

bytes. In the high-yield case, where $\theta = 2$, this is 447 bytes or 3.5K bits. In the minimum-yield case, where $\theta = 4$, this is 681 bytes or 5.5K bits. Compare this to our estimate of 9K bits for the happy path original pipeline (see Section 6.8.2). Even in the minimum-yield case, this is a significant improvement over the original, in two senses:

- the required bandwidth for each individual party is lower, which increases throughput, and

- even if we increase n to compensate for the lower corruption threshold, the overall communication cost across the system per signature will likely be lower, which decreases communication cost (and the amount of money that needs to be paid for it).

Indeed, assuming communication bandwidth bottlenecks throughput, with a 1 Gbps network connection, we can sustain a throughput of between 160K and 250K signatures per second (depending on yield). This is in contrast to the estimate of 100K signatures per second in the original pipeline (see Example 6.2 in Section 6.9.2).

8.6 Computational complexity

We do not carry out a detailed analysis of computational complexity. For a given value of t, the computational cost per signature per party will not vary greatly between the original pipeline (with $n = 3t + 1$) and simplified pipeline (with $n = 4t + 1$). Some costs will go down, while others will go up. For example, the encoding and hashing costs associated with SMD disappear altogether in the simplified pipeline.

One cost to pay special attention to is that of applying the Pascal matrix in the batch randomness extraction step. This is computationally the single most expensive step in the pipeline, accounting for 35% of the total computational cost in our running example with $t = 16$ and $n = 49$. Using the algorithm in Section 6.2 of [GS23], in the original pipeline with $n = 3t+1$, the amortized cost per presignature per party is $\approx \frac{1}{2}n$ group additions. The algorithm in Section 6.3 of [GS23], based on hyper-invertibility, can be used for certain values of t and n (which includes our running example with $t = 16$ and $n = 49$) at a cost of $\approx \frac{1}{3}n$ group additions per presignature.

In the simplified pipeline with $n = 4t + 1$, using the algorithm in Section 6.2 of [GS23], in the high-yield case, the amortized cost per presignature is $\frac{1}{2}n$ group additions, while in the minimum-yield case, it is $\frac{3}{8}n$ group additions. Note that when $t = 16$ and $n = 65$, in the minimum-yield case we can use the same hyper-invertible-based algorithm as in the $t = 16$ and $n = 49$ example, but in the high-yield case we cannot. Thus, in the minimum-yield case the cost of this step per presignature will generally be the same as in the original pipeline, while in the high-yield case, it will be roughly $1\frac{1}{3}$–2 times more expensive.

In our running example with $t = 16$ and $n = 49$, the estimated compute cost per signature was $\approx 5\,\mu s$ while the network transmission time was $\approx 10\,\mu s$, making the protocol network-bound (see Example 6.2). In the simplified protocol with $n = 65$, the network transmission time roughly halves, while the overall compute cost per signature may roughly double. This shifts the bottleneck from network to compute. However, nearly all of the computational work is trivially parallelizable across cores, so with modest additional compute resources, the simplified protocol can fully exploit its lower communication cost to achieve higher throughput.

8.7 An alternative: fully point-to-point GoAVSS

One can eliminate RBC entirely from the simplified GoAVSS by sending the group elements $\{\mathcal{S}_\ell\}$ point-to-point rather than via reliable broadcast. The commitment structure under ρ_1 is replaced by a flat hash vector, also sent point-to-point, and all other data ($\{\mathcal{T}_k\}$, $\{h_r\}$, $\{c_k\}$, h^*) is handled as before. Since parties outside the receiver set may not hold the group elements from all dealers, a presignature availability step is introduced after batch randomness extraction: the receivers disseminate the computed presignatures to all parties using an erasure-coded mechanism (as in Section 6.6). This reduces the additive overhead of the GoAVSS communication complexity from

$O(\kappa n^2 \log n)$ to $O(\kappa n^2)$, so that $L \gg n$ suffices (rather than $L \gg n \log n$) to achieve the stated amortized complexity. However, the presignature availability step introduces a communication asymmetry: parties outside the receiver set may incur a slightly higher per-presignature reception communication cost. We opted to present the RBC-based design presented above, which avoids this asymmetry and slight increase in reception communication, at the cost of the $\log n$ factor in the additive overhead. However, both approaches have their merits.

Acknowledgments

Thanks to Ran Canetti for discussions on star finding. The fast star finder algorithm in Section B was inspired by these discussions and much of the credit really belongs to Ran. This work was partially done while the author was employed at Offchain Labs. Claude Opus 4.5/4.6 was used in the writing of this paper as follows: (1) for writing various benchmark implementations; (2) for drafting certain portions of text, through an iterative process between the author and Claude; (3) for proofreading and checking logical correctness of protocol descriptions, security arguments, and cost estimates.

A Polynomial Extension via Finite Differences

A.1 Problem Statement

Let f be a polynomial of degree t with coefficients in some ring. Given the values $f(0), f(1), \ldots, f(t)$, we wish to compute $f(t+1), f(t+2), \ldots, f(n)$ efficiently. Our goals are: (i) use only additions (or subtractions) in the extension phase; (ii) use $O(t)$ working space beyond the output array; (iii) minimize the total number of arithmetic operations.

A.2 Background: Finite Differences

The **forward difference operator** Δ is defined by $\Delta g(x) := g(x+1) - g(x)$. Higher-order differences are defined recursively: $\Delta^k g := \Delta(\Delta^{k-1} g)$, with $\Delta^0 g := g$.

If g is a polynomial of degree m, then Δg is a polynomial of degree $m-1$. Consequently, $\Delta^t f$ is constant for any polynomial f of degree t.

The key property we exploit is the recurrence:

$$\Delta^k f(i+1) = \Delta^k f(i) + \Delta^{k+1} f(i). \tag{26}$$

This follows directly from the definition: $\Delta^k f(i+1) - \Delta^k f(i) = \Delta(\Delta^k f)(i) = \Delta^{k+1} f(i)$.

A.3 Algorithm

Algorithm: Polynomial Extension via Finite Differences

Input: $v[0..t]$ where $v[i] = f(i)$ for a degree-t polynomial f; target index $n \geq t$
Output: $result[0..n]$ where $result[i] = f(i)$

```
// Step 1: Build the difference table
a ← v
for k = 1 to t do
    for i = t down to k do
        a[i] ← a[i] − a[i − 1]

// Step 2: Extend to position n
result[0] ← a[0]
for i = 1 to n do
    for k = 1 to t do
        a[k − 1] ← a[k − 1] + a[k]
    result[i] ← a[0]

return result
```

Note that we could just copy the original vector v into the first $t+1$ entries of *result*, but this would not make the algorithm significantly more efficient.

A.4 Correctness

We establish correctness by stating invariants for each step.

Step 1 invariant. After k iterations of the outer loop (for $k = 0, 1, \ldots, t$), the array a satisfies:

$$a[j] = \begin{cases} \Delta^j f(0) & \text{if } j < k \\ \Delta^k f(j-k) & \text{if } j \geq k \end{cases}$$

Base case ($k = 0$): The condition $j < 0$ is never satisfied. For all $j \geq 0$, we need $a[j] = \Delta^0 f(j) = f(j)$, which matches the initial state $a[j] = v[j] = f(j)$.

Inductive step ($k > 0$): Assume the invariant holds after $k-1$ iterations. During iteration k, the inner loop processes $i = t, t-1, \ldots, k$. Each update $a[i] \leftarrow a[i] - a[i-1]$ computes $\Delta^{k-1} f(i-k+1) - \Delta^{k-1} f(i-k) = \Delta^k f(i-k)$, where the last equality follows from (26). After the inner loop, position $i \geq k$ holds $\Delta^k f(i-k)$. Positions $0, \ldots, k-1$ are unchanged; they satisfy $a[j] = \Delta^j f(0)$ for $j < k-1$, and $a[k-1] = \Delta^{k-1} f((k-1)-(k-1)) = \Delta^{k-1} f(0)$, so $a[j] = \Delta^j f(0)$ for all $j < k$. This establishes the invariant after k iterations.

After t iterations, we have $a[j] = \Delta^j f(0)$ for all $j = 0, \ldots, t$.

Step 2 invariant. After i iterations of the outer loop (for $i = 0, 1, \ldots, n$), the array a satisfies $a[k] = \Delta^k f(i)$ for all $k = 0, \ldots, t$.

Base case ($i = 0$): This is exactly the postcondition of Step 1. In particular, $a[0] = \Delta^0 f(0) = f(0)$, which is stored in *result*$[0]$.

Inductive step ($i > 0$): Assume the invariant holds after $i-1$ iterations. During iteration i, the update $a[k-1] \leftarrow a[k-1] + a[k]$ replaces $\Delta^{k-1} f(i-1)$ with $\Delta^{k-1} f(i-1) + \Delta^k f(i-1) = \Delta^{k-1} f(i)$, where the last equality follows from (26). Processing $k = 1, 2, \ldots, t$ in order ensures we use $a[k] = \Delta^k f(i-1)$ before it is overwritten with $\Delta^k f(i)$. The entry $a[t] = \Delta^t f$ is constant and remains correct. This establishes the invariant after i iterations.

In particular, $a[0] = f(i)$ after i iterations, and this value is stored in *result*$[i]$.

A.5 Operation Count

Theorem A.1. *The algorithm uses exactly $\frac{t(t+1)}{2}$ subtractions and tn additions.*

Proof. Step 1 (build differences): The inner loop executes $(t-k+1)$ times for each $k = 1, \ldots, t$. Total subtractions: $\sum_{k=1}^{t}(t-k+1) = t + (t-1) + \cdots + 1 = \frac{t(t+1)}{2}$.

Step 2 (extend): The outer loop runs n times, each with t additions in the inner loop (for $k = 1, \ldots, t$). Total: tn additions. □

A.6 Matrix Interpretation

The algorithm can be understood in terms of matrix operations. Let $M^{(-)}$ denote the $(t+1)\times(t+1)$ signed lower-triangular Pascal matrix with entries

$$M^{(-)}_{k,j} = (-1)^{k-j}\binom{k}{j} \quad \text{for } 0 \le j \le k \le t.$$

For example, with $t = 3$:

$$M^{(-)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ -1 & 3 & -3 & 1 \end{pmatrix}.$$

This matrix transforms evaluations to differences: multiplying $M^{(-)}$ by the vector $(f(0), f(1), \ldots, f(t))^T$ yields $(\Delta^0 f(0), \Delta^1 f(0), \ldots, \Delta^t f(0))^T$. This follows from the classical formula $\Delta^k f(0) = \sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j} f(j)$. Step 1 computes this matrix-vector product.

Let M denote the $(n+1) \times (t+1)$ lower-triangular Pascal matrix with entries

$$M_{i,k} = \binom{i}{k} \quad \text{for } 0 \le k \le \min(i,t), \quad 0 \le i \le n.$$

For example, with $t = 2$ and $n = 4$:

$$M = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 2 & 1 \\ 1 & 3 & 3 \\ 1 & 4 & 6 \end{pmatrix}.$$

This matrix transforms differences to evaluations: multiplying M by $(\Delta^0 f(0), \ldots, \Delta^t f(0))^T$ yields $(f(0), f(1), \ldots, f(n))^T$. This follows from Newton's forward difference formula $f(i) = \sum_{k=0}^{t}\binom{i}{k}\Delta^k f(0)$. Step 2 computes this matrix-vector product.

The full algorithm computes the product $M \cdot M^{(-)}$, which is an $(n+1) \times (t+1)$ matrix. Its top $(t+1) \times (t+1)$ block is the identity matrix (since the first $t+1$ outputs are just the inputs $f(0), \ldots, f(t)$). Its bottom $(n-t) \times (t+1)$ block contains the Lagrange interpolation coefficients: the entry in row $i > t$ and column j equals $\ell_j(i) = \prod_{m \ne j}\frac{i-m}{j-m}$, which is the value of the j-th Lagrange basis polynomial at i.

A.7 Summary

The finite difference method extends a degree-t polynomial from $t+1$ known values to $n+1$ values using $\frac{t(t+1)}{2}$ subtractions and tn additions. Working space is $O(t)$ beyond the output array. The extension phase (Step 2) uses only additions.

A.8 Application to Secret Sharing

In Shamir secret sharing, we generate a random degree-t polynomial f with $f(0) = s$ (the secret) and distribute shares $f(1), f(2), \ldots, f(n)$ to n parties. Typically, the polynomial is specified by choosing random coefficients. However, we could equivalently specify it by choosing random values for $f(1), \ldots, f(t)$ and setting $f(0) \coloneqq s$. Then we can apply the above algorithm to get the shares $f(1), \ldots, f(n)$ at a cost of $t(t+1)/2$ subtractions and tn additions. Moreover, we can equivalently specify the polynomial by choosing random values for $\Delta^1 f(0), \ldots, \Delta^t f(0)$ and setting $\Delta^0 f(0) = f(0) \coloneqq s$. This representation is equally valid because Step 1 is a bijection: each operation $a[i] \leftarrow a[i] - a[i-1]$ is invertible. With this approach, we can skip Step 1 entirely and run only Step 2. The cost to compute $f(1), \ldots, f(n)$ is then just tn additions.

A.9 An Optimized Algorithm for Step 2

The original Step 2 algorithm maintains the state $(\Delta^0 f(i), \Delta^1 f(i), \ldots, \Delta^t f(i))$ and advances i one step at a time, using t additions per step for a total of tn additions. We now present an alternative algorithm that reduces this to $tn - \frac{t(t-1)}{2}$ additions. This algorithm can be derived by applying the "transposition principle" [BLS03] to the algorithm in Section 6.2 of [GS23], but we give a self-contained presentation here.

The key idea is to process the differences in reverse order, from $k = t-1$ down to $k = 0$. For each k, we propagate the partial sums forward through the output array.

Algorithm: Optimized Step 2

Input: $diff[0..t]$ where $diff[k] = \Delta^k f(0)$; target index $n \geq t$
Output: $B[0..n]$ where $B[i] = f(i)$

```
// Initialize: B[j] = Δ^t f(0) for j ≥ t
for j = t to n do
    B[j] ← diff[t]

// Process differences from k = t − 1 down to 0
for k = t − 1 down to 0 do
    B[k] ← diff[k]
    for j = k to n − 1 do
        B[j + 1] ← B[j + 1] + B[j]

return B
```

A.9.1 Correctness

We prove correctness by establishing an invariant after processing each value of k.

Invariant. After processing iteration k (for $k = t, t-1, \ldots, 0$), the array B satisfies: for all $j \geq k$,

$$B[j] = \Delta^k f(j-k). \tag{27}$$

This is directly analogous to the Step 2 invariant "$a[k] = \Delta^k f(i)$" from the original algorithm—we have simply swapped which index is fixed and which varies.

We prove this by downward induction on k.

Base case ($k = t$): After initialization, $B[j] = \Delta^t f(0)$ for all $j \geq t$. Since $\Delta^t f$ is constant for a degree-t polynomial, we have $\Delta^t f(j-t) = \Delta^t f(0)$, so the invariant holds.

Inductive step: Assume the invariant holds after processing iteration $k+1$, i.e., $B[j] = \Delta^{k+1} f(j-k-1)$ for all $j \geq k+1$. When we process iteration k, we first set $B[k] = \Delta^k f(0)$. Then the inner loop performs $B[j+1] \leftarrow B[j+1] + B[j]$ for $j = k, k+1, \ldots, n-1$.

We show by induction on j (from k to n) that after the inner loop completes, $B[j] = \Delta^k f(j-k)$ for all $j \geq k$.

For $j = k$: After setting $B[k] = \Delta^k f(0)$, this position is not modified by the inner loop (since the update $B[j+1] \leftarrow B[j+1]+B[j]$ modifies $B[j+1]$, not $B[j]$). We have $B[k] = \Delta^k f(0) = \Delta^k f(k-k)$, as required.

For $j > k$: The update $B[j] \leftarrow B[j] + B[j-1]$ adds the already-updated value $B[j-1]$ to the previous value of $B[j]$. By the inductive hypothesis on k, the previous value is $B[j]^{\text{old}} = \Delta^{k+1} f(j-k-1)$. By the inductive hypothesis on j, the updated value of $B[j-1]$ is $B[j-1] = \Delta^k f(j-1-k)$. Therefore, the new value of $B[j]$ is

$$\begin{aligned} B[j] &= B[j]^{\text{old}} + B[j-1] \\ &= \Delta^{k+1} f(j-k-1) + \Delta^k f(j-k-1) \\ &= \Delta^k f(j-k), \end{aligned}$$

where the last equality follows from the recurrence (26).

Conclusion. After processing iteration $k = 0$, the invariant gives $B[j] = \Delta^0 f(j) = f(j)$, since $\Delta^0 g = g$ by definition.

A.9.2 Operation Count

Theorem A.2. *The optimized Step 2 algorithm uses exactly $tn - \frac{t(t-1)}{2}$ additions.*

Proof. The initialization loop performs no additions (only assignments). For each $k = t-1, t-2, \ldots, 0$, the inner loop executes $(n-k)$ additions. The total number of additions is

$$\sum_{k=0}^{t-1} (n-k) = tn - \sum_{k=0}^{t-1} k = tn - \frac{(t-1)t}{2} = tn - \frac{t(t-1)}{2}.$$

□

So the optimized algorithm saves exactly $\frac{t(t-1)}{2}$ additions compared to the original Step 2.

A.9.3 Trade-offs

The optimized algorithm uses $O(n)$ working space (the output array B) rather than $O(t)$ working space. However, the output array of size $n + 1$ must be allocated regardless, and for typical parameters of interest, it will fit in L1 cache.

A.10 Implementation: Constant-Time Lazy Reduction

In cryptographic applications, it is important that arithmetic operations run in constant time to prevent timing side-channel attacks. A common technique for modular addition is *lazy reduction*: instead of maintaining values in the canonical range $[0, P)$, we allow values in a larger range and reduce only enough to prevent overflow.

Invariant. We maintain all values in the range $[-P, P)$. After adding or subtracting two values in this range, the result lies in $[-2P, 2P)$. We then add or subtract P to bring the result back to $[-P, P)$: if the result is negative, add P; if non-negative, subtract P.

Constant-time conditional. The challenge is to add $+P$ or $-P$ without branching. We use the identity $-P = \sim P + 1$ (two's complement negation). Let $negate = 1$ if we want to subtract P, and $negate = 0$ if we want to add P. Then:

- If $negate = 1$: compute $P \oplus \texttt{0xFFF...} = \sim P$, then add with initial carry 1, yielding $\sim P + 1 = -P$.
- If $negate = 0$: compute $P \oplus 0 = P$, then add with initial carry 0, yielding $+P$.

Implementation. Fig. 1 gives a C++ implementation of constant-time lazy reduction for a 320-bit integer (five 64-bit limbs, to accommodate values in $[-P, P)$ for a 256-bit prime P):

This reduction is called after every addition or subtraction. The entire operation (add/subtract plus reduction) requires no branches and runs in constant time regardless of the input values.

Normalization for output. When storing results, we convert from the lazy representation $[-P, P)$ to the canonical representation $[0, P)$. This is simpler than lazy reduction: if the value is negative (indicated by the sign bit of the fifth limb), add P; otherwise do nothing. The output can then be stored in four limbs instead of five. This normalization is performed once per output element, adding negligible overhead.

A.11 Batched Extension with Blocking

In many applications, we must extend a large batch of L polynomials (all of degree t) from their values at $\{0, \ldots, t\}$ to values at $\{0, \ldots, n\}$. Furthermore, the output is often needed in *transposed* form: rather than L vectors of length $n + 1$, we need $n + 1$ vectors of length L, where vector i contains the value $f_j(i)$ for all polynomials j. For example, in secret sharing, the dealer wants to compute a vector of shares for each of n parties.

A naive approach—extending each polynomial independently and then transposing—leads to poor cache behavior. A better strategy is *blocked batching*: process polynomials in blocks of size B, chosen so that the working memory fits in L2 cache.

```
inline void reduce_ct(BigIntMod& a) {
    u64 sign = a.limbs[4] >> 63;      // 1 if negative, 0 otherwise
    u64 negate = 1 - sign;            // 1 to subtract P, 0 to add P
    u64 mask = -(i64)negate;          // 0xFFFF... if negate, else 0
    // Add (P ^ mask) with initial carry = negate
    // negate=1: P ^ 0xFFF... = ~P, carry=1, so we add ~P+1 = -P
    // negate=0: P ^ 0 = P, carry=0, so we add P
    u128 s;
    u64 carry = negate;
    s = (u128)a.limbs[0] + (P[0] ^ mask) + carry;
    a.limbs[0] = (u64)s; carry = (u64)(s >> 64);
    s = (u128)a.limbs[1] + (P[1] ^ mask) + carry;
    a.limbs[1] = (u64)s; carry = (u64)(s >> 64);
    s = (u128)a.limbs[2] + (P[2] ^ mask) + carry;
    a.limbs[2] = (u64)s; carry = (u64)(s >> 64);
    s = (u128)a.limbs[3] + (P[3] ^ mask) + carry;
    a.limbs[3] = (u64)s; carry = (u64)(s >> 64);
    s = (u128)a.limbs[4] + mask + carry;
    a.limbs[4] = (u64)s;
}
```

Figure 1: Constant-time lazy reduction

Algorithm. Let $diff[k][j] = \Delta^k f_j(0)$ be the input differences and $output[i][j] = f_j(i)$ be the desired output, with each $output[i]$ stored as a contiguous array of length L.

// Working array W: $(n+1) \times B$ elements
for each block $b = 0, 1, \ldots, \lceil L/B \rceil - 1$ do
    $j_{\text{start}} \leftarrow b \cdot B$
    $j_{\text{end}} \leftarrow \min(j_{\text{start}} + B, L)$

    // Initialize $W[t..n]$ with Δ^t values
    for $i = t$ to n do
        for $j = 0$ to $j_{\text{end}} - j_{\text{start}} - 1$ do
            $W[i][j] \leftarrow diff[t][j_{\text{start}} + j]$

    // Optimized Step 2 for all polynomials in block
    for $k = t - 1$ down to 0 do
        for $j = 0$ to $j_{\text{end}} - j_{\text{start}} - 1$ do
            $W[k][j] \leftarrow diff[k][j_{\text{start}} + j]$
        for $i = k$ to $n - 1$ do
            for $j = 0$ to $j_{\text{end}} - j_{\text{start}} - 1$ do
                $W[i+1][j] \leftarrow W[i+1][j] + W[i][j]$

    // Write block to output
    for $i = 0$ to n do
        copy $W[i][0..B-1]$ to $output[i][j_{\text{start}}..j_{\text{end}} - 1]$

Block size selection. The block size B should be small enough that the working data for both the matrix-vector and extension phases remains cache-friendly. In practice, we find that B in the range 16–32 works well across both phases and across architectures.

Benchmarks for the batched extension phase (and the full resample-and-extend pipeline) are reported in Section A.13.

A.12 Resampling from Arbitrary Points

In some applications, we are given values $f(x_0), \ldots, f(x_t)$ at arbitrary distinct points $x_0, \ldots, x_t$ and wish to compute $f(0), f(1), \ldots, f(n)$. For example, this arises when verifying that a set of points lies on a low-degree polynomial: we can interpolate through $t+1$ points, extend to additional evaluation points, and check consistency.

The map from $(f(x_0), \ldots, f(x_t))$ to $(f(0), \ldots, f(t))$ is linear. Let W denote the $(t+1) \times (t+1)$ matrix for this transformation. The entry $W_{i,j}$ equals $\ell_j(i)$, where ℓ_j is the j-th Lagrange basis polynomial for the nodes $x_0, \ldots, x_t$:

$$\ell_j(x) \coloneqq \prod_{m \neq j} \frac{x - x_m}{x_j - x_m}.$$

The matrix W can be computed in $O(t^2)$ field operations using the barycentric form of Lagrange interpolation.

Barycentric weights. Define $\omega(x) = \prod_{m=0}^{t}(x - x_m)$ and the barycentric weights $w_j = 1/\omega'(x_j)$ where $\omega'(x_j) = \prod_{m \neq j}(x_j - x_m)$. Then

$$\ell_j(i) = \frac{\omega(i)}{(i - x_j) \cdot \omega'(x_j)} = \frac{\omega(i) \cdot w_j}{i - x_j}$$

for $i \notin \{x_0, \ldots, x_t\}$. If $i = x_k$ for some k, then $\ell_j(i) = \delta_{jk}$ (Kronecker delta).

Algorithm to compute W.

1. Compute $\omega'(x_j) = \prod_{m \neq j}(x_j - x_m)$ for each $j = 0, \ldots, t$. This requires $O(t)$ operations per j, for $O(t^2)$ total.
2. Compute $\omega(i) = \prod_{m=0}^{t}(i - x_m)$ for each $i = 0, \ldots, t$. This requires $O(t)$ operations per i, for $O(t^2)$ total.
3. For each entry (i, j): if $i = x_k$ for some k, set $W_{i,j} \coloneqq \delta_{jk}$; otherwise, set $W_{i,j} \coloneqq \omega(i)/((i - x_j) \cdot \omega'(x_j))$. Each entry requires $O(1)$ operations, for $O(t^2)$ total.

The total cost is $O(t^2)$ field operations.

Application. Once W is computed, we can transform values at arbitrary points to values at $\{0, \ldots, t\}$ via a matrix-vector product in $O(t^2)$ operations. The extension algorithm then computes $f(t+1), \ldots, f(n)$ using only additions. When processing a large batch of instances with the same evaluation points $x_0, \ldots, x_t$, the matrix W is computed once and reused, amortizing its cost across the batch.

In fact, we can apply Step 1 of the extension algorithm to the columns of W to obtain a matrix that maps values at arbitrary points to the vector $\Delta^0 f(0), \ldots, \Delta^t f(0)$, so that to compute the values $f(0), \ldots, f(n)$, we just run Step 2 of the extension algorithm. Transforming W in this way costs $O(t^3)$ subtractions, but for large enough batches, the amortized cost is negligible. We can also transform W so that each entry is scaled by an appropriate Montgomery reduction constant. This way, to compute a matrix/vector product modulo an integer P, we can compute the inner product of each row of the matrix with the vector as an unreduced integer, and then quickly reduce these integers modulo P via Montgomery reduction.

Benchmarks for the matrix-vector product and the full resample-and-extend pipeline are reported in Section A.13.

A.13 Benchmarks: Resample-and-Extend Pipeline

We implemented the full resample-and-extend pipeline described in the preceding sections and benchmarked each phase independently, as well as the end-to-end pipeline. All arithmetic is over the secp256k1 group order (a 256-bit prime), using constant-time lazy reduction for the extension phase and Montgomery multiplication for the matrix-vector phase.

Parameters. We fix $t = 16$, $n = 49$, and a batch of $L = 10\,000$ polynomials. The block size is $B = 32$. The evaluation points $x_0, \ldots, x_t$ are the last $t+1$ integers in $\{0, \ldots, n\}$, i.e., $x_j = n - t + j$.

Baking Step 1 into the matrix. As described in Section A.12, we can precompute the matrix W so that it maps evaluations at arbitrary points directly to the finite difference vector $(\Delta^0 f(0), \ldots, \Delta^t f(0))$, rather than to evaluations at $(f(0), \ldots, f(t))$. This "bakes" Step 1 into the matrix, allowing the pipeline to skip the $O(t^2 B)$ subtractions per block that would otherwise be needed to convert evaluations to differences. The bake costs $O(t^3)$ extra subtractions during the one-time precomputation, which amortizes over the batch.

Implementation notes. The matrix entries are stored in *extended Montgomery form*: each entry $W_{i,j}$ is represented as $W_{i,j} \cdot R_{\text{ext}} \bmod P$, where $R_{\text{ext}} = 2^{320}$. This allows us to accumulate the inner product for each row (17 multiply-accumulate operations) into a single wide integer and perform a single 5-round Montgomery reduction at the end, rather than reducing after each multiplication. Since $R_{\text{ext}} = 2^{320}$ provides enough headroom for $t+1 = 17$ accumulated products without overflow, the result after reduction is fully reduced modulo P.

For the batched matrix-vector phase, the input arrives in *column-major* layout (one length-L vector per evaluation point) but each matrix-vector product needs all $t + 1$ values for a single polynomial contiguously. Rather than performing a full matrix transpose, we use a block-local gather/scatter pattern: for each block of B polynomials, we gather the $B \times (t + 1)$ input values into a row-major local buffer, perform B matrix-vector products, then scatter the results back to column-major output arrays. For $B = 32$ and $t = 16$, the local buffer is $32 \times 17 \times 32 = 17\,408$ bytes, fitting comfortably in L1 cache.

Results. Table 2 reports the measured timings. The "matrix build" rows report the one-time cost to construct the $(t + 1) \times (t + 1)$ matrix (with or without baking Step 1); this is dominated by $O(t^2)$ Montgomery multiplications, plus $O(t^3)$ subtractions when baking. The "single matvec"

Table 2 Resample-and-extend pipeline timings for $t = 16$, $n = 49$, $B = 32$, on Apple M5.

Phase	**Total (µs)**	**Per poly (µs)**
Matrix build (unbaked)	10.9	—
Matrix build (baked)	13.7	—
Single matvec	1.49	—
Batched matvec ($L = 10\,000$)	14 782	1.48
Batched Step 2 only ($L = 10\,000$)	12 974	1.30
Full pipeline, unbaked ($L = 10\,000$)	30 848	3.08
Full pipeline, baked ($L = 10\,000$)	27 924	2.79

row reports the cost of one $(t + 1) \times (t + 1)$ matrix-vector product in isolation. The "batched matvec" and "batched Step 2" rows report the amortized per-polynomial cost of each phase when processing the full batch; the "batched Step 2" benchmark starts from precomputed differences and measures only the optimized extension (no Step 1). The "full pipeline" rows report the end-to-end cost *including* the one-time matrix build. In the unbaked pipeline, Step 1 subtractions are applied per block during the batch processing; in the baked pipeline, Step 1 is absorbed into the matrix precomputation.

Discussion. The batched matvec cost (1.48 µs/poly) is essentially identical to the single-matvec cost (1.49 µs), indicating that the gather/scatter overhead and column-major access pattern do not introduce a measurable penalty. The batched Step 2 costs 1.30 µs/poly, consistent with $t(n - (t - 1)/2) = 16 \times 41.5 = 664$ constant-time modular additions.

The gap between the unbaked and baked pipeline costs (3.08 vs. 2.79 µs/poly, or about 2900 µs across the batch) reflects the cost of Step 1: in the unbaked pipeline, $O(t^2)$ subtractions are performed per block across the entire batch, while in the baked pipeline, this work is folded into the matrix precomputation at a one-time cost of only 2.8 µs (the difference between the two matrix build times). For any non-trivial batch size, baking is clearly preferable.

Summing the batched matvec and batched Step 2 costs gives $1.48 + 1.30 = 2.78$ µs, which closely matches the baked pipeline cost of 2.79 µs/poly, confirming that the fused pipeline introduces no significant overhead beyond the two constituent phases.

B Finding Stars in Graphs

In this section, we recall the notion of an (n, t)-star in a graph and review Canetti's polynomial-time algorithm [Can96] for finding such a star provided the graph contains a sufficiently large clique. In addition, we give a new (and perhaps more straightforward) proof of the correctness of Canetti's algorithm. Finally, we give a more efficient version of Canetti's algorithm, running in time $O(n^2)$, rather than time $O(n^{2.5})$.

B.1 Definition of an (n, t)-star

Definition B.1. *Let $G = (V, E)$ be an undirected graph with n nodes, where $n > 2t$. An (n, t)-**star** in G is a pair (C, D), where*

(i) $C \subseteq D \subseteq V$,

(ii) *for all* $u \in C$ *and* $v \in D$, *with* $u \neq v$, *the nodes* u *and* v *are adjacent in* G.

(iii) $|C| \geq n - 2t$ *and* $|D| \geq n - t$, *and*

B.2 The algorithm

We present an algorithm that, on input a graph G with n nodes and a parameter t with $n - 2t > 0$, runs in polynomial time and returns either FAIL or an (n, t)-star in G. Moreover, if G contains a clique of size $n - t$, then the algorithm does not output FAIL.

To describe the algorithm, it is convenient to work with the **complement graph** $G' = (V, E')$, where E' consists of all edges $\{u, v\}$ (with $u \neq v$) such that $\{u, v\} \notin E$.

We define an (n, t)-**star′** in G' in the same way, except that in condition (ii) of the definition, we require u and v to be *non-adjacent* in G'. Clearly, (C, D) is an (n, t)-star in G if and only if (C, D) is an (n, t)-star′ in G', and so it suffices to find an (n, t)-star′ in G'.

Definition B.2. *For an edge* $m = \{x, y\} \in E'$, *a node* $z \in V$ *is called a* ***triangle head for*** m *if* z *is adjacent to both* x *and* y *in* G'.

The algorithm proceeds as follows:

Algorithm FindStar(G, t)

Input: Graph $G = (V, E)$ with $n = |V|$ nodes; parameter t with $n - 2t > 0$
Output: An (n, t)-star in G, or FAIL

compute the complement graph $G' = (V, E')$
compute a maximum matching M in G'
if $|M| > t$ then return FAIL

let $C \leftarrow \{u \in V : u$ is unmatched and not a triangle head for any edge in $M\}$
let $D \leftarrow \{v \in V : v$ is not adjacent in G' to any node in $C\}$

if $|C| < n - 2t$ or $|D| < n - t$ then return FAIL
return (C, D)

Remark B.3. *The algorithm clearly runs in polynomial time: computing the complement graph takes* $O(n^2)$ *time, computing a maximum matching can be done in* $O(n^{2.5})$ *time using the algorithm of Micali and Vazirani [MV80], and the remaining steps take* $O(n^2)$ *time.* □

B.3 Correctness of the algorithm

We now prove that the algorithm is correct. The key observation is that if G contains a clique of size $n - t$, then G' contains an independent set of size $n - t$, and this independent set constrains the structure of any matching in G'.

Lemma B.4. *Let* M *be a maximal matching in* G'. *Then no two unmatched nodes are adjacent in* G'.

Proof. If two unmatched nodes u and v were adjacent in G', we could add the edge $\{u, v\}$ to M, contradicting maximality. □

Lemma B.5. *Let M be a maximum matching in G' and let $m = \{x, y\} \in M$. If w is an unmatched node adjacent to x and z is an unmatched node adjacent to y, then $w = z$. In particular, m has at most one unmatched triangle head.*

Proof. If $w \neq z$, then (w, x, y, z) is an augmenting path, contradicting the maximality of M — in particular, replacing the edge $\{x, y\}$ by the two edges $\{w, x\}$ and $\{y, z\}$ gives a match of size $|M| + 1$. □

Theorem B.6. *Algorithm* FindStar *outputs either* FAIL *or an (n, t)-star in G. Moreover, if G contains a clique of size $n - t$, then it does not output* FAIL.

Proof. We first show that for any maximal matching M, the sets C and D defined by the algorithm satisfy the inclusion and connectivity requirements of a star (parts (i) and (ii) of the definition). By Lemma B.4, C is an independent set in the complement graph G', meaning no two nodes in C are adjacent in G'. Therefore, every node $u \in C$ is not adjacent in G' to any node in C, which means $u \in D$. Hence $C \subseteq D$. Moreover, every node $v \in D$ is, by definition, not adjacent in G' to any node in C. Therefore, (C, D) satisfies the inclusion and connectivity requirements. This means that if the algorithm does not output FAIL, it outputs an (n, t)-star in G.

It remains to show that if G contains a clique of size $n - t$, then it does not output FAIL. To this end, assume that $X \subseteq V$ is a clique in G with $|X| = n - t$. Then X is an independent set in G'.

Claim 1: $|M| \leq t$**.** Since X is an independent set in G', any edge in M must have at least one of its endpoints outside of X. Therefore, $|M| \leq |V \setminus X| = n - |X| = t$.

Note that this claim holds for *any* matching M.

Claim 2: $|C| \geq n - 2t$**.** We show this by proving that C contains all but at most $|M|$ nodes of X.

Consider any edge $m = \{x, y\} \in M$. We show that m excludes at most one node of X from C. There are two cases:

Case 1: Exactly one endpoint of m is in X, say $x \in X$ and $y \notin X$. Then x is matched, and hence $x \notin C$. If m has a triangle head z, then z is adjacent in G' to $x \in X$. Since X is an independent set in G', we have $z \notin X$. Thus, m excludes exactly one node of X from C (namely, x).

Case 2: No endpoint of m is in X. Then the only way m can exclude a node of X from C is via a triangle head. By Lemma B.5, there is at most one unmatched triangle head for m. Thus, m excludes at most one node of X from C (namely, m's triangle head, if any).

Summing over all edges in M, at most $|M|$ nodes of X are excluded from C. Since $|X| = n - t$ and $|M| \leq t$, we have

$$|C| \geq |X| - |M| \geq (n - t) - t = n - 2t.$$

Claim 3: $|D| \geq n - t$**.** We show that $|V \setminus D| \leq |M|$, which implies $|D| \geq n - |M| \geq n - t$.

By Lemma B.4, only matched nodes can be adjacent in G' to nodes in C. We claim that for each edge $m = \{x, y\} \in M$, at most one of x, y is adjacent in G' to some node in C. Suppose, to the contrary, that $w \in C$ is adjacent to x and $z \in C$ is adjacent to y. Since C contains no triangle heads, $w \neq z$, contradicting Lemma B.5.

Conclusion. These claims establish that the algorithm does not output FAIL, assuming G contains a clique of size $n - t$. □

B.4 A faster variant

We now describe a variant of Canetti's algorithm that runs in $O(n^2)$ time. The key observation is that in the proof of Theorem B.6, we really only require that M is maximal and admits no augmenting path of length 3.

So our algorithm takes a "lazy" approach: instead of computing a maximum matching, we instead start with a maximal matching (which takes linear time) and iteratively improve it by augmenting along paths of length 3, only as needed to establish the conditions required for correctness. As we will see, the whole algorithm can be implemented so as to run in time $O(n + |E'|)$, plus the time to compute the complement graph, which is time $O(n^2)$. As the algorithm will only be run on dense graphs (which might contain a clique of the required size), this is the best one can do.

Algorithm LazyFindStar(G, t)

Input: Graph $G = (V, E)$ with $n = |V|$ nodes; parameter t with $n - 2t > 0$
Output: An (n, t)-star in G, or FAIL

compute the complement graph $G' = (V, E')$
compute a maximal matching M in G'

let $M_1 \leftarrow M$, $M_2 \leftarrow \varnothing$
while $M_1 \neq \varnothing$ and $|M| \leq t$ do
    remove an edge $m = \{x, y\}$ from M_1
    if there exist distinct unmatched nodes w, z with $\{w, x\}, \{y, z\} \in E'$ then
        add $\{w, x\}$ and $\{y, z\}$ to M_1 //augment
    else
        add $\{x, y\}$ to M_2
    $M \leftarrow M_1 \cup M_2$

if $|M| > t$ then return FAIL

let $C \leftarrow \{u \in V : u$ is unmatched and not a triangle head for any edge in $M\}$
let $D \leftarrow \{v \in V : v$ is not adjacent in G' to any node in $C\}$

if $|C| < n - 2t$ or $|D| < n - t$ then return FAIL
return (C, D)

Remark B.7. *The following invariants are useful for understanding the algorithm. Throughout execution, once a node becomes matched, it remains matched (even though its mate may change during augmentation). From this we can deduce:*

(a) Once an edge is removed from M_1, it is never added back to M_1. This is because any edge added to M_1 by augmentation has at least one endpoint that was previously unmatched.

(b) M_1 and M_2 are disjoint, and $M = M_1 \cup M_2$ is always a maximal matching.

(c) If an edge $\{x, y\}$ is moved to M_2 because it lacks the required augmenting path (i.e., there do not exist distinct unmatched w, z with $\{w, x\}, \{y, z\} \in E'$), then this condition persists for the remainder of the algorithm.

□

Lemma B.8. *The while loop executes at most $2t$ iterations.*

Proof. We may assume M is initially nonempty, as otherwise the loop does not execute. In every iteration, either $|M_2|$ or $|M|$ increases by exactly one: an augmenting step increases $|M|$ by one (and does not change M_2), while a non-augmenting step increases $|M_2|$ by one (and does not change $|M|$).

Since $|M| \geq 1$ initially and the loop exits when $|M| > t$, at most t augmenting steps can occur. Since $|M_2| = 0$ initially and $|M_2| \leq |M| \leq t$ while in the loop, we have $|M_2| \leq t$, so at most t non-augmenting steps can occur. Therefore, the total number of iterations is at most $2t$. □

Theorem B.9. *Algorithm* LazyFindStar *outputs either* FAIL *or an (n, t)-star in G. Moreover, if G contains a clique of size $n - t$, then it does not output* FAIL.

Proof. By Remark B.7(b), M is a maximal matching throughout. In particular, the conclusion of Lemma B.4 holds throughout.

From this, by the same argument as in the proof of Theorem B.6, the sets C and D computed satisfy the inclusion and connectivity requirements of a star. This means that if the algorithm does not output FAIL, it outputs an (n, t)-star in G.

Assume G contains a clique of size $n - t$. It remains to show the algorithm does not output FAIL. We have already shown above that the loop terminates after at most $2t$ iterations. Claim 1 from Theorem B.6 holds here as well (it holds for any matching), and so the loop must terminate with $M_1 = \emptyset$ and $|M| \leq t$. At this point, M is a maximal matching (as we remarked above), and by Remark B.7(c), M admits no augmenting path of length 3. This is all that we needed to prove Claims 2–3 from Theorem B.6. □

B.5 Implementation details

We now sketch an $O(n^2)$-time implementation of LazyFindStar. The $O(n^2)$ time is dominated by computing the complement graph G'. Beyond that, the algorithm runs in $O(n + |E'|)$ time.

We assume G' is represented in adjacency list form, with $adj[v]$ denoting the adjacency list of node v and $deg[v]$ denoting the length of that list. Note that each adjacency list is a simple array, not a linked list; indeed, the entire data structure uses only simple arrays, which makes it very cache-friendly. Additionally, we maintain the following data structures:

- a boolean array $matched[v]$, indicating whether v is currently matched;
- for each node v, an index $next[v]$ into $adj[v]$.

The key invariant is that all nodes in $adj[v][0], \ldots, adj[v][next[v] - 1]$ are matched. Thus, to find an unmatched neighbor of v, we scan forward from position $next[v]$, advancing $next[v]$ past any matched nodes we encounter.

These data structures can be used to build the initial maximal matching in the obvious way: scan through nodes, and for each unmatched node v, use the above procedure to find an unmatched

neighbor u of v; if found, add $\{u, v\}$ to the matching. We then continue using the same data structures for the remainder of the algorithm.

Using these data structures, we can efficiently test whether a matched edge $\{x, y\}$ admits an augmenting path of the form (w, x, y, z) with $w \neq z$:

> advance $next[x]$ to the first unmatched node w, if any
> advance $next[y]$ to the first unmatched node z, if any
> if w or z were not found, return none
> if $w \neq z$, return (w, z)
> //$w = z$; must search further
> $i \leftarrow next[x]$, $j \leftarrow next[y]$, increment $next[x]$ and $next[y]$
> advance $next[x]$ to the first unmatched node w', if any; if found return (w', z)
> advance $next[y]$ to the first unmatched node z', if any; if found return (w, z')
> //no augmenting path; store w at the end of both adjacency lists
> swap elements i and $deg[x] - 1$ of $adj[x]$, set $next[x] \leftarrow deg[x] - 1$
> swap elements j and $deg[y] - 1$ of $adj[y]$, set $next[y] \leftarrow deg[y] - 1$
> return none

When an augmenting path (w, z) is found, we set $matched[w] \leftarrow$ true and $matched[z] \leftarrow$ true.

The key observations for the secondary scans are as follows:

- If we find w' in the secondary scan, all nodes at positions $i + 1, \ldots, next[x] - 1$ of $adj[x]$ are matched, and the node at position i of $adj[x]$ (namely, w) will become matched when we augment.
- If we do not find w' in the secondary scan, but do find z', then all nodes at positions $i + 1, \ldots, next[x] - 1$ of $adj[x]$ are matched, as are all nodes at positions $j + 1, \ldots, next[y] - 1$ of $adj[y]$. The node at position i of $adj[x]$ (namely, w) will become matched when we augment, as will the node at position j of $adj[y]$ (namely, z).
- If neither secondary scan succeeds, all neighbors of both x and y are matched except $w = z$. We record this by storing w at the end of both adjacency lists and update both *next* pointers to point there. We do this by swapping to maintain the stated invariant (but we could have also just copied w into these positions).

For the running time, observe that each $next[v]$ pointer only increases, except for the adjustments made when the secondary scan fails. Each such adjustment decreases $next[v]$ from $deg[v]$ back to $deg[v] - 1$. By Lemma B.8, the procedure for testing for an augmenting path is performed $O(n)$ times. Thus, the total work is $O(n + \sum_v \deg(v))$, which is $O(n + |E'|)$.

B.6 ILP-based optimization

The combinatorial algorithm of the previous subsection runs in $O(n^2)$ time and guarantees correctness when a sufficiently large clique exists. However, the sets C and D it produces satisfy only the minimum size bounds $|C| \geq n - 2t$ and $|D| \geq n - t$. In applications where larger sets are desirable—for instance, to maximize the yield in batch randomness extraction—we can formulate the problem as an integer linear program (ILP) that maximizes $|C|$ while maintaining the required structure.

B.6.1 Problem formulation

Recall that the star-finding algorithm produces an (n, t)-star (C, D) in a graph $G = (V, E)$, where $C \subseteq D \subseteq V$ and every node in C is adjacent to every node in D. For our application, the constraint $C \subseteq D$ is not actually required—what matters is:

1. $|C| \geq n - 2t$ and $|D| \geq n - t$, and
2. for all $u \in C$ and $v \in D$, the nodes u and v are adjacent in G.

Relaxing the $C \subseteq D$ constraint gives the ILP additional flexibility, potentially allowing larger solution sets. Our goal is to find sets C and D satisfying these constraints while maximizing $|C|$.

B.6.2 ILP formulation

We introduce binary decision variables:

- $c_v \in \{0, 1\}$ for each $v \in V$: indicates whether $v \in C$
- $d_v \in \{0, 1\}$ for each $v \in V$: indicates whether $v \in D$

The ILP is:

$$\text{maximize} \quad \sum_{v \in V} c_v \tag{28}$$

$$\text{subject to} \quad \sum_{v \in V} c_v \geq n - 2t \tag{29}$$

$$\sum_{v \in V} d_v \geq n - t \tag{30}$$

$$c_u + d_v \leq 1 \qquad \forall (u, v) \notin E,\ u \neq v \tag{31}$$

Constraint (29) enforces the minimum size of C, constraint (30) enforces the minimum size of D, and constraint (31) ensures that every node in C is adjacent to every node in D: if nodes u and v are not adjacent, then $u \notin C$ or $v \notin D$.

Degree-based pruning. As an optional preprocessing step, one can apply a simple pruning procedure before invoking the ILP solver that eliminates nodes which cannot belong to any feasible solution. The key observation is that every node in C must be adjacent to every node in D. Thus, if a node v has fewer than $|D| - 1$ neighbors among the current D-candidates (accounting for the case where v itself might be in both C and D), then v cannot be in C. Symmetrically, if a node u has fewer than $|C| - 1$ neighbors among the current C-candidates, then u cannot be in D.

These pruning rules are applied iteratively until no further eliminations occur. If at any point the candidate sets shrink below the required thresholds ($|C| \geq n - 2t$ and $|D| \geq n - t$), we can immediately conclude that no feasible solution exists, bypassing the ILP entirely. When pruning succeeds in reducing the candidate sets, the resulting smaller ILP instance solves faster due to fewer variables and constraints. In our experiments (§B.6.4), pruning was not applied; the star finder's warm start proved sufficient for fast ILP convergence.

B.6.3 Cascade strategy

Although the ILP formulation is NP-hard in general, we have observed that in our application context, a cascade of increasingly expensive techniques provides excellent performance. Our approach proceeds in three phases:

Phase 1: Star finder. We first run the $O(n^2)$ star-finding algorithm. If this algorithm fails to find an (n, t)-star, the graph provably contains no clique of size $n - t$ or larger, and we terminate. When it succeeds, it provides sets C_0, D_0 with $|C_0| \geq n - 2t$ and $|D_0| \geq n - t$.

Phase 2: Feasibility ILP. If the star finder succeeds, we run a *feasibility* ILP that seeks any solution with $|C| \geq n - t$ and $|D| \geq n - t$. This is formulated by strengthening constraint (29) to $\sum_v c_v \geq n - t$ and removing the objective function. We provide the star finder's solution (C_0, D_0) as a warm start. Optionally, degree-based pruning can also be applied (with thresholds $|C| \geq n - t$ and $|D| \geq n - t$).

The key observation is that feasibility ILPs are significantly faster than optimization ILPs: they can terminate as soon as any valid solution is found, whereas optimization must explore the search space to prove optimality.

Phase 3: Optimization fallback. If the feasibility ILP fails (no solution with $|C| \geq n - t$ exists), we fall back to the full optimization ILP (28)–(31), again using the star finder's solution as a warm start. This finds the best achievable $|C|$ even when $|C| = n - t$ is not attainable.

B.6.4 Experimental results

Simulation model. We evaluate the cascade approach using a simulation of edge arrivals over time, modeling the gradual accumulation of edges in the communication graph during protocol execution.

Each edge arrival time follows a shifted gamma distribution: $T = d + Gamma(k, \theta)$, where d is a minimum delay (the physical propagation floor), $k = 1/CV^2$ is the shape parameter, and $\theta = (\mu - d)/k$ is the scale parameter. The distribution is parameterized by three quantities: the minimum delay d, the mean delay μ, and the coefficient of variation CV. This family of distributions is supported by empirical studies of Internet latency: Hooghiemstra and Van Mieghem [HM01] found that over 80% of end-to-end delay distributions measured on RIPE NCC paths are gamma-shaped; see also [WAK+24] for a recent study fitting gamma-based models to measured network delays.

We use a "GlobalWAN" preset with $d = 60\,\text{ms}$, $\mu = 200\,\text{ms}$, and $CV = 0.7$ (shape $k \approx 2.04$), which is calibrated for intercontinental round-trip times. Edges between two honest parties arrive at this base rate. Edges involving a corrupt party are slowed by a factor of 5, modeling corrupt parties that are unresponsive or deliberately delaying. Edges between two corrupt parties never form.

Caveat on absolute timing. In the actual protocol, an "edge" between parties i and j represents the event that both parties have exchanged and validated attestations—a process involving multiple message exchanges and cryptographic verification. The shifted gamma model captures the qualitative shape of the delay distribution (a hard minimum followed by a skewed tail), but absolute timing values should not be taken literally as predictions for a specific deployment. To

Table 3 Simulation results for $n = 65$, $t = 16$, $h = 49$ (worst case: t corruptions). Target: $|C| \geq 49$.

τ/μ	e	star%	star $\lvert C\rvert$	feas%	max $\overline{\lvert C\rvert}$	min $\lvert C\rvert$
2.75	110	0	—	—	—	—
3.00	114	2	33.0	100	49.0	49
3.25	117	14	33.0	86	48.7	47
3.50	121	28	33.0	86	48.7	47
3.75	124	56	33.0	89	48.8	47
4.00	128	82	33.0	98	49.0	47
4.25	131	86	33.0	100	49.0	49
4.50	134	96	33.0	100	49.0	49
4.75	137	100	33.0	100	49.0	49

τ/μ: elapsed time τ in units of mean delay μ. e: total number of edges in the graph (including honest-to-corrupt edges), expressed as a percentage of $\binom{h}{2}$; thus $e = 100$ does not imply that the honest clique is complete, only that the total edge count equals $\binom{h}{2}$. star%: percentage of trials where star finder succeeded. star $|C|$: average $|C|$ from star finder (among successful trials). feas%: of star-successful trials, percentage where feasibility ILP achieved $|C| \geq n - t$. max $\overline{|C|}$, min $|C|$: average and minimum $|C|$ from the maximize ILP (among star-successful trials; the maximize ILP always succeeds when the star finder does).

aid portability across network settings, we index the tables below by the dimensionless ratio τ/μ, where τ is the elapsed time and μ is the mean delay, rather than by absolute time. We also report the column e, defined as the total number of edges in the graph (including honest-to-corrupt edges) expressed as a percentage of $\binom{h}{2}$, where h is the number of honest parties. This serves as a measure of graph density that is independent of the delay model.

The simulation runs 50 independent trials for each parameter setting. At each snapshot, the star finder and both ILP variants (feasibility and maximize) are run on the current graph. The ILP is solved using Gurobi 13.0 [Gur24].

Results for $n = 65$. Tables 3 and 4 show results for $n = 65$, $t = 16$ in two scenarios: the worst case $h = n - t = 49$ (maximum corruptions) and an optimistic case $h = 57$ (only 8 corruptions). Each table begins at the last snapshot where the star finder has not yet succeeded and continues through the first snapshot where the star finder succeeds in all trials.

Results for $n = 101$. We also ran simulations for $n = 101$, $t = 25$ in both the worst case ($h = 76$) and an optimistic case ($h = 88$). The qualitative behavior matches $n = 65$: the star finder exhibits a sharp transition (from 0% to 100% success over the range $\tau/\mu \approx 3.5$ to 5.5 for $h = 76$, and $\tau/\mu \approx 2.75$ to 3.25 for $h = 88$), and the feasibility ILP succeeds in nearly all star-successful trials (reaching 100% shortly after the star finder does). The star finder output is $|C| = 51 = n - 2t$ throughout the transition for $h = 76$, while the maximize ILP achieves $|C|$ near $n - t = 76$. For $h = 88$, the maximize ILP reaches $|C|$ near $h = 88$, well above the $n - t = 76$ threshold.

ILP running times. Table 5 summarizes the aggregate ILP running times across all four configurations. Times are aggregated over all snapshots where the respective ILP was invoked (and do

Table 4 Simulation results for $n = 65$, $t = 16$, $h = 57$ (optimistic: 8 corruptions). Target: $|C| \geq 49$.

τ/μ	e	star%	star $\lvert C\rvert$	feas%	max $\overline{\lvert C\rvert}$	min $\lvert C\rvert$
2.25	100	0	—	—	—	—
2.50	102	2	33.0	100	49.0	49
2.75	104	64	36.2	100	52.0	49
3.00	106	96	40.0	100	54.9	49
3.25	107	100	43.6	100	56.5	49
3.50	109	100	45.8	100	56.9	55
3.75	110	100	47.4	100	57.0	57
4.75	116	100	49.0	100	57.0	57

Columns as in Table 3. Rows below the rule show star $|C|$ converging to $n - t = 49$ as the honest clique completes, confirming that the star finder alone suffices when $h \geq n - t/2$ (see text). The maximize ILP simultaneously converges to $|C| = h = 57$.

Table 5 Aggregate ILP running times (ms) over all snapshots and trials.

		feasibility		maximize	
n	h	mean	max	mean	max
65	49	0.3	1.6	1.4	3.1
65	57	0.2	1.9	0.9	1.6
101	76	0.8	8.6	3.8	10.1
101	88	0.5	5.7	2.3	7.3

not include the star finder itself, which runs in microseconds). Even for $n = 101$, the maximum observed feasibility time is under 9 ms and the maximum maximize time is about 10 ms—well within reasonable latency budgets.

Observations. *ILP provides immediate improvement.* The central finding is that once the star finder succeeds, the ILP achieves a substantially larger center set C with negligible additional computation. In the worst case ($h = n - t$), the star finder returns $|C| = n - 2t = 33$, while the maximize ILP immediately achieves $|C|$ at or near $n - t = 49$—a gap of 16. This improvement comes "for free" from the same graph snapshot; no additional edges are needed.

Transition window. The star finder exhibits a sharp threshold: it transitions from 0% to 100% success over a window of roughly 2μ in elapsed time. During this window, the feasibility ILP occasionally fails (e.g., 86% success at $\tau/\mu = 3.25$ for $h = 49$), but the maximize ILP still finds solutions with $|C|$ only slightly below $n - t$ (minimum 47 in our experiments, just 2 below target). The transition window narrows as h increases: for $h = 57$, it spans less than μ.

Star finder threshold and the role of the ILP. The star finder returns $|C| = n - 2 \cdot match$, where *match* is the size of a maximal matching in the complement graph. When all h honest parties form a clique, $match = n - h$ and $|C| = 2h - n$. The star finder alone achieves $|C| \geq n - t$ precisely when $h \geq n - t/2$. For $n = 65$, $t = 16$, this threshold is $h = 57$—exactly the optimistic scenario in Table 4, where indeed star $|C|$ approaches 49 as the honest clique completes. Below this threshold (i.e., when more than $t/2$ parties are corrupt), the ILP is essential for closing the gap between the

star finder's $|C| = n - 2t$ and the target $|C| = n - t$.

Value of the maximize ILP. Even when the feasibility ILP succeeds, the maximize ILP can find a strictly larger center set. This is most pronounced when $h > n - t$: in Table 4, the feasibility ILP returns $|C| = 49 = n - t$ (its threshold) while the maximize ILP achieves $|C|$ up to $57 = h$, a difference of 8 nodes. Since the performance of the downstream protocol improves with larger $|C|$ (for instance, larger $|C|$ increases the yield in batch randomness extraction), running the maximize ILP after feasibility succeeds may be worthwhile despite the modest additional cost (see Table 5). Whether to do so is an application-level decision that depends on the value of additional nodes in C relative to the latency budget.

The star finder as an effective filter. In our target application, edges in G accumulate over time as information is gathered, and we know that eventually a clique of size at least $n-t$ will form. The algorithm runs periodically (say, roughly every 10 ms) to check whether sufficient structure has emerged. In this setting, the star finder serves as an effective filter: when it succeeds, we know the graph contains robust structure and can proceed to the ILP with high confidence.

The cascade strategy provides multiple layers of protection:

1. The star finder runs in $O(n^2)$ time and filters out graphs lacking sufficient structure.
2. The feasibility ILP quickly confirms whether $|C| \geq n - t$ is achievable, succeeding in the common case.
3. The optimization fallback handles edge cases where the star finder succeeds but the full target is not achievable.

As an additional safeguard, the ILP solver can be configured with a time limit (e.g., 10 ms); if the limit is reached, the solver returns its best solution found so far, which is at least as good as the star finder's warm-start solution. This guarantees bounded response time even on adversarially constructed inputs.

We found experimentally that running either the feasibility or optimization ILP unfiltered, that is, without first finding a star (even with degree-based pruning applied), could result in running times of hundreds of milliseconds (on graphs with a few hundred nodes), often without finding any solution.

The star finder thus serves as an effective filter. When it fails (in microseconds), there is usually little point in running an expensive ILP, which would likely either time out or return no solution. Conversely, when it succeeds, the ILP solver will usually (using the feasibility ILP) find a solution with $|C|$ at least $n - t$ or occasionally (using the optimization ILP) not much below $n - t$.

Provoking bad ILP behavior. We also ran experiments that attempted to intentionally provoke poor performance from the ILP solver. For example, we found that we could do so by planting two large overlapping cliques in a static random graph: for $n = 401$, we sometimes saw times of 80 ms to 90 ms for the feasibility ILP (while for $n = 65$ and $n = 101$, we still got times under just a few milliseconds). Nevertheless, we do not think that overlapping-clique configurations will be typical in practice. In addition, as we mentioned above, we can always limit the running time of the ILP solver and fall back to the star finder's solution in these rare cases.

Assessment. The simulation results give us reasonable confidence that the cascade strategy will perform well in practice. The edge-arrival model directly captures the qualitative features of the realistic scenario: honest parties form a clique that emerges over time, corrupt parties' edges may arrive more slowly (or not at all), and the star finder triggers once the honest-party structure is sufficiently complete. That said, the absolute timing values depend on the delay model and on details of the actual edge-formation protocol that are not fully captured by the simulation, and it remains to verify these predictions in a fully operational system.

References

[ANRX21] I. Abraham, K. Nayak, L. Ren, and Z. Xiang. Good-case latency of byzantine broadcast: A complete categorization, 2021. arXiv:2102.07240, http://arxiv.org/abs/2102.07240.

[BDLR24] R. Bacho, S. Das, J. Loss, and L. Ren. Glacius: Threshold Schnorr signatures from DDH with full adaptive security. Cryptology ePrint Archive, Paper 2024, 2024.

[BDLR25] R. Bacho, S. Das, J. Loss, and L. Ren. Adaptively secure three-round threshold Schnorr signatures from DDH. Cryptology ePrint Archive, Paper 2025/1009, 2025. https://eprint.iacr.org/2025/1009.

[BHK+24] F. Benhamouda, S. Halevi, H. Krawczyk, Y. Ma, and T. Rabin. SPRINT: high-throughput robust distributed schnorr signatures. In M. Joye and G. Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part V*, volume 14655 of *Lecture Notes in Computer Science*, pages 62–91. Springer, 2024. https://doi.org/10.1007/978-3-031-58740-5_3. Also at https://eprint.iacr.org/2023/427.

[BJJ+25] A. Bandarupalli, X. Ji, S. Jog, A. Kate, C.-D. Liu-Zhang, and Y. Song. Computationally and communication efficient batched asynchronous DPSS from lightweight cryptography. Cryptology ePrint Archive, Paper 2025/1631, 2025. https://eprint.iacr.org/2025/1631.

[BJK+24] A. Bandarupalli, X. Ji, A. Kate, C.-D. Liu-Zhang, and Y. Song. Computationally efficient asynchronous MPC with linear communication and low additive overhead. Cryptology ePrint Archive, Paper 2024/1666, 2024. https://eprint.iacr.org/2024/1666.

[BJK+25] A. Bandarupalli, X. Ji, A. Kate, C.-D. Liu-Zhang, D. Pöllmann, and Y. Song. Velox: Scalable fair asynchronous MPC from lightweight cryptography. Cryptology ePrint Archive, Paper 2025/1630, 2025. https://eprint.iacr.org/2025/1630.

[BLS01] D. Boneh, B. Lynn, and H. Shacham. Short signatures from the Weil pairing. In C. Boyd, editor, *Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9-13, 2001, Proceedings*, volume 2248 of *Lecture Notes*

in Computer Science, pages 514–532. Springer, 2001. https://doi.org/10.1007/3-540-45682-1_30.

[BLS03] A. Bostan, G. Lecerf, and É. Schost. Tellegen's principle into practice. In J. R. Sendra, editor, *Symbolic and Algebraic Computation, International Symposium ISSAC 2003, Drexel University, Philadelphia, Pennsylvania, USA, August 3-6, 2003, Proceedings*, pages 37–44. ACM, 2003. https://doi.org/10.1145/860854.860870.

[BLSW24] R. Bacho, J. Loss, G. Stern, and B. Wagner. HARTS: High-threshold, adaptively secure, and robust threshold Schnorr signatures. Cryptology ePrint Archive, Paper 2024, 2024.

[BOCG93] M. Ben-Or, R. Canetti, and O. Goldreich. Asynchronous secure computation. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing (STOC)*, pages 52–61. ACM, 1993. https://doi.org/10.1145/167088.167109.

[Bol03] A. Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Y. Desmedt, editor, *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*, volume 2567 of *Lecture Notes in Computer Science*, pages 31–46. Springer, 2003. https://doi.org/10.1007/3-540-36288-6_3.

[BP23] L. T. A. N. Brandão and M. Peralta. NIST call for multi-party threshold schemes. https://csrc.nist.gov/Projects/threshold-cryptography, 2023.

[Bra87] G. Bracha. Asynchronous byzantine agreement protocols. *Inf. Comput.*, 75(2):130–143, 1987. https://doi.org/10.1016/0890-5401(87)90054-X.

[BS23] D. Boneh and V. Shoup. *A Graduate Course in Applied Cryptography (v0.6)*. 2023. Available at https://toc.cryptobook.us/.

[BY01] M. Bellare and B. Yee. Forward-security in private-key cryptography. Cryptology ePrint Archive, Paper 2001/035, 2001. https://eprint.iacr.org/2001/035. Subsequently published in *CT-RSA 2003*.

[Can96] R. Canetti. *Studies in Secure Multiparty Computation and Applications*. PhD thesis, Weizmann Institute of Science, 1996. Available at https://www.wisdom.weizmann.ac.il/~oded/PSX/ran-phd.pdf.

[CDG+18] J. Camenisch, M. Drijvers, T. Gagliardoni, A. Lehmann, and G. Neven. The wonderful world of global random oracles. Cryptology ePrint Archive, Report 2018/165, 2018. https://eprint.iacr.org/2018/165.

[CGG+20] R. Canetti, R. Gennaro, S. Goldfeder, N. Makriyannis, and U. Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In J. Ligatti, X. Ou, J. Katz, and G. Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1769–1787. ACM, 2020. https://doi.org/10.1145/3372297.3423367. Also at https://eprint.iacr.org/2021/060.

[CP17] A. Choudhury and A. Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Trans. Inf. Theory*, 63(1):428–468, 2017. https://doi.org/10.1109/TIT.2016.2614685.

[CS25] E. Crites and A. Stewart. A plausible attack on the adaptive security of threshold Schnorr signatures. Cryptology ePrint Archive, Paper 2025/1001, 2025. https://eprint.iacr.org/2025/1001.

[DDL+24] S. Das, S. Duan, S. Liu, A. Momose, L. Ren, and V. Shoup. Asynchronous consensus without trusted setup or public-key cryptography. Cryptology ePrint Archive, Paper 2024/677, 2024. https://eprint.iacr.org/2024/677. Subsequently published in *ACM CCS* 2024.

[DN07] I. Damgård and J. B. Nielsen. Scalable and unconditionally secure multiparty computation. In A. Menezes, editor, *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 572–590. Springer, 2007. https://doi.org/10.1007/978-3-540-74143-5_32.

[DR23] S. Das and L. Ren. Adaptively secure BLS threshold signatures from DDH and co-CDH. Cryptology ePrint Archive, Paper 2023/1553, 2023. https://eprint.iacr.org/2023/1553.

[DRY25] S. Das, L. Ren, and Z. Yang. Adaptively secure threshold ElGamal decryption from DDH. Cryptology ePrint Archive, Paper 2025/1477, 2025. https://eprint.iacr.org/2025/1477.

[DXR21] S. Das, Z. Xiang, and L. Ren. Asynchronous data dissemination and its applications. In Y. Kim, J. Kim, G. Vigna, and E. Shi, editors, *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, pages 2705–2721. ACM, 2021. https://doi.org/10.1145/3460120.3484808. Also at https://eprint.iacr.org/2021/777.

[GGM86] O. Goldreich, S. Goldwasser, and S. Micali. How to construct random functions. *J. ACM*, 33(4):792–807, 1986. https://doi.org/10.1145/6490.6503.

[GS22] J. Groth and V. Shoup. Design and analysis of a distributed ECDSA signing service. Cryptology ePrint Archive, Paper 2022/506, 2022. https://eprint.iacr.org/2022/506.

[GS23] J. Groth and V. Shoup. Fast batched asynchronous distributed key generation. Cryptology ePrint Archive, Paper 2023/1175, 2023. https://eprint.iacr.org/2023/1175. Subsequently published in *Eurocrypt 2024*.

[Gur24] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2024. https://www.gurobi.com.

[HM01] G. Hooghiemstra and P. V. Mieghem. Delay distributions on fixed internet paths. Technical Report 20011020, Delft University of Technology, 2001. https://www.nas.ewi.tudelft.nl/people/Piet/papers/e2eDelayRipe_IEEE.pdf.

[KG20a] C. Komlo and I. Goldberg. FROST: flexible round-optimized Schnorr threshold signatures. In O. Dunkelman, M. J. J. Jr., and C. O'Flynn, editors, *Selected Areas in Cryptography - SAC 2020 - 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers*, volume 12804 of *Lecture Notes in Computer Science*, pages 34–65. Springer, 2020. https://doi.org/10.1007/978-3-030-81652-0_2. Also at https://eprint.iacr.org/2020/852.

[KG20b] C. Komlo and I. Goldberg. FROST: Flexible round-optimized schnorr threshold signatures. Cryptology ePrint Archive, Paper 2020/852, 2020. https://eprint.iacr.org/2020/852. Subsequently published in *Selected Areas in Cryptography - SAC 2020*, https://doi.org/10.1007/978-3-030-81652-0_2.

[LLTW20] Y. Lu, Z. Lu, Q. Tang, and G. Wang. Dumbo-MVBA: Optimal multi-valued validated asynchronous byzantine agreement, revisited. In Y. Emek and C. Cachin, editors, *PODC '20: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, August 3-7, 2020*, pages 129–138. ACM, 2020. https://doi.org/10.1145/3382734.3405707.

[LS24] T. Locher and V. Shoup. MiniCast: Minimizing the communication complexity of reliable broadcast. Cryptology ePrint Archive, Paper 2024/571, 2024. https://eprint.iacr.org/2024/571. Subsequently published in *Eurocrypt 2024*.

[MV80] S. Micali and V. V. Vazirani. An $O(\sqrt{|V|}\cdot|E|)$ algorithm for finding maximum matching in general graphs. In *21st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 17–27. IEEE, 1980. https://doi.org/10.1109/SFCS.1980.12.

[RRJ+22] T. Ruffing, V. Ronge, E. Jin, J. Schneider-Bensch, and D. Schröder. ROAST: Robust asynchronous schnorr threshold signatures. Cryptology ePrint Archive, Paper 2022/550, 2022. https://eprint.iacr.org/2022/550.

[Sho00] V. Shoup. Practical threshold signatures. In B. Preneel, editor, *Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques, Bruges, Belgium, May 14-18, 2000, Proceeding*, volume 1807 of *Lecture Notes in Computer Science*, pages 207–220. Springer, 2000. https://doi.org/10.1007/3-540-45539-6_15. Also at https://eprint.iacr.org/1999/011.

[Sho23a] V. Shoup. The many faces of Schnorr: a toolkit for the modular design of threshold Schnorr signatures. Cryptology ePrint Archive, Paper 2023/1019, 2023. https://eprint.iacr.org/2023/1019. Subsequently published in *IACR Communications in Cryptology* 2(1), 2025, https://doi.org/10.62056/ahsgvurzn.

[Sho23b] V. Shoup. Sing a song of simplex. Cryptology ePrint Archive, Paper 2023/1916, 2023. https://eprint.iacr.org/2023/1916. Subsequently published in *DISC 2024*, https://doi.org/10.4230/LIPICS.DISC.2024.37.

[Sho24] V. Shoup. A theoretical take on a practical consensus protocol. Cryptology ePrint Archive, Paper 2024/696, 2024. https://eprint.iacr.org/2024/696.

[SS23] V. Shoup and N. P. Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. Cryptology ePrint Archive, Paper 2023/536, 2023. https://eprint.iacr.org/2023/536. Subsequently published in *J. Cryptol.* 37(3), 2024, https://doi.org/10.1007/S00145-024-09505-6.

[SSV25] V. Shoup, J. Sliwinski, and Y. Vonlanthen. Kudzu: Fast and simple high-throughput BFT, 2025. arXiv:2505.08771, http://arxiv.org/abs/2505.08771.

[ST87] T. K. Srikanth and S. Toueg. Simulating authenticated broadcasts to derive simple fault-tolerant algorithms. *Distributed Comput.*, 2(2):80–94, 1987. https://doi.org/10.1007/BF01667080.

[WAK+24] R. Wallace, X. G. Andrade, P. Kayser, Z. Luo, H. Mukherjee, R. Nunes, and M. Warrior. Models of network delay. In *Developments in Statistical Modelling—IWSM 2024*, Contributions to Statistics. Springer, 2024. https://doi.org/10.1007/978-3-031-65723-8_36.

[XLL+25] X. Xing, Y. Liu, B. Liao, J. Liu, B. Hu, X. Lin, Y. Lu, and T. Zhang. GoSSamer: Lightweight and linear-communication asynchronous (dynamic proactive) secret sharing and the applications. Cryptology ePrint Archive, Paper 2025/1516, 2025. https://eprint.iacr.org/2025/1516.

[YPA+21] L. Yang, S. J. Park, M. Alizadeh, S. Kannan, and D. Tse. DispersedLedger: High-throughput byzantine consensus on variable bandwidth networks, 2021. arXiv:2110.04371, http://arxiv.org/abs/2110.04371.